

Spatial Analysis in R for Forest Carbon Verification

A Practical Training in GIS & Carbon Offset Audits

Dr Seamus Murphy

ORCID: 0000-0002-1792-0351

seamusrobertmurphy@gmail.com

2026-08-03

PREFACE	1
TARGET AUDIENCE.....	1
CHAPTER OUTLINE.....	1
INPUT DATASETS.....	2
USER GUIDES	3
1. GETTING STARTED	1
1.1 REPRODUCIBILITY	1
1.2 SOFTWARE STACK	1
1.3 PROJECTS	2
1.4 COURSE DATASET	2
1.5 VECTOR AND RASTER	3
1.6 FIRST MAP	4
1.7 COORDINATE CHECKS	5
1.8 EXERCISES	6
2. VECTOR DATA.....	6
2.1 GEOMETRY TYPES	6
2.2 SF ANATOMY	7
2.3 BUILDING POLYGONS	7
2.4 SPATIAL JOINS	7
2.5 SUPPORT	8
2.6 AREA CHECKS	9
2.7 EXERCISES	10
3. RASTER DATA	11
3.1 RASTER MODEL	11
3.2 INSPECTING RASTERS	12
3.3 RASTER ALGEBRA	12
3.4 RECLASSIFICATION	13
3.5 EXTRACTION	14
3.6 AREA ACCOUNTING.....	15
3.7 EXERCISES	20
4. TERRAIN AND HYDROLOGY.....	20
4.1 ELEVATION MODELS	20
4.2 TERRAIN DERIVATIVES	21
4.3 HYDRAULIC CONDITIONING	23
4.4 FLOW ROUTING	26
4.5 WATERSHED DELINEATION.....	27

44	4.6 STREAM NETWORKS.....	29
45	4.7 INUNDATION MAPPING	32
46	4.8 EXERCISES	36
47	5. LIDAR AND POINT CLOUDS	37
48	5.1 POINT CLOUDS.....	37
49	5.2 GROUND CLASSIFICATION	38
50	5.3 NOISE REMOVAL	39
51	5.4 TERRAIN MODELS.....	39
52	5.5 HEIGHT NORMALISATION	40
53	5.6 CANOPY MODELS	41
54	5.7 TREE DETECTION.....	43
55	5.8 CROWN SEGMENTATION	45
56	5.9 DOMINANT HEIGHT	46
57	5.10 AREA-BASED INVENTORY	47
58	5.11 EXERCISES	48
59	6. DISTURBANCE AND RISK	49
60	6.1 DISTURBANCE SCREENING	49
61	6.2 SPECTRAL INDICES.....	49
62	6.3 THRESHOLD SELECTION	50
63	6.4 COMPOSITE CONSTRUCTION	53
64	6.5 ATTRIBUTION	54
65	6.6 RISK MAPPING.....	55
66	6.7 LEAKAGE BELTS	58
67	6.8 EXERCISES	59
68	7. ALLOMETRY AND BIOMASS	60
69	7.1 INDIRECT MEASUREMENT.....	60
70	7.2 EQUATION FORM	60
71	7.3 EQUATION SELECTION	61
72	7.4 SELECTION COST	61
73	7.5 EXTRAPOLATION RISK.....	63
74	7.6 GENERIC EQUATIONS	64
75	7.7 STAND AGGREGATION.....	65
76	7.8 EXERCISES	66
77	8. EMISSION FACTORS	66
78	8.1 FACTOR DEFINITION	66
79	8.2 TIERS	67
80	8.3 DEFAULT FACTORS	67
81	8.4 FIRE FACTORS	68
82	8.5 STOCK-DIFFERENCE METHOD.....	71
83	8.6 GAIN-LOSS ASYMMETRY	72
84	8.7 UNCERTAINTY PROPAGATION	73
85	8.8 DOMINANT UNCERTAINTY	73
86	8.9 COST OF IMPRECISION	74
87	8.10 EXERCISES	76
88	9. ACTIVITY DATA	76
89	9.1 EXTENT TERM	76
90	9.2 THREE APPROACHES	76
91	9.3 SCENE CLASSIFICATION	77

92	9.4 RARE-CLASS TRAP	78
93	9.5 REWEIGHTING LIMITS	78
94	9.6 PROBABILITY SURFACES	79
95	9.7 SAMPLE ALLOCATION	82
96	9.8 EXERCISES	84
97	10. BASELINE MODELLING	85
98	10.1 COUNTERFACTUAL ACCOUNTING	85
99	10.2 BASELINE VERSUS FORECAST	85
100	10.3 STANDARD REQUIREMENTS	88
101	10.4 RATCHET MECHANISM	89
102	10.5 DISCOUNTING FOR UNCERTAINTY	89
103	10.6 JURISDICTIONAL ALLOCATION	90
104	10.7 EXERCISES	93
105	11. UNCERTAINTY	94
106	11.1 UNCERTAINTY RATIONALE	94
107	11.2 CARBON ESTIMATE	94
108	11.3 SAMPLING ERROR	94
109	11.4 MEASURING ERROR	95
110	11.5 CROSS-VALIDATION	97
111	11.6 MONTE CARLO PROPAGATION	100
112	11.7 CONSERVATIVE ESTIMATES	102
113	11.8 AREA-ADJUSTED ACCURACY	102
114	11.9 REPORTED AGAINST ADJUSTED	103
115	11.10 REGISTRY DEDUCTIONS	105
116	11.11 EXERCISES	109
117	12. TIME SERIES	109
118	12.1 DATA CUBES	109
119	12.2 TWO PLANES	110
120	12.3 SEASONALITY AND RANDOMNESS	113
121	12.4 SAMPLING AND ALIASING	115
122	12.5 ANNUAL COMPOSITING	116
123	12.6 TEMPORAL REDUCTION	116
124	12.7 TRENDS	117
125	12.8 EVENT DETECTION	118
126	12.9 TIER 1 EMISSIONS	118
127	12.10 EXERCISES	120
128	13. SPATIAL PATTERNS	120
129	13.1 POINT PATTERNS	120
130	13.2 SPATIAL RANDOMNESS	121
131	13.3 SPECIES MAPS	122
132	13.4 INTENSITY	123
133	13.5 NEAREST NEIGHBOUR	124
134	13.6 RIPLEY'S K	124
135	13.7 SEEDLING PATTERN	125
136	13.8 PATTERN TO PROCESS	127
137	13.9 COVARIATES	127
138	13.10 SCREENING COVARIATES	129
139	13.11 BANDWIDTH SELECTION	130
140	13.12 CLUSTER PROCESS	131

141	13.13 CORRECTED STANDARD ERRORS	132
142	13.14 RESIDUAL TESTING	133
143	13.15 PUBLISHED COMPARISON	135
144	13.16 EXERCISES	136
145	APPENDIX A — STATISTICAL PRIMERS.....	137
146	A.1 SPREAD AND INTERVALS.....	137
147	A.2 COMBINING UNCERTAINTIES	140
148	A.3 MONTE CARLO METHODS.....	141
149	A.4 APPLICATIONS	142
150	APPENDIX B — REPRODUCIBILITY & VERSION CONTROLS.....	1
151	B.1 BEYOND ANALYSIS	1
152	B.2 REPRODUCIBLE RUNTIMES.....	1
153	B.3 DATA PROTOCOLS	1
154	B.4 CARBON REGISTRIES.....	2
155	B.5 ADDITIONAL RESOURCES	2
156	APPENDIX C — REFERENCES	1
157		

Preface

Carbon projects are audited on numbers that come out of a geographic information system, and very few of the people who produce or check those numbers were trained to build one. The methods literature stops at the discipline boundary: it will teach you to project a raster and never mention what a verifier does with the result. The standards documents start on the far side of it: they will tell you an uncertainty deduction is required and never show you how to compute one. Practitioners reconcile the two in undocumented spreadsheets.

This book is written to close that gap. It teaches the spatial and statistical methods that forest carbon measurement and verification actually rests on, from reading a shapefile to reporting an area-adjusted emission with a defensible interval, entirely in R and entirely reproducibly. It assumes an introductory course in statistics and no prior GIS.

Target audience

The book is written for two audiences at once, and it is honest about which parts belong to whom.

The main text is for anyone who needs the method: graduate students in forestry, environmental science and remote sensing, and analysts moving into carbon work from an adjacent field. It teaches spatial analysis and applied statistics, and it can be read straight through without reference to any crediting standard.

Interleaved with it are verification tracks, set off in tinted boxes. These carry the registry-specific material that only practitioners need: what ART-TREES, Verra and the American Carbon Registry require, where their equations are written down, and how each converts a statistic into forgone credits. They are written for analysts and technical staff at project developers and verification bodies, and for national forest monitoring teams.

A reader who skips every verification track still gets a complete book. Nothing in the main narrative depends on what is inside them. A reader who wants only the compliance material can read the tracks alone, though the methods will be less use without the chapters around them.

Chapter outline

The first six chapters build the spatial toolkit, each teaching a foundational concept and closing on a worked problem from a real verification project.

1. Getting Started: reproducible projects, the spatial stack, vector versus raster, a first map and coordinate check.

- 187 2. Vector Data: geometries, attributes and support, spatial joins; closing on an area check.
188 3. Raster Data: grids, map algebra, reclassification, extraction; closing on land-cover reclassification.
189 4. Terrain and Hydrology: elevation, slope and aspect, flow; closing on inundation mapping.
190 5. LiDAR and Point Clouds: returns, ground and canopy, tree detection; closing on a LiDAR forest inventory.
191 6. Disturbance and Risk: change detection, accuracy assessment, risk modelling; closing on VM0048 risk and
192 ACR accuracy.

193 The next four turn that toolkit on the accounting problem, and are where the book departs from
194 every other R-spatial text.

- 195 7. Allometry and Biomass: equation form and selection, extrapolation, wood density, scaling to the stand.
196 8. Emission Factors: tiers, the stock-difference method, factor selection, propagation in quadrature.
197 9. Activity Data and Classification: IPCC approaches, training design, probability surfaces, active learning.
198 10. Baseline Modelling: counterfactuals, reference periods, the ratchet, risk-weighted allocation.

199 The last three quantify and close.

- 200 11. Uncertainty: sampling error, accuracy metrics, cross-validation, Monte Carlo propagation, area-adjusted
201 accuracy, and how three registries turn one statistic into three different deductions.
202 12. Time Series: space-time cubes, seasonality and randomness, sampling and aliasing; closing on IPCC Tier 1
203 fire emissions.
204 13. Spatial Patterns: point patterns, intensity, nearest-neighbour, Ripley's K; closing on post-fire conifer
205 regeneration.

206 An appendix covers the surrounding discipline: reproducible runtimes, spatial-data protocols, and
207 registry practice.

208 **Input datasets**

209 One plot anchors the book: the Smithsonian ForestGEO census at the Conservation Biology
210 Institute in Front Royal, Virginia, recording 38,517 living stems across 39 genera in a 400 by 640
211 metre footprint, each identified to species and measured for diameter. It carries the vector,
212 allometry, biomass and error chapters.

213 Four further datasets, each real and each committed so the book builds offline, carry the chapters
214 the plot cannot answer.

215 The raster, activity data and disturbance chapters run on a 2022 Sentinel-2 land-cover map of
216 Rondonia, Brazil, with a matching vegetation index composite and a validation sample of 2,032
217 points. Labelled disturbance classes make it possible to measure what an index threshold actually
218 catches rather than assume it.

219 The terrain and hydrology chapter runs on Lake Chilwa in southern Malawi: an elevation model
220 on a 100 metre grid, with the lake, its catchment and its river network from HydroSHEDS. Chilwa
221 is endorheic, so it is a genuine test of watershed delineation rather than a demonstration, and it
222 is the reference case for wetland inundation mapping.

223 The LiDAR chapter runs on airborne laser tiles distributed with the `lidR` package, real surveys over
224 real terrain, so ground classification, terrain interpolation and tree detection are performed rather
225 than described.

226 The spatial patterns chapter runs on the published coordinates of a post-fire conifer regeneration
227 survey in the Selkirk Mountains of British Columbia (Murphy et al., 2026).

228 Published allometric equations and a set of oaks with biomass computed five ways support the
229 allometry chapter, because the cost of choosing an equation cannot be shown without several
230 equations.

Secondary sources

Every dataset above is real and is committed to the repository, and two scripts under `data-raw/` rebuild the derived ones from their sources. Within-plot coordinates for the ForestGEO stems were generated across the real footprint because the source census table omitted them, so those maps are representative rather than a survey of record; this is stated again where they are used.

A small number of illustrative surfaces are constructed rather than observed, in order to isolate one effect at a time. Each says so where it is built, and none carries a reported number.

The keystone closings and verification tracks describe workflows from real verification projects, and reproduce their analytical core on this shared data.

231 **User guides**

232 Each chapter is a self-contained lesson that builds on the last. Code is shown in full and is meant
233 to be run; the fastest way to learn is to work through it line by line and then attempt the exercises.

234 Every figure and every number in the text is generated by code that ships with the book and runs
235 offline. Nothing is quoted from a source that cannot be re-executed, and where a registry
236 document is cited the citation is to the equation and section, so a reader can check it against the
237 original. Any package the book uses can be installed from CRAN in one command.

238 Readers who want a deeper or more general treatment of the spatial methods are well served by
239 the canonical texts in the field (Bivand et al., 2013; Lovelace et al., 2019; Pebesma & Bivand,
240 2023), and the `sf` and `terra` packages that underpin the modern R-spatial stack are described by
241 Pebesma (2018) and Hijmans (2024). Where the applied material touches forest-carbon
242 conventions it follows IPCC (2006) and the 2019 Refinement (IPCC, 2019).

243 **1. Getting Started**

244 **1.1 Reproducibility**

245 An audit stands or falls on whether someone else can reproduce the number. A spreadsheet built
246 with forty manual clicks cannot be re-run, cannot be reviewed line by line, and cannot be trusted
247 a year later when the inputs have changed. A script can. This is the reason to do spatial work in
248 R: every decision, from the projection you chose to the buffer distance you applied, is written
249 down and repeatable.

250 R is a free, open-source language with a mature ecosystem for geographic data. In one session
251 you can read a shapefile a surveyor sent you, pull an elevation model from an open server, overlay
252 the two, extract terrain at your field plots, fit a model, and export a map, without leaving the
253 environment or paying for a licence. For a forest-carbon or agriculture auditor that means the
254 whole chain from raw data to reported estimate lives in one reviewable document. This course
255 uses the modern R-spatial stack, built on two packages that have become the standard tools of
256 the field (Hijmans, 2024; Pebesma, 2018).

257 **1.2 Software stack**

258 You will meet these packages throughout the book. Install them once before you begin.

Package	Role
sf	Vector data: points, lines, polygons
terra	Raster data: imagery, elevation, gridded surfaces
dplyr / tidyverse	Data wrangling and pipes
tmap	Thematic maps, static and interactive
ggplot2	Statistical graphics
gstat	Geostatistics and interpolation
spdep	Spatial autocorrelation and neighbours

259 To install everything the course needs, run this once in the console. It is set not to execute when
260 the book renders.

```
261 install.packages(c(  
262   "sf", "terra", "dplyr", "tidyverse", "tmap",  
263   "ggplot2", "gstat", "spdep", "elevatr", "rnaturalearth"  
264 ))
```

265 Load the core three at the top of any spatial script:

```
266 library(sf)
267 library(terra)
268 library(dplyr)
```

269 The messages `sf` and `terra` print on loading name the versions of GDAL, GEOS and PROJ you
270 are linked against. Note them: a projection or geometry result can occasionally differ between
271 library versions.

272 1.3 Projects

273 Before loading data, put your work in an RStudio Project, a folder with an `.Rproj` marker. It fixes
274 two things that otherwise cause endless confusion: it sets the working directory to the project root,
275 and it lets you refer to data with short relative paths like `data/scbi_stems.csv` instead of a full path
276 tied to your machine. A tidy layout looks like:

```
277 training-rspatial/
278 | data/                # input datasets
279 | 01-getting-started.qmd
280 | 02-vector-data.qmd
281 | training-rspatial.Rproj
```

282 With a project open, every path in this book that begins with `data/` resolves correctly.

283 1.4 Course dataset

284 One dataset runs through the whole course: a mapped forest-dynamics plot, modelled on the
285 Smithsonian ForestGEO plot at the Conservation Biology Institute in Front Royal, Virginia. Every
286 stem in a 400 by 640 metre plot is stem-mapped, identified to species, and measured for diameter.
287 It is an ideal teaching set because it is large, real in structure, and every tree carries a location.

Data provenance

The stem list (species, diameter) follows the published ForestGEO census used in the companion uncertainty book. The within-plot coordinates were generated across the real plot footprint because the source census table omitted them, so the map is representative rather than a survey of record. Terrain, imagery and LiDAR layers in later chapters are simulated over the same footprint for a clean, always-runnable progression, and every synthetic layer says so where it is built.

288 The stem table is an ordinary CSV, so base R reads it without any spatial machinery:

```
289 stems <- read.csv("data/scbi_stems.csv")
290 str(stems)
291 #> 'data.frame':   38517 obs. of  10 variables:
292 #> $ treeID : int  1 1 2 3 3 4 4 5 5 5 ...
293 #> $ stemID : int  1 31194 31195 3 31196 4 31197 5 31198 36479 ...
```

```

294 #> $ dbh : num 2.8 1.6 1.1 2.2 1.5 13.5 12.5 8.7 8 4.4 ...
295 #> $ genus : chr "Lindera" "Lindera" "Lindera" "Lindera" ...
296 #> $ species: chr "benzoin" "benzoin" "benzoin" "benzoin" ...
297 #> $ family : chr "Lauraceae" "Lauraceae" "Lauraceae" "Lauraceae" ...
298 #> $ gx : num 3.7 3.7 17.3 9 9 14.2 14.2 9.4 9.4 9.4 ...
299 #> $ gy : num 73 73 58.9 197.1 197.1 ...
300 #> $ lon : num -78.1 -78.1 -78.1 -78.1 -78.1 ...
301 #> $ lat : num 38.9 38.9 38.9 38.9 38.9 ...

```

Two functions tell you almost everything about a new dataset. `summary()` shows the distribution of each column; a grouped `dplyr` summary reads the forest:

```

304 summary(stems[, c("dbh", "gx", "gy")])
305 #>      dbh      gx      gy
306 #> Min.   : 0.900   Min.   : 0.0   Min.   : 0.0
307 #> 1st Qu.: 1.400   1st Qu.:129.4   1st Qu.: 49.3
308 #> Median : 2.400   Median :228.8   Median :159.6
309 #> Mean   : 8.801   Mean   :217.9   Mean   :216.4
310 #> 3rd Qu.: 8.400   3rd Qu.:314.3   3rd Qu.:373.1
311 #> Max.   :151.100   Max.   :400.0   Max.   :640.0
312
313 stems |>
314   group_by(genus) |>
315   summarise(n = n(), mean_dbh = round(mean(dbh), 1)) |>
316   arrange(desc(n)) |>
317   head(8)

```

genus	n	mean_dbh
Lindera	13875	1.6
Carya	4276	13.0
Asimina	3876	2.1
Carpinus	2927	4.4
Liriodendron	2496	35.9
Quercus	1599	39.0
Nyssa	1438	10.0
Hamamelis	1406	3.1

Diameters run from saplings at 1 cm to canopy trees near a metre, and a handful of genera dominate the stem count. That is the size and composition structure every biomass and carbon calculation later in the book will rest on.

1.5 Vector and raster

Spatial data come in two great families, and knowing which you hold is the first decision in any analysis. Vector data represent discrete objects by their geometry: a tree is a point, a road a line, a stand boundary a polygon, each carrying attributes in a table. Raster data represent a continuous surface as a grid of equal cells, each holding one value: elevation, reflectance, canopy height. Our stems are vector; the elevation and imagery we bring in later are raster. Most real

analysis moves between the two, and the middle chapters of this book are about doing that cleanly.

1.6 First map

The CSV knows nothing about geography yet: `lon` and `lat` are just numbers. `sf` turns them into geometry by naming the coordinate columns and the coordinate reference system, here WGS 84 longitude/latitude, whose EPSG code is 4326:

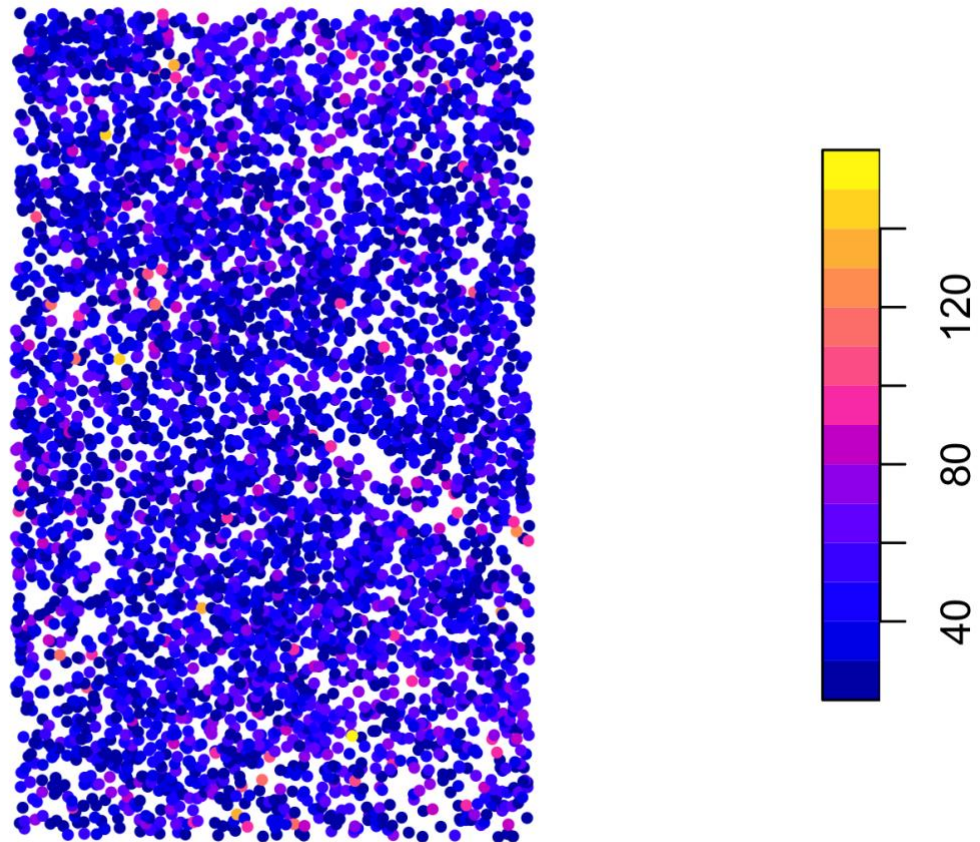
```
stems_sf <- st_as_sf(stems, coords = c("lon", "lat"), crs = 4326)
print(stems_sf, n = 5)
#> Simple feature collection with 38517 features and 8 fields
#> Geometry type: POINT
#> Dimension: XY
#> Bounding box: xmin: -78.1454 ymin: 38.8935 xmax: -78.14078 ymax: 38.89924
#> Geodetic CRS: WGS 84
#> First 5 features:
#>   treeID stemID dbh  genus species  family   gx   gy
#> 1      1      1  2.8 Lindera benzoin Lauraceae  3.7  73.0
#> 2      1  31194  1.6 Lindera benzoin Lauraceae  3.7  73.0
#> 3      2  31195  1.1 Lindera benzoin Lauraceae 17.3  58.9
#> 4      3      3  2.2 Lindera benzoin Lauraceae  9.0 197.1
#> 5      3  31196  1.5 Lindera benzoin Lauraceae  9.0 197.1
#>      geometry
#> 1 POINT (-78.14536 38.89415)
#> 2 POINT (-78.14536 38.89415)
#> 3 POINT (-78.1452 38.89402)
#> 4 POINT (-78.1453 38.89527)
#> 5 POINT (-78.1453 38.89527)
```

Notice what `sf` prints: the geometry type (`POINT`), the bounding box, the CRS (Geodetic CRS: WGS 84), and the attribute table with a `geometry` column on the end. An `sf` object is a data frame that also knows where each row sits on Earth. The plot layer also ships as a shapefile, read the same way with `st_read("data/scbi_stems.shp")`.

Plotting one attribute draws a map coloured by that value. We map the larger stems to keep the picture legible:

```
canopy <- stems_sf[stems_sf$dbh >= 20, ]
plot(canopy["dbh"], pch = 19, cex = 0.5, main = "Stems ≥ 20 cm DBH")
```

Stems ≥ 20 cm DBH



361

362 There is the plot: canopy trees scattered across the footprint, shaded by diameter. From here the
363 course is about doing this properly.

364 1.7 Coordinate checks

365 The first thing to do on any real project is not analysis but a coordinate sanity check. A layer
366 whose CRS is wrong, or missing, will silently misplace every downstream result, and the error is
367 invisible on a plot that looks fine on its own axes. Two questions answer it: does the layer declare
368 a CRS, and does it land where it should on Earth?

```
369 st_crs(stems_sf)$epsg           # 4326: the layer knows its CRS
370 #> [1] 4326
371 st_bbox(stems_sf)              # does the extent sit where you expect?
372 #>      xmin      ymin      xmax      ymax
373 #> -78.14540  38.89350 -78.14078  38.89925
```

374 The bounding box should fall over Front Royal, Virginia (near 78.1° W, 38.9° N). If it landed in the
375 Gulf of Guinea at (0, 0), the coordinates were never georeferenced. This is exactly the opening

move of any mapping template: read the layer, confirm its CRS, confirm its extent, and only then begin. Every keystone project in this book starts here, because a defensible result begins with knowing your data are where they claim to be.

1.8 Exercises

1. Read `scbi_stems.csv` and report the number of stems, the number of distinct genera, and the mean and maximum diameter. Which genus has the most stems?
2. Convert the table to an `sf` object with `st_as_sf()`, then confirm its CRS with `st_crs()` and its extent with `st_bbox()`. In one sentence, why is checking the CRS the first thing you should do?
3. Map the stems coloured by genus for the three most common genera only. Do they appear mixed or spatially segregated?
4. Add a `basal_area` column (the cross-sectional area of each stem, $\pi * (dbh/2)^2$ in cm^2) and report total basal area by genus. Which genus dominates the stand by basal area rather than by stem count?

2. Vector Data

2.1 Geometry types

Vector data represent the world as discrete shapes. The simple-features standard, which `sf` implements, defines a small family of geometry types built from coordinates: a point is a single location, a linestring an ordered path, a polygon an enclosed area, and each has a *multi* form for features made of several parts. Almost everything an auditor handles is one of these: stems and plot centres are points, streams and roads are lines, stands and ownership parcels are polygons.

```
library(sf)
library(dplyr)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
                  coords = c("lon", "lat"), crs = 4326)
st_geometry_type(stems)[1]
#> [1] POINT
#> 18 Levels: GEOMETRY POINT LINESTRING POLYGON MULTIPOINT ... TRIANGLE
```

Every metric operation, area, distance, overlap, is meaningless on longitude and latitude, because a degree is not a fixed length. So the first move in vector analysis is to project to a coordinate system whose units are metres. For this plot in Virginia that is UTM zone 17 North, EPSG 32617:

```
stems <- st_transform(stems, 32617)
st_bbox(stems) # now in metres
#>      xmin      ymin      xmax      ymax
#> 747556.9 4308832.8 747977.3 4309482.2
```

2.2 sf anatomy

An `sf` object is a table whose rows are features and whose last column, of class `sfc`, holds the geometry. Because it is a data frame first and spatial second, the whole `dplyr` grammar applies and the geometry is carried along automatically:

```
stems |>
  filter(dbh >= 30) |>
  mutate(basal_area_cm2 = pi * (dbh / 2)^2) |>
  select(treeID, genus, dbh, basal_area_cm2) |>
  head()
```

treeID	genus	dbh	basal_area_cm2	geometry
29	Acer	32.6	834.6898	POINT (747606.9 4308879)
77	Quercus	63.6	3176.9042	POINT (747602 4309139)
79	Tilia	47.5	1772.0546	POINT (747604.9 4309213)
79	Tilia	31.6	784.2672	POINT (747604.9 4309213)
95	Quercus	52.0	2123.7166	POINT (747614.4 4309128)
95	Quercus	34.7	945.6901	POINT (747614.4 4309128)

`st_drop_geometry()` turns the layer back into an ordinary table when you want a plain summary, and `st_geometry()` extracts just the shapes when you want to plot outlines with no attribute shading.

2.3 Building polygons

Analysis usually needs areas, not just points. We divide the plot into 100-metre quadrats with `st_make_grid()`, the standard way to impose a regular sampling frame, and keep only the cells that cover the stems:

```
grid <- st_make_grid(stems, cellsize = 100) |> st_as_sf()
grid$quadrat <- seq_len(nrow(grid))
grid <- grid[lengths(st_intersects(grid, stems)) > 0, ]
nrow(grid)
#> [1] 35
```

2.4 Spatial joins

A spatial join attaches attributes by location rather than by a shared key. `st_join()` labels each stem with the quadrat that contains it, after which the counting is ordinary `dplyr`:

```
stems_q <- st_join(stems, grid, join = st_within)

per_quadrat <- stems_q |>
  st_drop_geometry() |>
  group_by(quadrat) |>
  summarise(n_stems = n(),
            mean_dbh = round(mean(dbh), 1))
head(per_quadrat)
```

quadrat	n_stems	mean_dbh
1	2031	4.9
2	1839	7.6
3	5107	3.2
4	5213	2.8
5	693	4.1
6	1135	8.7

441 This point-in-polygon join is the workhorse of inventory: it is how you assign field stems to strata,
 442 plots to ownership parcels, or detections to management units.

443 2.5 Support

444 An attribute value stands in a particular relationship to the geometry it is attached to, and getting
 445 that relationship wrong is one of the most common and least visible errors in spatial analysis. The
 446 support of an attribute is the area over which it was measured or to which it applies. A stem count
 447 has each quadrat as its support; a per-hectare density refers to a hectare. Two numbers can look
 448 identical yet mean different things because their support differs.

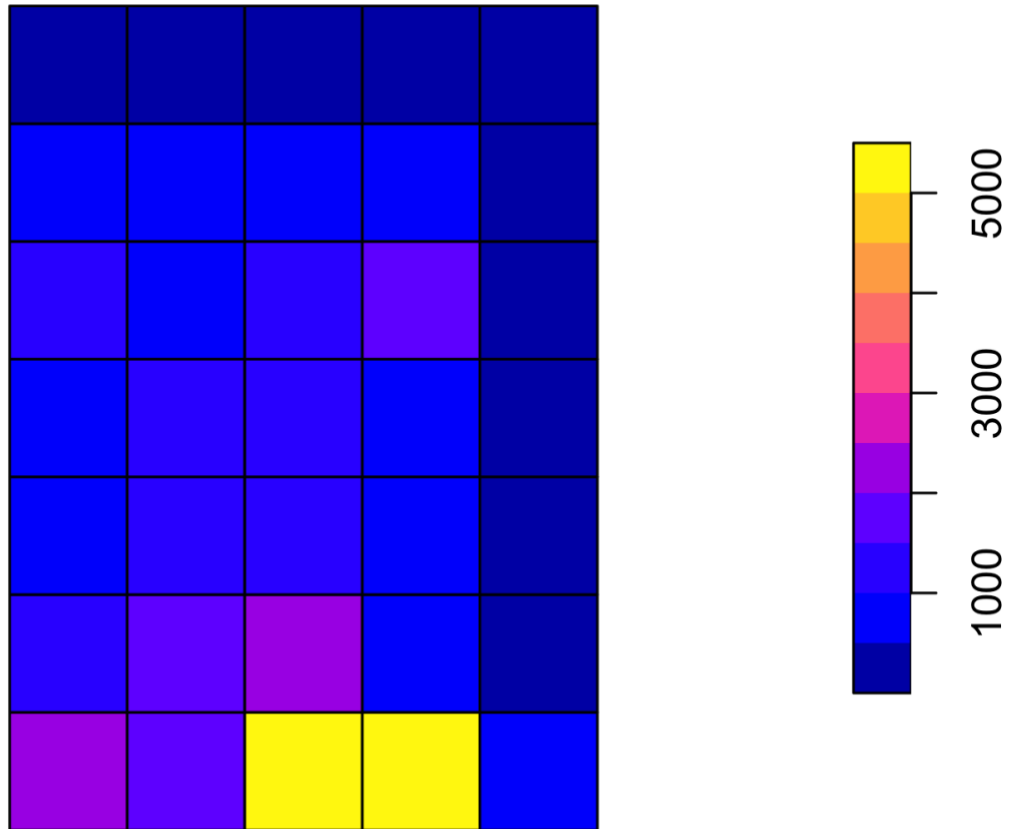
449 Attributes divide into two kinds by how they behave when a geometry changes size. An extensive
 450 attribute is a total that scales with area, a count of stems or a tonnage of carbon; split a polygon
 451 and the value divides between the pieces. An intensive attribute is a rate or density that does not
 452 scale, stems per hectare or mean diameter; split the polygon and each piece keeps the value.
 453 The distinction dictates how you aggregate: sum the extensive total, average the intensive rate.

```
454 grid_join <- left_join(grid, per_quadrat, by = "quadrat")
455 grid_join$area_ha <- as.numeric(st_area(grid_join)) / 1e4      # support, in ha
456 grid_join$stems_ha <- grid_join$n_stems / grid_join$area_ha    # intensive
457
458 # extensive -> sum; intensive -> area-weighted mean
459 c(total_stems = sum(grid_join$n_stems),
460   mean_density = round(weighted.mean(grid_join$stems_ha, grid_join$area_ha), 0))
461 #> total_stems mean_density
462 #>      38515      1100
```

463 Summing a density, or averaging a total, is simply wrong, and it is exactly how reported area totals
 464 silently inflate. Always be able to state the support of a number, and never combine values of
 465 different support without an explicit rule.

```
466 plot(grid_join["stems_ha"], main = "Stem density (stems/ha)")
```

Stem density (stems/ha)



467

468 2.6 Area checks

469 The applied heart of vector GIS in verification work is the area check: confirming that the area a
 470 project *claims* matches the area its geometry actually encloses, and that its parcels do not overlap
 471 or leave gaps. Overstated area is overstated crediting, so this is often the first quantitative finding
 472 in an audit. The tools are exactly those above, `st_area`, `st_is_valid`, `st_intersection`, applied to the
 473 project's own polygons.

474 We stand in for the project boundary with the plot's nominal extent and check it against the
 475 mapped stems. First, geometry validity, because area and overlap operations on an invalid
 476 polygon return nonsense:

```
477 boundary <- st_as_sf(st_bbox(stems)) |> st_sf(id = "claimed_boundary")
478 all(st_is_valid(boundary))
479 #> [1] TRUE
```

Now the area reconciliation. A claimed area is compared with the geometry's measured area; the auditor reports the difference. Measure it two ways, once from the bounding box and once from the convex hull of the stems:

```
claimed_ha <- 25.6 # nominal 400 x 640 m plot
bbox_ha <- as.numeric(st_area(st_as_sfc(st_bbox(stems)))) / 1e4
hull_ha <- as.numeric(st_area(st_convex_hull(st_union(stems)))) / 1e4

data.frame(
  geometry      = c("bounding box", "convex hull"),
  measured_ha   = round(c(bbox_ha, hull_ha), 2),
  difference_ha = round(c(bbox_ha, hull_ha) - claimed_ha, 2),
  pct_error     = round(100 * (c(bbox_ha, hull_ha) - claimed_ha) / claimed_ha, 1)
)
```

geometry	measured_ha	difference_ha	pct_error
bounding box	27.30	1.70	6.6
convex hull	25.56	-0.04	-0.2

The two disagree, and the disagreement is the lesson. The plot grid runs about 1.8 degrees off UTM north, and a bounding box is axis-aligned by construction, so it wraps a tilted rectangle inside a larger upright one. The convex hull follows the stems and recovers the nominal area almost exactly. Neither figure reflects a survey error. When a claimed area and a mapped area disagree, the first question is not whether the survey is wrong but whether the thing being measured is the boundary or a box drawn around it, because that choice alone moved the answer by more than a hectare.

Finally, overlap detection between parcels. Where two subareas are submitted, their intersection must be empty; any positive intersection area is double-counted land:

```
e <- st_bbox(stems) # two parcels on the plot
x0 <- unname(e["xmin"]); y0 <- unname(e["ymin"])
a <- st_as_sfc(st_bbox(c(xmin = x0, xmax = x0 + 250,
                        ymin = y0, ymax = y0 + 400), crs = 32617))
b <- st_as_sfc(st_bbox(c(xmin = x0 + 200, xmax = x0 + 450,
                        ymin = y0, ymax = y0 + 400), crs = 32617))
overlap <- st_intersection(a, b)
as.numeric(st_area(overlap)) / 1e4 # hectares of double-counted area
#> [1] 2
```

A non-zero result is a finding: those hectares are claimed twice and must be reconciled before any area total is accepted. The full project workflow layers on shapefile ingest, coordinate-system checks, and a written report, but the logic is this: validate the geometry, measure the area, compare to the claim, and test every parcel pair for overlap.

2.7 Exercises

1. Project the stems to UTM 17N and build a 50-metre quadrat grid. How many quadrats cover the plot, and how does the mean stem count per quadrat change from the 100-metre version? Relate this to support.

2. Classify `n_stems` and `stems_ha` as extensive or intensive, and state which should be summed and which averaged when quadrats are merged into blocks.
3. Take the `claimed_ha` of 25.6 and imagine the true mapped boundary were 5% smaller. Compute the over-crediting in hectares and explain why an auditor treats positive area error differently from negative.
4. (*Applied.*) Two submitted parcels overlap by 3 ha out of a claimed 40 ha total. Write the `st_intersection` and `st_area` calls that quantify the overlap, and state the corrected non-overlapping total.

3. Raster Data

3.1 Raster model

A raster represents a surface as a grid of equal cells, each holding one value. Where vector data draw discrete objects, rasters describe things that vary continuously across space, elevation, reflectance, temperature, canopy height, and the things we derive from imagery: land cover, disturbance, biomass. The data model is nothing more than a matrix plus the georeferencing that ties cell (1,1) to a place and fixes the cell size. `terra` is the modern package for it (Hijmans, 2024).

We build a raster over the study plot by summarising the stems onto a grid. First project the stems and give each a basal area, then lay down a 10-metre grid and total the basal area falling in each cell:

```
library(terra)
library(sf)

stems_sf <- st_as_sf(read.csv("data/scbi_stems.csv"),
                     coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
stems_sf$ba <- pi * (stems_sf$dbh / 2)^2 / 1e4 # basal area, m² per stem

stems <- vect(stems_sf)
r <- rast(ext(stems), resolution = 10, crs = "EPSG:32617")
ba <- rasterize(stems, r, field = "ba", fun = sum, background = 0)
names(ba) <- "basal_area"
ba
#> class      : SpatRaster
#> size       : 65, 42, 1 (nrow, ncol, nlyr)
#> resolution : 10.00752, 9.990686 (x, y)
#> extent     : 747556.9, 747977.3, 4308833, 4309482 (xmin, xmax, ymin, ymax)
#> coord. ref.: WGS 84 / UTM zone 17N (EPSG:32617)
#> source(s)  : memory
#> name       : basal_area
#> min value  : 0.000000
#> max value  : 2.175199
```

The printout is the raster data model made concrete: rows and columns, cell size, extent, CRS, and the value range. Each 10-metre cell now holds the total stem basal area within it, a crude but genuine biomass proxy.

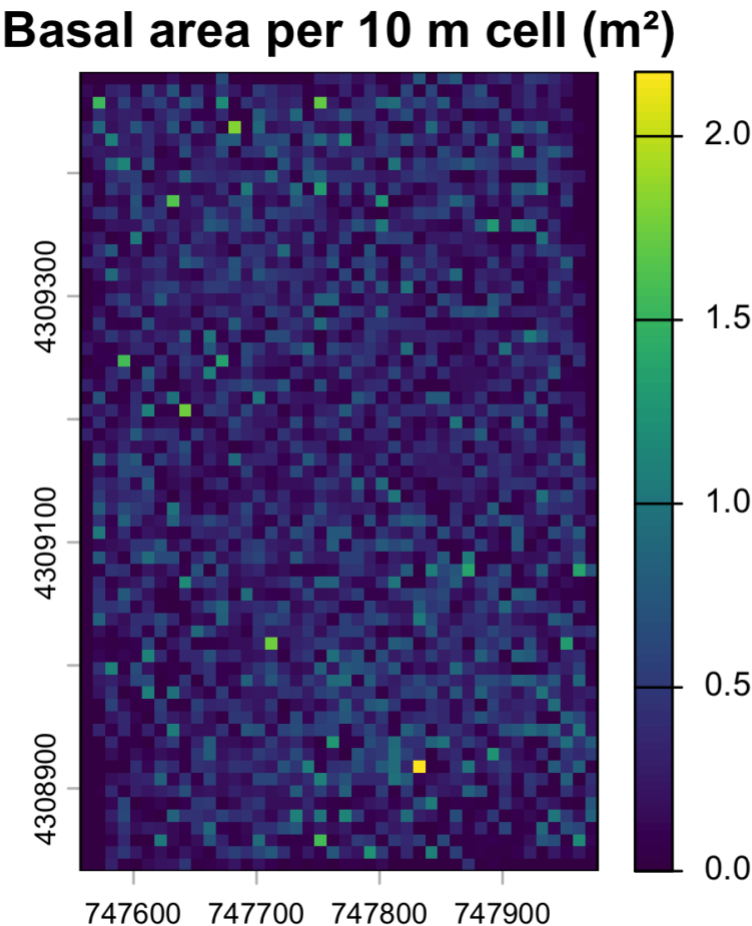
3.2 Inspecting rasters

The same handful of accessors describe any raster:

```
res(ba) # cell size, metres
#> [1] 10.007519 9.990686
ext(ba) # bounding box
#> SpatExtent : 747556.94226603, 747977.258068353, 4308832.82917391, 4309482.22379355 (xmin, xmax, ymin, ymax)
global(ba, c("min", "max", "mean")) # value range
```

	min	max	mean
basal_area	0	2.1752	0.3196863

```
plot(ba, main = "Basal area per 10 m cell (m²)")
```



3.3 Raster algebra

Because a raster is a grid of numbers, arithmetic on rasters is arithmetic on cells. This is map algebra, and it is why continuous surfaces are so easy to work with. Converting basal area to an aboveground-carbon proxy is one expression applied to every cell at once:

```

573 carbon <- ba * 12 # illustrative MgC per cell from basal area
574 names(carbon) <- "carbon"

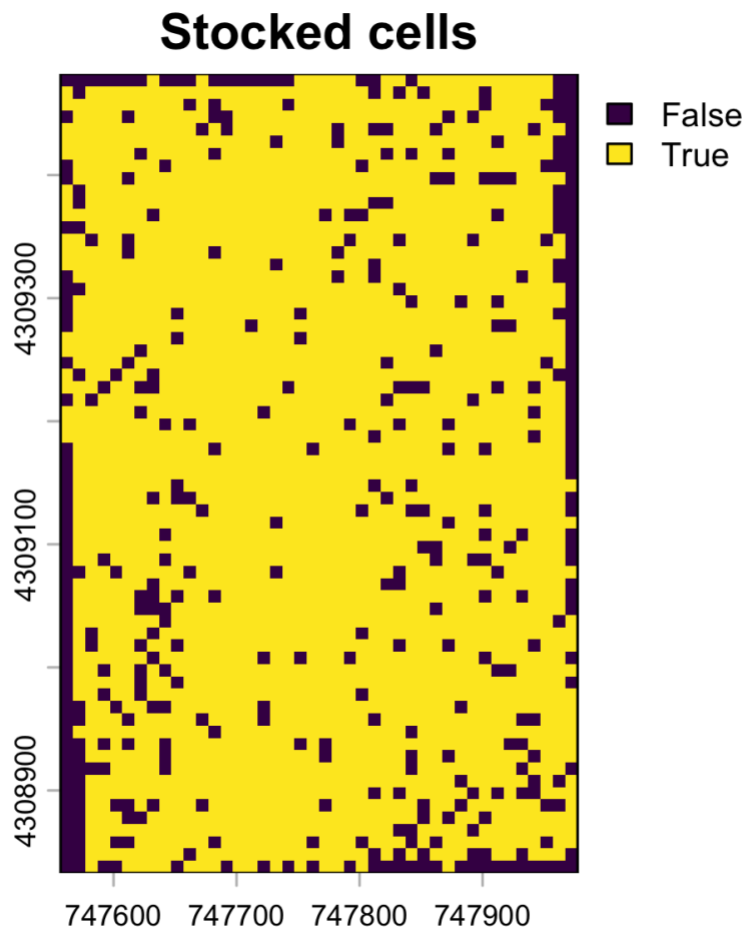
```

575 Logical operators work the same way and return a mask of ones and zeros. A “stocked” mask,
 576 cells carrying any measurable basal area, is a single comparison:

```

577 stocked <- ba > 0.05
578 plot(stocked, main = "Stocked cells")

```



579

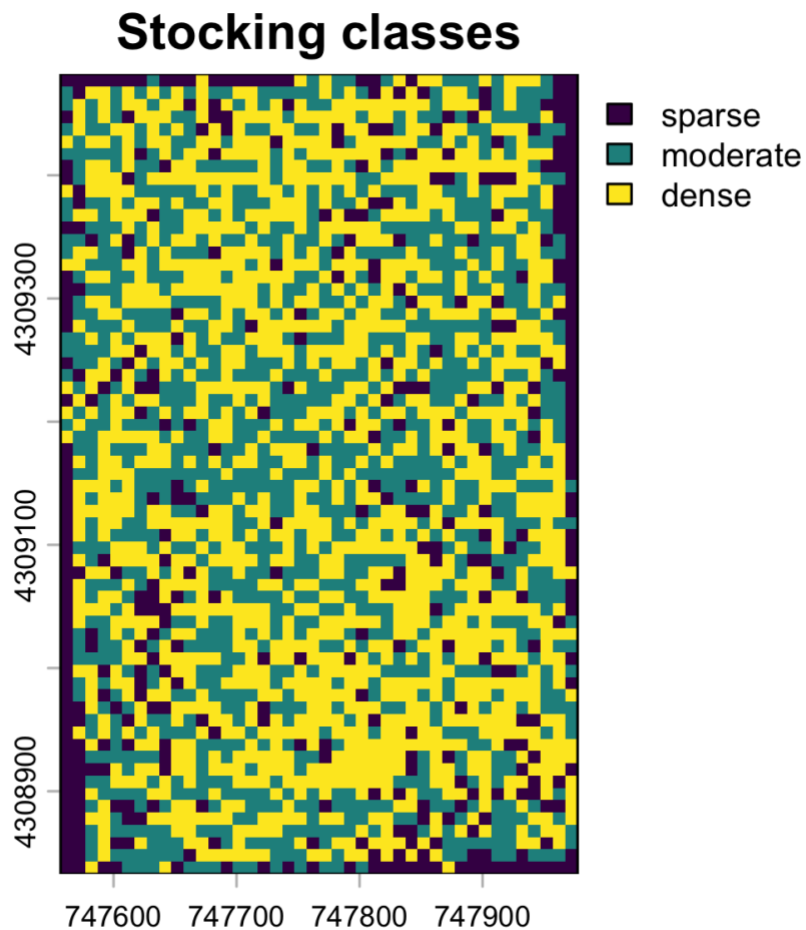
580 3.4 Reclassification

581 Turning a continuous surface into a small number of classes is reclassification, the operation
 582 behind every land-cover and zonation map. `classify()` takes a matrix of from, to and becomes
 583 rows and cuts the surface at the thresholds:

```

584 rcl <- matrix(c(-Inf, 0.05, 1, # sparse
585               0.05, 0.30, 2, # moderate
586               0.30, Inf, 3), # dense
587             ncol = 3, byrow = TRUE)
588 zones <- classify(ba, rcl)
589 levels(zones) <- data.frame(id = 1:3, class = c("sparse", "moderate", "dense"))
590 plot(zones, main = "Stocking classes")

```



591

592 3.5 Extraction

593 The bridge from raster back to field data is extraction: reading the raster value at each point
 594 location. `terra::extract()` samples the surface wherever a plot, a tree, or a sample sits, which is
 595 how gridded covariates enter a plot-level model:

```
596 pts <- spatSample(as.polygons(ext(ba), crs = crs(ba)), 5, method = "random")
597 extract(ba, pts)
```

ID	basal_area
1	0.5316431
2	0.3700364
3	0.0149069
4	0.4351978
5	0.0622051

598 Extraction and reclassification are the two operations that connect the raster and vector worlds,
599 and everything applied in the rest of the book uses one or both.

600 **3.6 Area accounting**

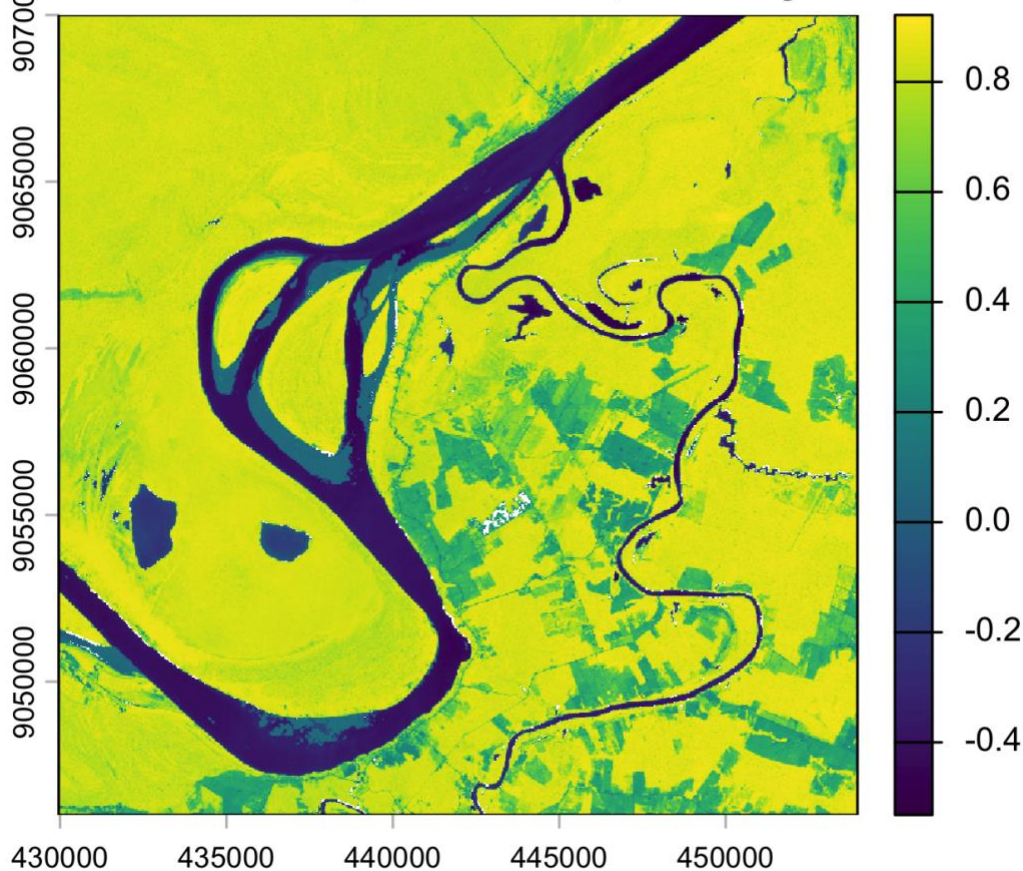
601 Remote-sensing verification turns imagery into a land-use / land-cover (LULC) map and then
602 reasons about it: how much forest, how much converted, how the classes change through time.
603 The Bolivia national land-cover work does this at country scale, and the mechanics are the
604 reclassification and area accounting shown here, which we later carry into the uncertainty chapter
605 to attach a confidence interval to each class area.

606 Everything from here on is a real product rather than a demonstration surface. The data are a
607 Sentinel-2 classification of Rondonia, in the Brazilian Amazon, for 2022, produced with the `sits`
608 package and distributed with `sitsdata` (Simões et al., 2021). Rondonia is among the most heavily
609 cleared states in the Amazon, which makes it the pattern a verifier spends their career looking at.

610 Start with what a classifier actually sees. A spectral index compresses several bands into one
611 number that separates vegetation from everything else, and the normalised difference vegetation
612 index is the standard choice. This is a real sixteen-day Sentinel-2 composite over a twenty-four
613 kilometre window on the Madeira river. `sits` stores the index as an integer scaled by ten thousand,
614 so it has to be divided back:

```
615 ndvi <- rast("data/rondonia_ndvi_2022-07-16.tif") / 1e4  
616 names(ndvi) <- "ndvi"  
617 plot(ndvi, main = "Sentinel-2 NDVI, Rondonia, 16 July 2022")
```

Sentinel-2 NDVI, Rondonia, 16 July 2022



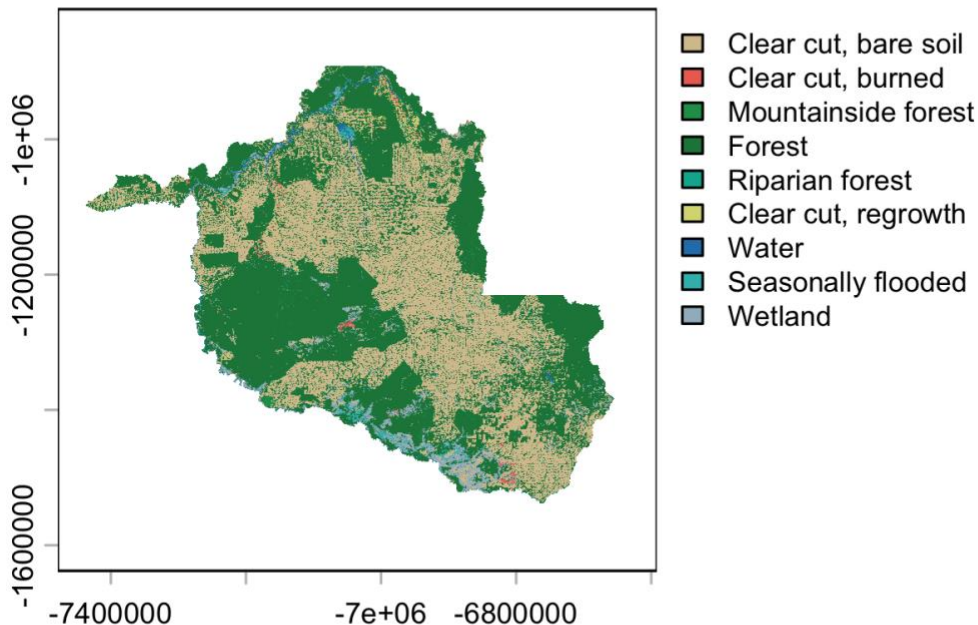
618

619 Read the image as an analyst would. Water is strongly negative because it absorbs in the near
 620 infrared. Intact forest saturates near the top of the scale. The rectangles of intermediate value
 621 east of the river are pasture and cropland cut out of the forest, and their straight edges are the
 622 giveaway that the boundary is a property line rather than anything ecological.

623 A classifier turns that continuous surface into labelled classes. Rather than threshold the index
 624 here, we take the published nine-class map, which was produced from a full year of imagery
 625 instead of a single date. Its legend and colours ship with the product:

```
626 legend <- read.csv("data/rondonia_legend.csv")
627
628 lulc <- rast("data/rondonia_class_2022.tif")
629 levels(lulc) <- legend[, c("id", "cover")]
630 coltab(lulc) <- legend[, c("id", "colour")]
631 plot(lulc, main = "Rondonia land cover, 2022", mar = c(2, 2, 2, 11),
632      plg = list(cex = 0.85))
```

Rondonia land cover, 2022



633

634 The fishbone is the signature of Amazon frontier clearing: a road is cut, side roads run off it at
635 regular spacing, and lots are cleared either side. It is visible here at the scale of an entire state.

636 The class areas are the deliverable. `freq()` counts cells per class, and multiplying by the cell area
637 gives hectares:

```
638 cell_ha <- prod(res(lulc)) / 1e4
639 f <- freq(lulc)
640 f$area_ha <- round(f$count * cell_ha)
641 f[, c("value", "area_ha")]
```

value	area_ha
Clear cut, bare soil	9537618
Clear cut, burned	124018
Mountainside forest	113107
Forest	13376070
Riparian forest	136127

value	area_ha
Clear cut, regrowth	228470
Water	190752
Seasonally flooded	190620
Wetland	932298

642 Reclassifying the nine classes into a forest mask, the exact step behind a forest-area or
643 deforestation figure, is one more `classify()`. Three of the nine classes are forest, and deciding
644 which is a definitional choice, not a technical one:

```
645 forest <- classify(lulc, cbind(legend$id, as.integer(legend$forest)))
646 #>
647 |-----|-----|-----|-----|
648 =====
649
650 forest_ha <- global(forest, "sum", na.rm = TRUE)[1, 1] * cell_ha
651 round(forest_ha)                                # hectares of forest
652 #> [1] 13625304
```

653 That single number is what a project claims and an auditor recomputes. Before accepting it,
654 though, ask what the map gets wrong. The product ships a validation sample of independently
655 interpreted points, and comparing it against the map is exactly the extraction operation from
656 earlier in the chapter:

```
657 val <- read.csv("data/rondonia_validation.csv")
658 pts <- project(vect(val, geom = c("longitude", "latitude"), crs = "EPSG:4326"),
659               crs(lulc))
660
661 val$mapped <- levels(lulc)[[1]]$cover[extract(lulc, pts)[, 2]]
662 val <- val[!is.na(val$mapped), ]          # a few points fall outside the mosaic
663
664 cm <- table(mapped = val$mapped, reference = val$cover)
665 round(100 * sum(diag(cm)) / sum(cm), 1)    # overall accuracy, per cent
666 #> [1] 83
667
668 data.frame(users      = round(100 * diag(cm) / rowSums(cm), 1),
669            producers  = round(100 * diag(cm) / colSums(cm), 1))
```

	users	producers
Clear cut, bare soil	81.5	98.3
Clear cut, burned	87.5	33.3
Clear cut, regrowth	82.0	51.6
Forest	84.8	98.0
Mountainside forest	68.8	18.2
Riparian forest	66.5	93.3
Seasonally flooded	85.5	92.9
Water	96.8	95.3

	users	producers
Wetland	87.1	69.3

Eighty-three per cent overall would pass most reporting thresholds without comment, and the per-class figures show why that is not enough. Water is found almost perfectly. Mountainside forest is caught less than a fifth of the time, and burned clear-cut a third, because both are confused with the larger class they sit next to. An omission rate that high on a minor class barely moves overall accuracy, but it moves that class's area a great deal, and if the class in question is the one being credited, it moves the credits. The uncertainty chapter returns to this matrix to adjust the mapped areas and put a confidence interval on each, because a class area is only as good as the map's accuracy.

Classification with sits

The map above was read from disk. Producing it is the `sits` workflow, which organises satellite time series into regularised data cubes and classifies the whole cube rather than a single date (Simões et al., 2021). The sequence below is not executed here, because the book renders offline and the dependency stack is heavy, but it is the chain that generated the file this section reads.

```
library(sits)
library(sitsdata)

# a local cube: the shipped tile, no network access
cube <- sits_cube(source = "MPC", collection = "SENTINEL-2-L2A",
                  data_dir = system.file("extdata/Rondonia-20LMR",
                                         package = "sitsdata"))

# training samples carry a full year of observations per point, not one date
data("samples_deforestation_rondonia")
model <- sits_train(samples_deforestation_rondonia,
                    sits_rfor(num_trees = 120, mtry = 10))

probs <- sits_classify(cube, model, output_dir = tempdir(), multicores = 4)

# Bayesian smoothing over the probability cube, before any hard label exists
bayes <- sits_smooth(probs, output_dir = tempdir(), multicores = 4)
map <- sits_label_classification(bayes, output_dir = tempdir())

# area estimation and accuracy in the Olofsson sense, from a stratified sample
design <- sits_sampling_design(map, expected_ua = c(Forest = 0.75))
samples <- sits_stratified_sampling(map, design, alloc = "alloc_120")
acc <- sits_accuracy(map, validation = samples)
```

Two steps deserve attention because they have no equivalent in a single-date workflow. `sits_smooth()` applies Bayesian smoothing to the probability cube before labelling, which suppresses isolated misclassified pixels while preserving real edges. And `sits_stratified_sampling()` allocates the validation sample across classes by their mapped area, so that rare classes receive enough points to estimate their accuracy at all, which is precisely the failure the per-class table above exposes.

3.7 Exercises

1. Build the basal-area raster at 20-metre resolution instead of 10. How do the mean cell value and the number of cells change, and why does total basal area stay the same?
2. Reclassify the basal-area surface into four stocking classes and report the area in hectares of each. Which class covers the most ground?
3. Extract the basal-area value at the location of every stem with `extract()`. Why is the extracted value not the same as the stem's own basal area?
4. (*Applied.*) The forest mask counts three of Rondonia's nine classes as forest. Recompute the forest area with riparian forest excluded, and again with clear-cut regrowth included. Report the range in hectares, and explain why the definition of forest, rather than the accuracy of the classification, is often the largest single lever on a reported area.
5. (*Applied.*) Mountainside forest has a producer's accuracy under twenty per cent. Find which class absorbs the missing points in the confusion matrix, and state what you would require of a project before accepting an area figure for that class.

4. Terrain and Hydrology

4.1 Elevation models

Terrain is the covariate behind almost everything else in a forest. It drives productivity, access, erosion, and where water collects. It arrives as a digital elevation model (DEM), a raster whose cell value is height above a datum.

This chapter works on Lake Chilwa in southern Malawi, the area of interest used throughout the companion map-templates material. Chilwa is a shallow endorheic lake: water enters from the surrounding hills and leaves only by evaporation. There is no outlet. Its area swings by a factor of several between wet and dry years, and it has dried out completely three times in living memory, in 1968, 1995 and 2012. That makes it a hard case for watershed delineation, whose standard algorithms assume drainage eventually reaches the sea, and an obvious case for inundation mapping, which is the deliverable a wetland project needs.

In operational work you download elevation for your own site. The `elevatr` package fetches open tiles for any area:

```
library(elevatr)
dem <- get_elev_raster(aoi, z = 10, clip = "locations") |> rast()
```

So the book builds offline, `data-raw/prepare-chilwa-data.R` did this once and committed the result. The grid is EPSG:3857 at 100 m, matching the map-templates workflow, and covers the lake plus 60 km of margin so the catchment divide falls inside the raster rather than being clipped by it:

```

744 library(terra)
745 library(sf)
746 library(whitebox)
747
748 sf_use_s2(FALSE)
749
750 dem <- rast("data/chilwa_dem.tif")
751 lake <- st_read("data/chilwa_hydro.gpkg", layer = "lake", quiet = TRUE)
752 basin <- st_read("data/chilwa_hydro.gpkg", layer = "basin", quiet = TRUE)
753 rivers <- st_read("data/chilwa_hydro.gpkg", layer = "rivers", quiet = TRUE)
754
755 dim(dem)
756 #> [1] 1733 1536 1
757 global(dem, range, na.rm = TRUE)

```

	X1	X2
elevation	234	2975

758 4.1.1 Projection distortion

759 One property of the grid has to be settled before any area is quoted, because it silently corrupts
760 every hydrological number that follows.

761 EPSG:3857 is a Mercator projection. It is conformal, not equal-area, and its scale factor grows
762 with latitude. A cell that is nominally 100 m square does not cover a hectare of ground anywhere
763 except the equator. At Chilwa's latitude the error is large enough to matter:

```

764 cell_ha <- cellSize(dem, unit = "ha")
765 global(cell_ha, range, na.rm = TRUE)

```

	X1	X2
area	0.9185027	0.9315157

766 The convenient identity that one 100 m cell equals one hectare, so a flow accumulation count
767 multiplied by 0.01 gives square kilometres, is therefore wrong here by about seven per cent. It is
768 a tempting shortcut and it appears in a great deal of published hydrological code. This chapter
769 uses `cellSize()` throughout and reports areas after reprojection to UTM zone 36S (EPSG:32736),
770 which is equal-area enough at this scale to serve as truth.

771 4.2 Terrain derivatives

772 Elevation is rarely the variable of interest. What it implies, slope, aspect and ruggedness, is.
773 `terrain()` derives these directly. Slope and aspect need a projected DEM so that horizontal and
774 vertical distances share units:

```

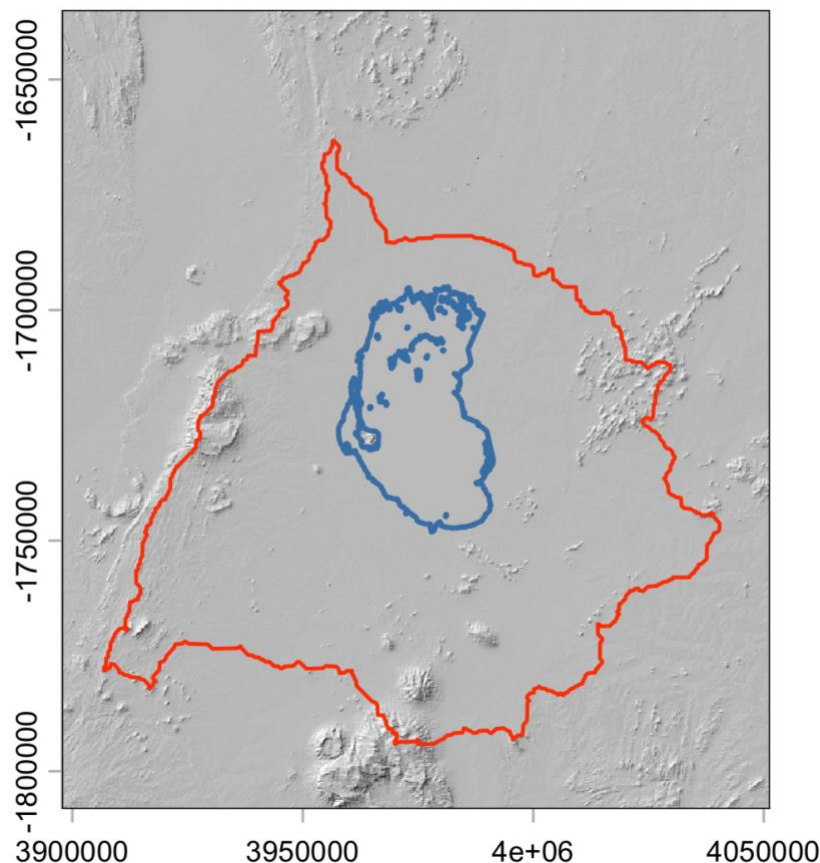
775 slope <- terrain(dem, "slope", unit = "degrees")
776 aspect <- terrain(dem, "aspect", unit = "degrees")
777 tri <- terrain(dem, "TRI")
778
779 global(slope, c("mean", "max"), na.rm = TRUE)

```

	mean	max
slope	2.974133	82.15639

780 Slope governs operability and erosion risk. Ruggedness is a useful productivity covariate,
 781 because sheltered, gentler terrain often carries taller and denser stands. A hillshade turns terrain
 782 into an intuitive relief image by simulating illumination, and is the standard base layer under any
 783 map:

```
784 hs <- shade(terrain(dem, "slope", unit = "radians"),
785             terrain(dem, "aspect", unit = "radians"),
786             angle = 40, direction = 315)
787
788 plot(hs, col = grey(0:100 / 100), legend = FALSE, mar = c(2, 2, 2, 2))
789 plot(st_geometry(lake), add = TRUE, border = "steelblue", lwd = 2)
790 plot(st_geometry(basin), add = TRUE, border = "orangered", lwd = 1.5)
```



791
 792 *Relief of the Lake Chilwa basin. The lake occupies the flat floor at centre; the Zomba and Mulanje massifs*
 793 *rise on the western and southern margins.*

4.3 Hydraulic conditioning

Water moves downhill, so a DEM already encodes drainage. It does not encode it cleanly. Every DEM contains spurious depressions: cells with no lower neighbour, produced by interpolation error, vegetation returns, and the finite vertical precision of the sensor. A routing algorithm entering one of these has nowhere to send the water, and the drainage network fragments into disconnected pieces.

Conditioning removes the artefacts while preserving the real depressions. For Chilwa that distinction is the whole problem, because the lake is itself a depression, and one the algorithm must not fill:

```
dir.create("data/chilwa-hydro", showWarnings = FALSE)
wd <- "data/chilwa-hydro/"
file.copy("data/chilwa_dem.tif", paste0(wd, "dem.tif"), overwrite = TRUE)
#> [1] TRUE

wbt_depth_in_sink(dem = "dem.tif", output = "sink_depth.tif",
                  wd = wd, zero_background = TRUE, verbose_mode = FALSE)

sink_depth <- rast(paste0(wd, "sink_depth.tif"))
global(sink_depth > 0, "sum", na.rm = TRUE)[1, 1]
#> [1] 492310
global(sink_depth, "max", na.rm = TRUE)[1, 1]
#> [1] 407
```

Roughly half a million cells sit in some depression, and the deepest sits hundreds of metres below its spillover point. No threshold on depth alone separates real basins from interpolation pits, because the two overlap in size: the deepest single pit in this grid is an artefact, not the lake.

4.3.1 Stochastic depressions

Depth cannot make the distinction, but uncertainty can. A DEM cell is an estimate with an error, and a depression that survives being perturbed by that error is probably real, while one that disappears was probably never there.

Lindsay's stochastic depression analysis (**lindsay2006distinguishing?**) formalises this as a Monte Carlo experiment. Each iteration adds a Gaussian error field to the DEM, drawn with the sensor's root mean square error and a spatial autocorrelation range, then identifies the depressions in the perturbed surface. The output is the proportion of iterations in which each cell was in a depression, which reads directly as a probability. Radar DEMs at this resolution carry a vertical error of around three metres, which sets `rmse`:

```
wbt_stochastic_depression_analysis(dem = "dem.tif", output = "sink_prob.tif",
                                  wd = wd, rmse = 3.0, range = 10.0,
                                  iterations = 50, verbose_mode = FALSE)
```

```

833 sink_prob <- rast(paste0(wd, "sink_prob.tif"))
834
835 data.frame(
836   threshold = c(0.2, 0.5, 0.8),
837   cells      = vapply(c(0.2, 0.5, 0.8),
838                       function(p) global(sink_prob >= p, "sum", na.rm = TRUE)[1, 1],
839                       numeric(1))
840 )

```

threshold	cells
0.2	1094087
0.5	395172
0.8	260747

841 Raising the threshold from 0.2 to 0.8 discards about three quarters of the candidate depressions
842 as artefacts of DEM error. The question is which side of that line the lake falls on:

```

843 lake_mask <- rasterize(vect(lake), dem)
844 round(global(ifel(!is.na(lake_mask), sink_prob, NA), "mean", na.rm = TRUE)[1, 1], 3)
845 #> [1] 0.999

```

846 Effectively unity. The lake is a depression in every iteration, which is the evidence needed to treat
847 it as a genuine terminal sink rather than something to be conditioned away. Running this before
848 conditioning, rather than assuming a depth threshold will separate signal from noise, is the
849 difference between a defensible endorheic analysis and a lucky one.

850 4.3.2 Depression breaching

851 Two families of algorithm exist. Filling raises the floor of a depression until it overflows, which is
852 simple but destroys the depression. Breaching carves a channel out through the lowest point of
853 the rim, which changes far fewer cells and preserves the basin's shape.

854 WhiteboxTools implements Lindsay's hybrid method (**lindsay2016efficient?**), which breaches what
855 it can within a depth limit and fills whatever remains. The two parameters do the work. `max_depth`
856 caps how deep a channel may be carved, so shallow artefacts are breached while a genuine 400
857 m basin is left alone. `flat_increment` adds a small monotonic gradient along the carved channel, so
858 flow directions stay unambiguous across ground the DEM records as perfectly level:

```

859 t_breach <- system.time(
860   wbt_breach_depressions(dem = "dem.tif", output = "breached.tif", wd = wd,
861                          max_depth = 3, flat_increment = 0.001,
862                          verbose_mode = FALSE)
863 )
864
865 breached <- rast(paste0(wd, "breached.tif"))
866 carved <- dem - breached
867
868 data.frame(
869   cells_changed = global(carved != 0, "sum", na.rm = TRUE)[1, 1],
870   max_carve_m   = round(global(carved, "max", na.rm = TRUE)[1, 1], 2),

```

```

871 max_raise_m = round(-global(carved, "min", na.rm = TRUE)[1, 1], 2),
872 seconds     = round(t_breach[["elapsed"]], 1)
873 )

```

cells_changed	max_carve_m	max_raise_m	seconds
483553	2.45	393	10.8

874 Read both columns. The carving obeys its limit exactly: no cell is lowered more than the three
875 metres imposed. The raising does not, because it is not constrained by `max_depth` at all. Lindsay's
876 method is a hybrid. It breaches what it can within the depth limit and fills everything else, and the
877 filling is unbounded.

878 That asymmetry is easy to miss, and it is the single most consequential behaviour in this chapter.

879 4.3.3 Depression filling

880 A second pass with a filling algorithm normally follows, to clear whatever the breaching step could
881 not reach. Wang and Liu's scan-line method (**wang2006efficient?**) is the usual choice for low-
882 relief terrain:

```

883 wbt_fill_depressions_wang_and_liu(dem = "breached.tif", output = "filled.tif",
884                                   wd = wd, verbose_mode = FALSE)
885
886 filled <- rast(paste0(wd, "filled.tif"))
887 global((filled - breached) != 0, "sum", na.rm = TRUE)[1, 1]
888 #> [1] 0

```

889 Zero cells change. That is the correct result and it is worth pausing on, because it is easy to run
890 a fill step, see a new file appear, and assume it did something. The breaching step already filled
891 every depression it declined to breach, so its output was depression-free before Wang and Liu
892 ever saw it. The second pass is redundant here. It is retained because it costs seconds and
893 because it is not redundant after a pure breaching algorithm, but the honest report is that it
894 changed nothing.

895 4.3.4 Endorheic cost

896 If the conditioned surface has no depressions left, then it has no Lake Chilwa. The stochastic
897 analysis established that the lake is a real basin with probability one, and the conditioning step
898 then filled it in:

```

899 raised <- breached - dem
900 lake_raise <- ifel(!is.na(lake_mask), raised, NA)
901
902 data.frame(
903   mean_raise_m = round(global(lake_raise, "mean", na.rm = TRUE)[1, 1], 1),
904   max_raise_m   = round(global(lake_raise, "max", na.rm = TRUE)[1, 1], 1),
905   raw_surface_m = as.numeric(global(ifel(!is.na(lake_mask), dem, NA),
906                                     quantile, probs = 0.5, na.rm = TRUE)),
907   cond_surface_m = round(as.numeric(global(ifel(!is.na(lake_mask), breached, NA),

```

```

908 quantile, probs = 0.5, na.rm = TRUE)), 1)
909 )

```

mean_raise_m	max_raise_m	raw_surface_m	cond_surface_m
14.1	83.5	623	637.2

910 The lake bed has been raised by fourteen metres on average and by more than eighty at its
 911 deepest, and its median surface has moved from 623 m to 637 m. In the conditioned DEM, Chilwa
 912 is a gentle ramp that water crosses on its way somewhere else.

913 This is not a bug and it is not avoidable by tuning `max_depth`. Flow routing requires that every cell
 914 have a downhill neighbour, and a terminal sink by definition does not. Any algorithm that produces
 915 a routable surface must remove the sink. The published claim that a depth constraint “preserves
 916 deeper basin structures while breaching shallow artefacts” does not survive measurement: the
 917 constraint bounds the carving, not the filling.

918 Two rules follow, and they govern the rest of the chapter.

919 Flow routing uses the conditioned DEM, and the endorheic basin is expected to be gone from it.
 920 That is fine, because what is wanted from routing is the convergence pattern, and filling a flat sink
 921 with a monotonic gradient preserves where water arrives even though it invents where it leaves.

922 Anything that depends on the shape of the depression, which means depression depth,
 923 inundation extent and any stage-area relationship, uses the raw DEM. Both `wbt_depth_in_sink` and
 924 `wbt_stochastic_depression_analysis` above were run on `dem.tif` for exactly this reason. Running an
 925 inundation model on a conditioned surface is a silent and total error: the basin whose flooding is
 926 being modelled has already been filled.

927 4.4 Flow routing

928 The D8 algorithm (**o1984extraction?**) sends all water from a cell to whichever of its eight
 929 neighbours lies steepest downhill. Applied across the grid it yields a pointer raster, and following
 930 the pointers upstream from each cell counts how many cells drain through it. That count is flow
 931 accumulation, and it is the basis of every stream network and catchment boundary:

```

932 t_flow <- system.time({
933   wbt_d8_pointer(dem = "filled.tif", output = "d8.tif",
934                 wd = wd, verbose_mode = FALSE)
935   wbt_d8_flow_accumulation(input = "d8.tif", output = "acc.tif",
936                            wd = wd, pntr = TRUE, verbose_mode = FALSE)
937 })
938
939 acc <- rast(paste0(wd, "acc.tif"))
940 round(t_flow[["elapsed"]], 1)
941 #> [1] 11

```

```
942 global(acc, "max", na.rm = TRUE)[1, 1]
943 #> [1] 1152591
```

944 4.4.1 Reading accumulation

945 The maximum accumulation on the grid is a little over 1.15 million cells. It is tempting to read that
946 as the size of the Chilwa catchment, and to convert it to area with the one-cell-one-hectare
947 shortcut. Both steps would be wrong.

948 The conversion is wrong for the reason established above: these cells are not hectares. The
949 interpretation is wrong for a subtler reason. The DEM extends 60 km beyond the lake in every
950 direction, so it contains parts of neighbouring catchments whose water leaves the grid edge. The
951 global maximum belongs to whichever drainage is largest, not necessarily to Chilwa. Checking
952 where it falls settles it:

```
953 peak_cell <- which.max(values(acc))
954 peak_xy   <- xyFromCell(acc, peak_cell)
955
956 st_intersects(st_as_sf(data.frame(x = peak_xy[1], y = peak_xy[2]),
957                                coords = c("x", "y"), crs = 3857),
958              lake, sparse = FALSE)[1, 1]
959 #> [1] FALSE
```

960 The cell of maximum accumulation is not in the lake. The number describes some other river
961 leaving the raster, and quoting it as Chilwa's contributing area would overstate the catchment by
962 more than a third. The figure that matters is the largest accumulation reached inside the lake,
963 which is where Chilwa's own drainage converges:

```
964 lake_acc <- ifel(!is.na(lake_mask), acc, NA)
965
966 outlet_acc <- global(lake_acc, "max", na.rm = TRUE)[1, 1]
967 outlet_cell <- which.max(values(lake_acc))
968 outlet_xy   <- xyFromCell(lake_acc, outlet_cell)
969
970 outlet_acc
971 #> [1] 911440
```

972 4.5 Watershed delineation

973 That cell is the terminal pour point: the single location on which the whole endorheic system
974 converges. Delineation traces the pointer raster upstream from it and returns every cell that drains
975 there:

```
976 outlet <- st_as_sf(data.frame(x = outlet_xy[1], y = outlet_xy[2]),
977                      coords = c("x", "y"), crs = 3857)
978 st_write(outlet, paste0(wd, "outlet.shp"), delete_dsn = TRUE, quiet = TRUE)
979
980 wbt_watershed(d8_pntr = "d8.tif", pour_pts = "outlet.shp",
981              output = "watershed.tif", wd = wd, verbose_mode = FALSE)
982
983 ws <- rast(paste0(wd, "watershed.tif"))
984 ws_poly <- as.polygons(ws) |> st_as_sf() |> st_transform(32736)
985 ws_km2 <- sum(as.numeric(st_area(ws_poly))) / 1e6
```

```

986 round(ws_km2, 1)
987 #> [1] 8430.5
988

```

989 4.5.1 Independent comparison

990 A delineated catchment is a claim, and claims get checked. HydroSHEDS publishes an
991 independent global product, HydroBASINS, derived from a different DEM by a different group.
992 Comparing against it is the cheapest validation available:

```

993 basin_utm <- st_transform(basin, 32736)
994 basin_km2 <- as.numeric(st_area(basin_utm)) / 1e6
995
996 overlap_km2 <- st_intersection(st_union(ws_poly), st_union(basin_utm)) |>
997   st_area() |> as.numeric() |> sum() / 1e6
998
999 data.frame(
1000   source = c("This delineation", "HydroBASINS v1.0"),
1001   area_km2 = round(c(ws_km2, basin_km2), 1),
1002   agreement_pct = round(100 * overlap_km2 / c(ws_km2, basin_km2), 1)
1003 )

```

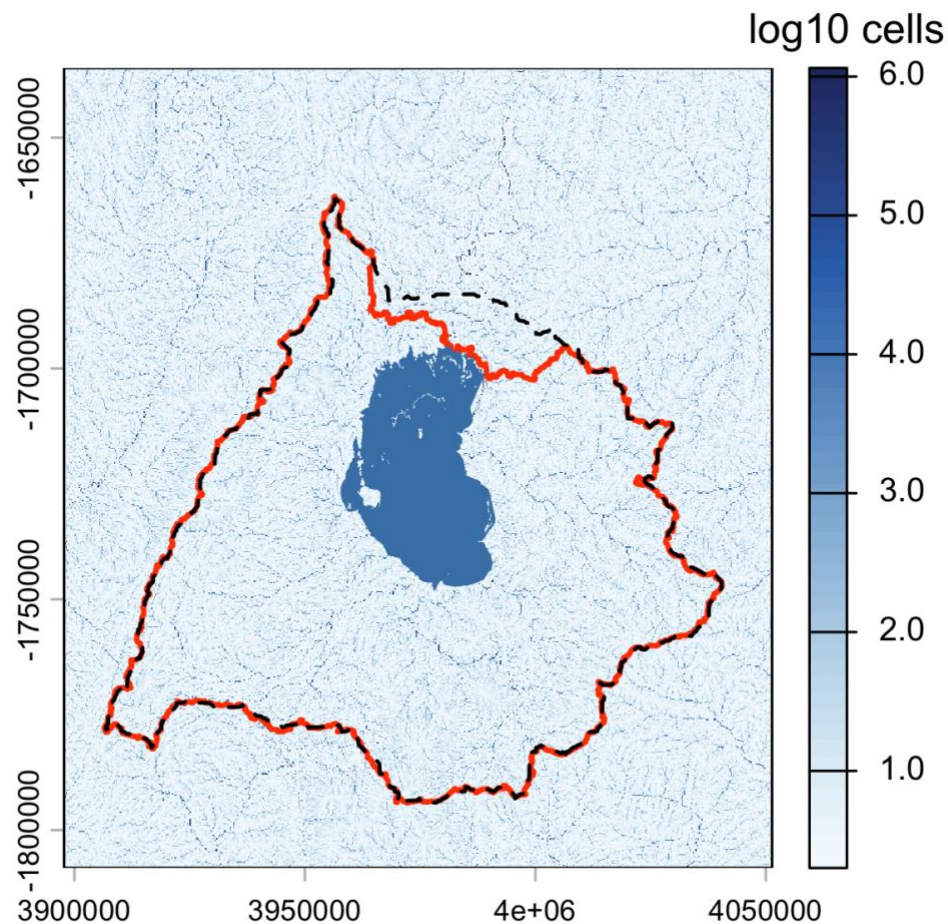
source	area_km2	agreement_pct
This delineation	8430.5	99.2
HydroBASINS v1.0	8763.9	95.5

1004 The two agree closely. Where they differ is along the flat eastern margin, which is the expected
1005 place: on ground with almost no gradient, small differences in the source DEM move the divide a
1006 long way, and neither answer is more correct than the other. A verifier reading a catchment
1007 boundary should ask what the relief is along the divide before treating the line as precise.

```

1008 plot(log10(acc + 1), col = hcl.colors(50, "Blues", rev = TRUE),
1009      mar = c(2, 2, 2, 4), plg = list(title = "log10 cells"))
1010 plot(st_geometry(st_transform(ws_poly, 3857)), add = TRUE,
1011      border = "orangered", lwd = 2)
1012 plot(st_geometry(basin), add = TRUE, border = "black", lwd = 1.5, lty = 2)
1013 plot(st_geometry(lake), add = TRUE, col = "steelblue", border = NA)

```



Delineated catchment against the HydroBASINS boundary, over the flow accumulation surface.

4.6 Stream networks

A stream is a cell whose accumulation exceeds a threshold. The threshold is a choice, not a property of the landscape: lower it and ephemeral gullies appear, raise it and only trunk rivers survive. Strahler ordering (**strahler1957quantitative?**) then ranks the network, with headwaters at order one and the order rising wherever two channels of equal rank meet:

```
wbt_extract_streams(flow_accum = "acc.tif", output = "streams.tif", wd = wd,
                    threshold = 1000, zero_background = TRUE,
                    verbose_mode = FALSE)

wbt_strahler_stream_order(d8_ptr = "d8.tif", streams = "streams.tif",
                          output = "order.tif", wd = wd,
                          zero_background = TRUE, verbose_mode = FALSE)

order_rast <- rast(paste0(wd, "order.tif"))
table(values(order_rast)[values(order_rast) > 0])
#>
#>      1      2      3      4      5      6
#> 29789 19784  7059  2937  1915    1
```

The counts fall off geometrically, which is Horton's law of stream numbers and a useful sanity check: a network that does not thin as order rises has usually been built on a broken pointer raster. Retaining order three and above leaves the primary channels, the ones a field team could actually find.

Raster streams are awkward to work with. Vectorising them gives line geometry that can be measured, buffered and joined, and dropping short fragments first removes the single-cell spurs that raster thinning always leaves behind:

```
wbt_remove_short_streams(d8_pntr = "d8.tif", streams = "streams.tif",
  output = "streams_long.tif", wd = wd,
  min_length = 200, verbose_mode = FALSE)

wbt_raster_streams_to_vector(streams = "streams_long.tif", d8_pntr = "d8.tif",
  output = "streams.shp", wd = wd,
  verbose_mode = FALSE)

streams_v <- st_read(paste0(wd, "streams.shp"), quiet = TRUE)
st_crs(streams_v) <- 3857

data.frame(
  reaches      = nrow(streams_v),
  total_km     = round(sum(as.numeric(st_length(st_transform(streams_v, 32736)))) / 1000, 1),
  ref_reaches  = nrow(rivers),
  ref_total_km = round(sum(rivers$LENGTH_KM, na.rm = TRUE), 1)
)
```

reaches	total_km	ref_reaches	ref_total_km
1427	7061.8	554	2375.1

The derived network is far denser than the HydroRIVERS reference, which is the expected consequence of a 1,000-cell threshold against a global product built for cartographic legibility. Neither is wrong. The comparison is only meaningful once both are cut to the same threshold, and a project quoting stream length without quoting its extraction threshold has quoted nothing.

4.6.1 Wetness index

Flow accumulation says how much water arrives at a cell. Slope says how fast it leaves. The topographic wetness index combines them as the log of the ratio, and it is the standard proxy for where soil stays saturated:

The `sca` argument wants specific catchment area and the `slope` argument wants a slope raster in degrees. Passing the DEM to the second, which is an easy slip because both are elevation-shaped files, returns numbers that look plausible and mean nothing:

```
wbt_slope(dem = "filled.tif", output = "slope_deg.tif",
  wd = wd, verbose_mode = FALSE)

wbt_wetness_index(sca = "acc.tif", slope = "slope_deg.tif",
```

```

1073         output = "twi.tif", wd = wd, verbose_mode = FALSE)
1074
1075 twi <- rast(paste0(wd, "twi.tif"))
1076 global(twi, quantile, probs = c(0.05, 0.5, 0.95), na.rm = TRUE)

```

	X5.	X50.	X95.
twi	1.974806	4.648026	12.56757

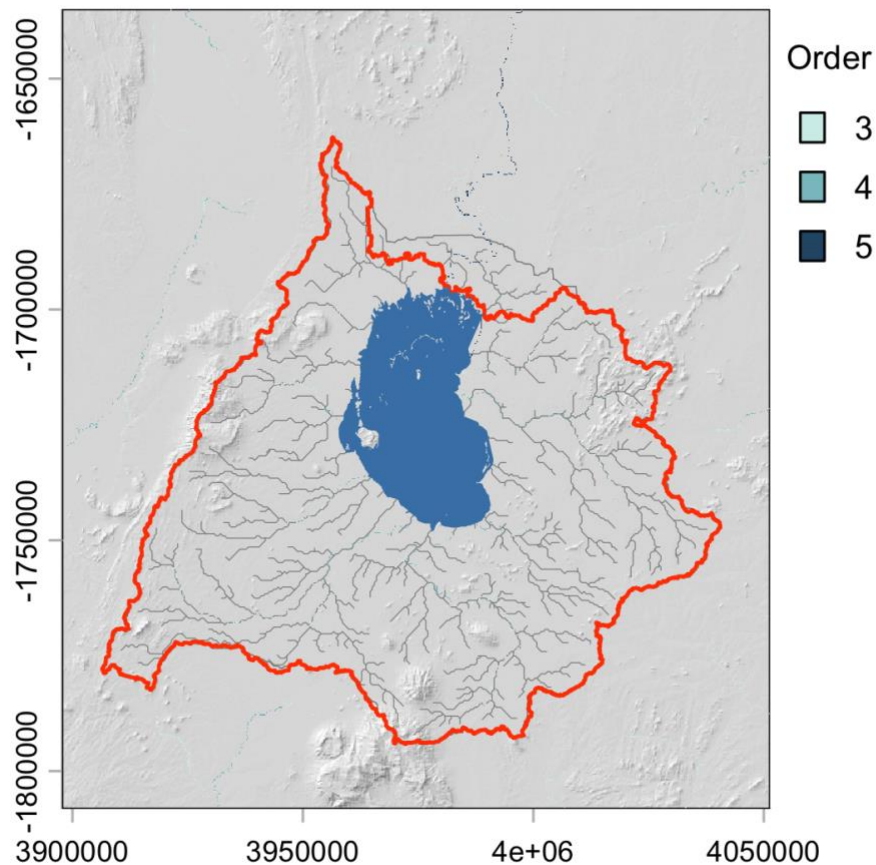
1077 High values mark valley floors and flat convergent ground, low values mark steep divides. For
 1078 carbon work the index is a cheap covariate for soil organic carbon and for the wetland and
 1079 seasonally flooded classes that land-cover maps confuse most often. It is derived, not observed,
 1080 and it inherits every artefact of the conditioning that produced the accumulation surface, so it
 1081 belongs in a model as a predictor rather than in a report as a measurement.

1082 The primary channels, mapped against the reference network:

```

1083 major <- ifel(order_rast >= 3, order_rast, NA)
1084
1085 plot(hs, col = grey(40:100 / 100), legend = FALSE, mar = c(2, 2, 2, 4))
1086 plot(st_geometry(rivers), add = TRUE, col = "grey55", lwd = 0.4)
1087 plot(major, add = TRUE, col = hcl.colors(4, "Teal", rev = TRUE),
1088      plg = list(title = "Order"))
1089 plot(st_geometry(st_transform(ws_poly, 3857)), add = TRUE,
1090      border = "orangered", lwd = 1.5)
1091 plot(st_geometry(lake), add = TRUE, col = "steelblue", border = NA)

```



Strahler stream network of order three and above, with the delineated catchment and the HydroRIVERS reference channels.

4.7 Inundation mapping

Wetland accounting asks a direct question of a DEM: given a water level, what land floods? For Chilwa the question is not academic. The lake's area is the quantity a wetland carbon project would be crediting, and it is not stable.

The method is a threshold on elevation. Cells below the chosen level flood. Two refinements make it work. The flooded set is restricted to the catchment delineated above, and then to the connected depression that actually contains the lake, found by labelling connected components. Without both, the threshold floods every unrelated valley in the raster.

4.7.1 Anchoring the datum

The level to sweep around has to come from somewhere, and the obvious choice is wrong. Taking the minimum elevation inside the lake polygon gives this:

```

1106 lake_elev <- ifel(!is.na(lake_mask), dem, NA)
1107 global(lake_elev, quantile, probs = c(0, 0.05, 0.5, 0.95, 1), na.rm = TRUE)

```

	X0.	X5.	X50.	X95.	X100.
layer	552	622	623	623	1069

1108 The minimum is seventy metres below the median, and the maximum is four hundred metres
 1109 above it. Neither extreme is the lake. The low value is an interpolation pit, a single cell of DEM
 1110 noise, and the high value is ground inside a polygon drawn at a coarser resolution than the raster.
 1111 A rating curve anchored on the minimum sits seventy metres below the water and returns zero
 1112 area at every step a wetland project would care about.

1113 The median is robust to both, and it is what the DEM actually recorded as the water surface:

```

1114 surface_m <- as.numeric(global(lake_elev, quantile, probs = 0.5, na.rm = TRUE))
1115 surface_m
1116 #> [1] 623

```

1117 Six hundred and twenty-three metres, which is the elevation Lake Chilwa is conventionally given.

1118 4.7.2 Stage and area

```

1119 dem_ws <- mask(dem, ws)
1120
1121 flood_at <- function(level_m) {
1122   wet <- ifel(dem_ws <= level_m, 1, NA)
1123   parts <- patches(wet, directions = 8)
1124   keep <- unique(values(ifel(!is.na(lake_mask), parts, NA)))
1125   keep <- keep[!is.na(keep)]
1126   if (!length(keep)) return(0)
1127   global(ifel(parts %in% keep, cell_ha, NA), "sum", na.rm = TRUE)[1, 1]
1128 }
1129
1130 round(flood_at(surface_m) / 100, 1)
1131 #> [1] 1081.7

```

1132 At the surface the DEM itself records, the threshold returns an area within about one per cent of
 1133 the independently mapped HydroLAKES extent. That is a real check, and it is worth more than
 1134 the single number: it says the method and the datum are both sound before any counterfactual
 1135 level is imposed on them.

1136 The deliverable is the rating curve. Sweep the level, record the area, and you have the relationship
 1137 between stage and flooded extent that drives flood risk, wetland carbon and reservoir planning:

```

1138 offsets <- c(-2, -1, 0, 1, 2, 3, 4, 6)
1139 areas_ha <- vapply(surface_m + offsets, flood_at, numeric(1))
1140
1141 curve <- data.frame(
1142   stage_m = offsets,
1143   level_m = surface_m + offsets,
1144   area_km2 = round(areas_ha / 100, 1)
1145 )
1146 curve

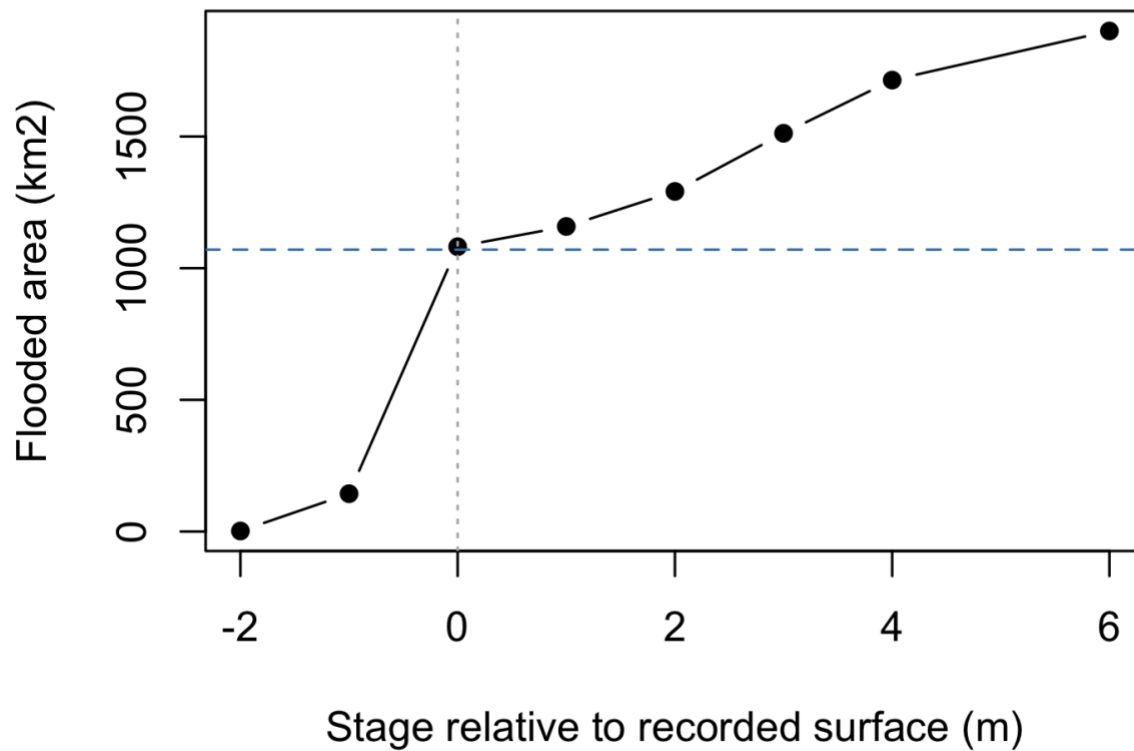
```

stage_m	level_m	area_km2
-2	621	2.0
-1	622	143.6
0	623	1081.7
1	624	1158.5
2	625	1291.5
3	626	1512.3
4	627	1714.2
6	629	1900.7

```

1147 plot(curve$stage_m, curve$area_km2, type = "b", pch = 16,
1148       xlab = "Stage relative to recorded surface (m)",
1149       ylab = "Flooded area (km2)")
1150 abline(h = as.numeric(st_area(st_transform(lake, 32736))) / 1e6,
1151        col = "steelblue", lty = 2)
1152 abline(v = 0, col = "grey70", lty = 3)

```



1153

1154 *Stage-area rating curve for the Chilwa basin. The dashed line is the HydroLAKES mapped extent.*

1155 The curve is violently asymmetric about the recorded surface. Two metres below it the lake is
1156 gone, a residue of two square kilometres. One metre below it, about a seventh of its mapped area
1157 survives. Six metres above it the lake is nearly twice its mapped size. This is not a defect of the
1158 method; it is the lake. Chilwa's bed is almost flat, so stage and area are related by the area of the
1159 shoreline contour, which here is enormous.

1160 Three consequences follow, and they are the reason this method is worth a chapter rather than
1161 a footnote.

1162 A wetland-area claim that does not state its assumed water level is unverifiable. The same one
1163 metre of stage uncertainty that would be immaterial in a steep catchment moves Chilwa's area
1164 by a factor of seven.

1165 The vertical accuracy of the DEM sets the floor on what can be claimed. Radar-derived elevation
1166 at this resolution carries several metres of error, and several metres here spans the entire range
1167 from dry lake to double extent. No amount of care in the horizontal delineation compensates.

1168 The agreement with HydroLAKES at zero stage is a consistency check, not a validation.
1169 HydroLAKES maps a long-run maximum extent, and the DEM records the surface on whatever
1170 dates its source scenes were acquired. That the two coincide is reassuring about the datum and
1171 says nothing about any particular year. Confirming a predicted extent requires a water mask from
1172 imagery acquired on a known date, which is the subject of the activity data chapter.

1173 *Under wetland methodologies*

1174 A project claiming wetland area from a DEM threshold has to defend three things, and a verifier
1175 should ask for all three in writing.

1176 The first is the datum and the stage. An elevation threshold is meaningless without the vertical
1177 datum of the DEM and the water level it assumes, expressed in that datum. A level quoted in
1178 metres above an unnamed reference is not checkable.

1179 The second is the sensitivity. The rating curve above is the evidence: the project should show
1180 how much its claimed area moves for a plausible error in stage, and a curve this steep will not
1181 support a point estimate quoted without an interval.

1182 The third is independent confirmation. A DEM threshold predicts inundation; it does not observe
1183 it. Confirmation requires an optical or radar water mask from a dated acquisition, compared

against the prediction at the stage recorded on that date. Where the two disagree the mask governs, because it is a measurement and the threshold is a model.

Alternative implementations

WhiteboxTools is not the only route. The `flowdem` package wraps RichDEM (**barnes2014priority?**), whose priority-flood algorithms take almost no parameters and are convenient for exploratory work:

```
library(flowdem)
conditioned <- rast("data/chilwa_dem.tif") |>
  breach() |>
  fill(epsilon = TRUE)
dirs_fd <- dirs(conditioned, mode = "d8")
acc_fd <- accum(dirs_fd, mode = "d8")
```

Two differences matter when choosing between them.

The parameter surface. `flowdem::breach()` takes no depth constraint, so it carves without limit. On this DEM that produces elevation changes an order of magnitude larger than the constrained WhiteboxTools run, across a wider extent. Neither is right in general, but only one lets a project state what it did.

The reporting burden. `fill(epsilon = TRUE)` adds an infinitesimal gradient to flat ground, which is the same job `flat_increment` does above. A method statement that says “depressions were filled” without naming the algorithm, the depth constraint and the flat increment does not describe a reproducible procedure, and two implementations of “the same” conditioning step can differ by hundreds of metres on ground like this.

4.8 Exercises

1. Report the relief of the Chilwa DEM, the difference between its maximum and minimum elevation. Derive slope and describe where the steepest ground lies relative to the lake.
2. Recompute the delineated catchment area from the raw cell count, using the one-cell-one-hectare shortcut, and compare it against the `cellSize()` figure in the text. By what percentage does the shortcut err, and in which direction?
3. Rerun the breaching step with `max_depth = 10` instead of 3. How many more cells change, and does the lake basin survive? Explain what a project would lose by conditioning too aggressively.
4. Lower the stream extraction threshold from 1,000 cells to 100. How does the Strahler order distribution change, and which order now corresponds to the channels that HydroRIVERS maps?
5. (*Applied.*) A project claims a 500 square kilometre seasonal wetland. Using the rating curve, state the stage that claim implies and the stage error that would move the area by ten per cent. Identify

1220 the single assumption that most affects your answer, and say what evidence you would require to
1221 close it.

1222 5. LiDAR and Point Clouds

1223 5.1 Point clouds

1224 Airborne laser scanning measures the forest in three dimensions. The sensor fires pulses and
1225 times their returns, and a single pulse passing through a canopy produces several: the first off
1226 the treetops, later ones off branches and understorey, the last off the ground. The result is a point
1227 cloud of millions of x, y, z coordinates from which both the ground surface and the vegetation
1228 above it can be reconstructed.

1229 The R package is `lidR` (**lidR?**), which reads the LAS and LAZ formats laser data ships in. It bundles
1230 sample tiles, so this chapter runs on real airborne data without downloading anything:

```
1231 library(lidR)
1232 library(terra)
1233 library(sf)
1234
1235 topo <- readLAS(system.file("extdata", "Topography.laz", package = "lidR"),
1236                 filter = "-drop_z_below 0")
1237
1238 topo
1239 #> class      : LAS (v1.2 format 1)
1240 #> memory     : 3.6 Mb
1241 #> extent     : 273357.1, 273642.9, 5274357, 5274643 (xmin, xmax, ymin, ymax)
1242 #> coord. ref.: NAD83(CSRS) / MTM zone 7
1243 #> area       : 68728 m²
1244 #> points     : 73.4 thousand points
1245 #> type       : airborne
1246 #> density    : 1.07 points/m²
1247 #> density    : 0.78 pulses/m²
```

1247 Two properties govern everything that follows. Point density sets the finest structure that can be
1248 resolved, and the classification field records what each return is assumed to have hit. The LAS
1249 standard reserves class 2 for ground and class 1 for unclassified, and the numbers matter
1250 because every later step keys off them:

```
1251 data.frame(
1252   points    = npoints(topo),
1253   area_ha   = round(as.numeric(st_area(topo)) / 1e4, 2),
1254   density   = round(density(topo), 2),
1255   z_min     = round(min(topo$Z), 1),
1256   z_max     = round(max(topo$Z), 1)
1257 )
```

points	area_ha	density	z_min	z_max
73403	6.87	1.07	789	829.8

```
1258
1259 table(topo$Classification)
1260 #>
```

```

1261 #>      1      2      9
1262 #> 61347 8159 3897

```

Elevations here are hundreds of metres above the datum, not heights above ground. That distinction is the subject of most of this chapter.

5.2 Ground classification

Separating returns that hit soil from those that hit low vegetation is the delicate step, and everything downstream inherits its errors. Two algorithms dominate.

Cloth simulation filtering (Zhang et al 2016) inverts the point cloud and drapes a simulated cloth over it; the cloth settles onto the ground and the points it touches are classified. Progressive morphological filtering (Zhang et al 2003) instead opens the surface with windows of increasing size, removing anything that rises too fast over too short a distance.

Stripping the existing classification and running both makes the comparison honest:

```

1273 raw <- topo
1274 raw@data$Classification <- 0L
1275
1276 t_csf <- system.time(
1277   g_csf <- classify_ground(raw, csf(sloop_smooth = TRUE, 0.5, 1))
1278 )
1279 t_pmf <- system.time(
1280   g_pmf <- classify_ground(raw, pmf(ws = c(3, 6, 9), th = c(0.5, 1, 1.5)))
1281 )
1282
1283 data.frame(
1284   algorithm = c("CSF", "PMF"),
1285   ground    = c(sum(g_csf$Classification == 2L), sum(g_pmf$Classification == 2L)),
1286   seconds   = round(c(t_csf[["elapsed"]], t_pmf[["elapsed"]]), 2)
1287 )

```

algorithm	ground	seconds
CSF	19789	3.19
PMF	25744	1.09

```

1288
1289 round(100 * mean((g_csf$Classification == 2L) == (g_pmf$Classification == 2L)), 1)
1290 #> [1] 90.7

```

The two agree on about nine returns in ten and disagree on the tenth, with PMF admitting substantially more points as ground. Neither is a reference standard. The disagreement is concentrated where it always is, under dense low vegetation and on steep breaks of slope, and it propagates directly into terrain height and therefore into every canopy height derived from it. A project that does not state which algorithm produced its terrain model, and with what parameters, has not described a reproducible method.

5.3 Noise removal

Laser data contains returns that are physically impossible: birds, low cloud, multipath reflections, and points hundreds of metres below the ground. Statistical outlier removal flags a point when its mean distance to its k nearest neighbours exceeds m standard deviations of that statistic over the cloud:

```
noisy <- classify_noise(g_csf, sor(k = 10, m = 3))
sum(noisy$Classification == LASNOISE)
#> [1] 902
clean <- filter_poi(noisy, Classification != LASNOISE)
```

A few hundred points out of seventy thousand. The count is small and the consequence is not, because a single spurious high return becomes a spurious treetop, and a single spurious low return drags the interpolated ground surface down with it across the whole neighbourhood.

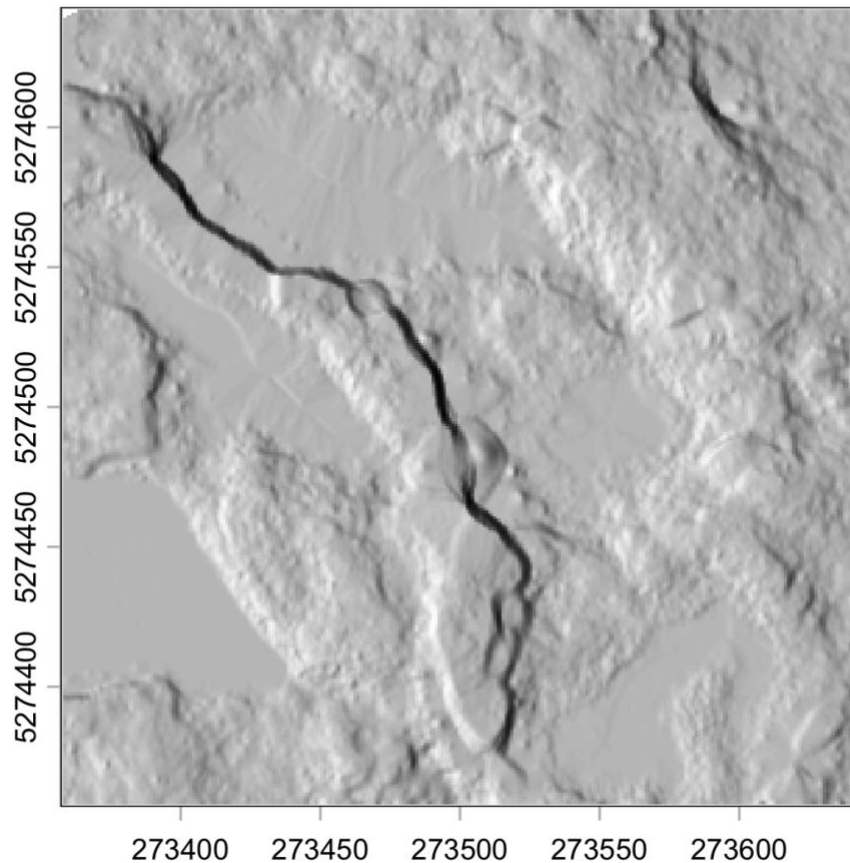
5.4 Terrain models

Interpolating the ground returns onto a grid gives the digital terrain model. `lidR` offers inverse-distance weighting on the k nearest neighbours, a triangulated surface, and kriging. The first is fast and robust at moderate density:

```
dtm <- rasterize_terrain(clean, res = 1, knnidw(k = 10, p = 2, rmax = 50))
global(dtm, range, na.rm = TRUE)
```

	X1	X2
Z	789.0362	814.6387

```
hs <- shade(terrain(dtm, "slope", unit = "radians"),
            terrain(dtm, "aspect", unit = "radians"),
            angle = 40, direction = 270)
plot(hs, col = grey(0:100 / 100), legend = FALSE, mar = c(2, 2, 2, 2))
```



Hillshaded terrain model interpolated from the classified ground returns.

5.5 Height normalisation

Biomass relates to height above ground, not height above sea level. Normalising subtracts the terrain from every point, so a return four metres up a slope at 800 m and one four metres up a slope at 810 m become the same number:

```
norm <- normalize_height(clean, dtm)
data.frame(
  z_min = round(min(norm$Z), 2),
  z_max = round(max(norm$Z), 2)
)
```

z_min	z_max
-3.14	22.1

The check on a normalisation is the residual on the ground returns themselves. They should centre on zero, because the surface was interpolated through them:

```

1335 g <- filter_ground(norm)$Z
1336
1337 data.frame(
1338   mean_m = round(mean(g), 3),
1339   sd_m    = round(sd(g), 3),
1340   q95_abs = round(unnamed(quantile(abs(g), 0.95)), 3)
1341 )

```

mean_m	sd_m	q95_abs
-0.001	0.089	0.193

1342 Centred, with a spread of a few centimetres. A mean that departs from zero indicates a systematic
 1343 terrain bias, and a spread of metres rather than centimetres indicates that ground classification
 1344 has admitted vegetation.

1345 5.5.1 Normalisation trap

1346 The order of operations here is not interchangeable, and getting it wrong fails silently. Interpolating
 1347 a terrain model from an already-normalised cloud produces this:

```

1348 dtm_wrong <- rasterize_terrain(norm, res = 1, knnidw(k = 10, p = 2, rmax = 50))
1349 global(dtm_wrong, range, na.rm = TRUE)

```

	X1	X2
Z	-0.558	0.585

1350 Twenty-five metres of relief have become one metre of residual noise about zero. The arithmetic
 1351 is correct and the result is useless: the ground returns of a normalised cloud are already at height
 1352 zero, so interpolating them recovers the interpolation residual rather than the terrain.

1353 Nothing errors. The object is a valid raster of the right extent, resolution and projection, and it
 1354 plots without complaint. Slope computed from it is noise, aspect is undefined, and any model
 1355 carrying terrain covariates loses them entirely while continuing to fit, predict and report. The rule
 1356 is that terrain models are built before normalisation, from the classified but un-normalised cloud,
 1357 and there is no exception.

1358 5.6 Canopy models

1359 The canopy height model is the surface of the vegetation over the terrain. Built from a normalised
 1360 cloud it is a direct rasterisation of the highest return in each cell, and the difficulty is the cells that
 1361 receive no return at all:

```

1362 mc <- readLAS(system.file("extdata", "MixedConifer.laz", package = "lidR"),
1363               filter = "-drop_z_below 0")
1364
1365 chm_p2r <- rasterize_canopy(mc, res = 0.5, p2r(subcircle = 0.2))
1366 chm_pitfree <- rasterize_canopy(mc, res = 0.5, pitfree(subcircle = 0.2))

```

```

1367
1368 data.frame(
1369   algorithm = c("points-to-raster", "pit-free"),
1370   empty     = c(global(is.na(chm_p2r), "sum")[1, 1],
1371                 global(is.na(chm_pitfree), "sum")[1, 1]),
1372   cells     = ncell(chm_p2r)
1373 )

```

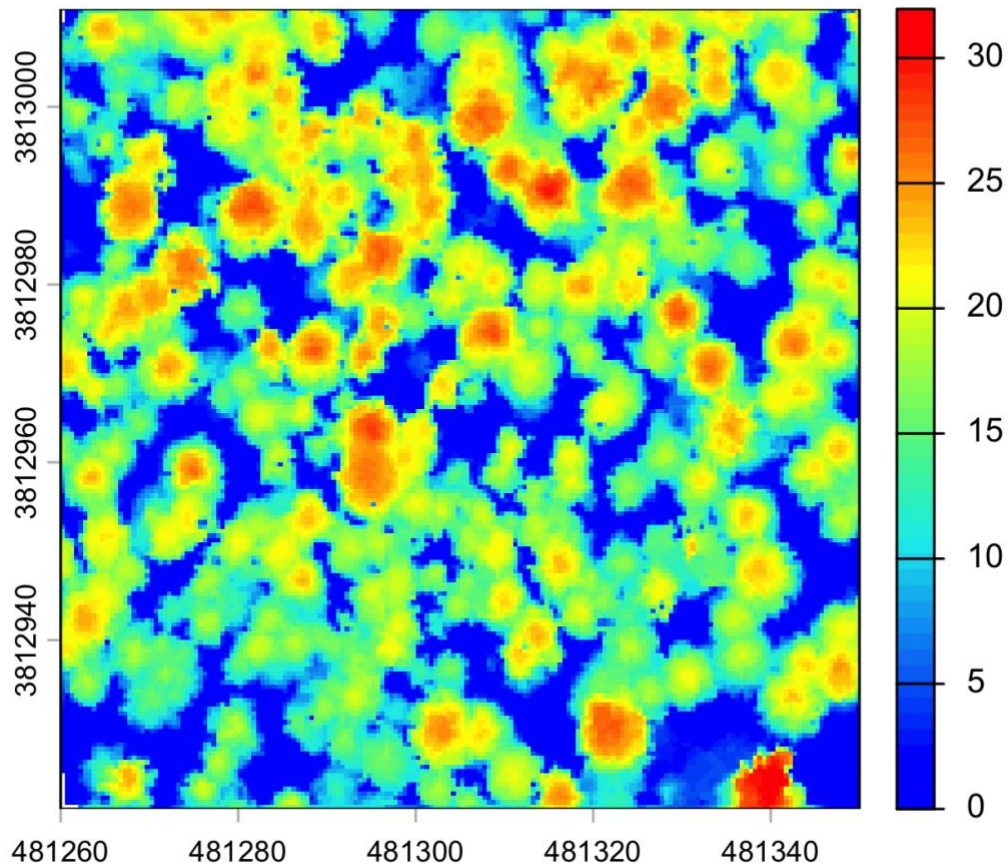
algorithm	empty	cells
points-to-raster	875	32400
pit-free	14	32400

1374 Points-to-raster leaves several hundred empty cells: the pits that appear wherever a pulse
1375 penetrated to the ground between crowns. The pit-free algorithm
1376 (**khosravipour2014generating?**) builds a set of triangulated surfaces at successive height
1377 thresholds and takes their maximum, which closes almost all of them. Pits matter because local-
1378 maximum tree detection reads a pit as a crown edge and splits one tree into two.

```

1379 plot(chm_pitfree, col = height.colors(50), mar = c(2, 2, 2, 4))

```



Pit-free canopy height model of the mixed conifer stand.

5.7 Tree detection

Each treetop is a local maximum: a cell higher than every other cell within some window. The window size is the entire method, and it is not a detail.

5.7.1 Window size

```
counts <- sapply(c(3, 5, 11), function(w) nrow(locate_trees(mc, lmf(ws = w))))
area_ha <- as.numeric(st_area(mc)) / 1e4

data.frame(
  window_m = c(3, 5, 11),
  trees     = counts,
  stems_ha  = round(counts / area_ha)
)
```

window_m	trees	stems_ha
3	297	368
5	177	219

window_m	trees	stems_ha
11	65	81

A three-metre window finds nearly five times as many trees as an eleven-metre window over the same 0.8 hectares. Neither count is a measurement of the stand. The small window splits broad crowns into multiple maxima; the large one merges adjacent trees and detects only dominants. The correct window depends on crown size, which depends on height, which is why fixed windows are a starting point rather than an answer.

5.7.2 Variable windows

Making the window a function of height is the standard refinement. A linear function with a floor is the usual form, and the coefficients are stand-specific:

```
wf <- function(x) pmax(0.07 * x + 0.6, 0.5)
ttops <- locate_trees(mc, lmf(wf), uniqueness = "bitmerge")
data.frame(
  trees = nrow(ttops),
  stems_ha = round(nrow(ttops) / area_ha)
)
```

trees	stems_ha
1707	2115

This is a calibrated function taken from a different forest, a regenerating burn, and applied here it returns an implausible stem density: an order of magnitude above what a mixed conifer stand carries. The window it opens at typical heights in this stand is a couple of metres, far narrower than the crowns, so it fragments them.

The lesson is not that variable windows are wrong. It is that a window function is a calibration, valid for the stand structure it was fitted to, and transferring one between forests without refitting produces confident nonsense. Detected stem counts must be checked against field plots before they are used.

The `uniqueness = "bitmerge"` argument is a separate and mandatory precaution. When a survey is processed tile by tile with overlapping buffers, sequential tree identifiers collide between tiles and the same tree is counted twice. Bitmerge derives the identifier from the tree's coordinates instead, so the duplicate resolves to one tree.

5.8 Crown segmentation

Detection gives points. Segmentation assigns every return to a tree, which is what yields crown area and per-tree metrics. Dalponte and Coomes' algorithm (Dalponte et al 2016) grows regions outward from each detected top, bounded by height thresholds relative to the seed:

```
ttops_fixed <- locate_trees(mc, lmf(ws = 5))
ttops_fixed$treeID <- seq_len(nrow(ttops_fixed))

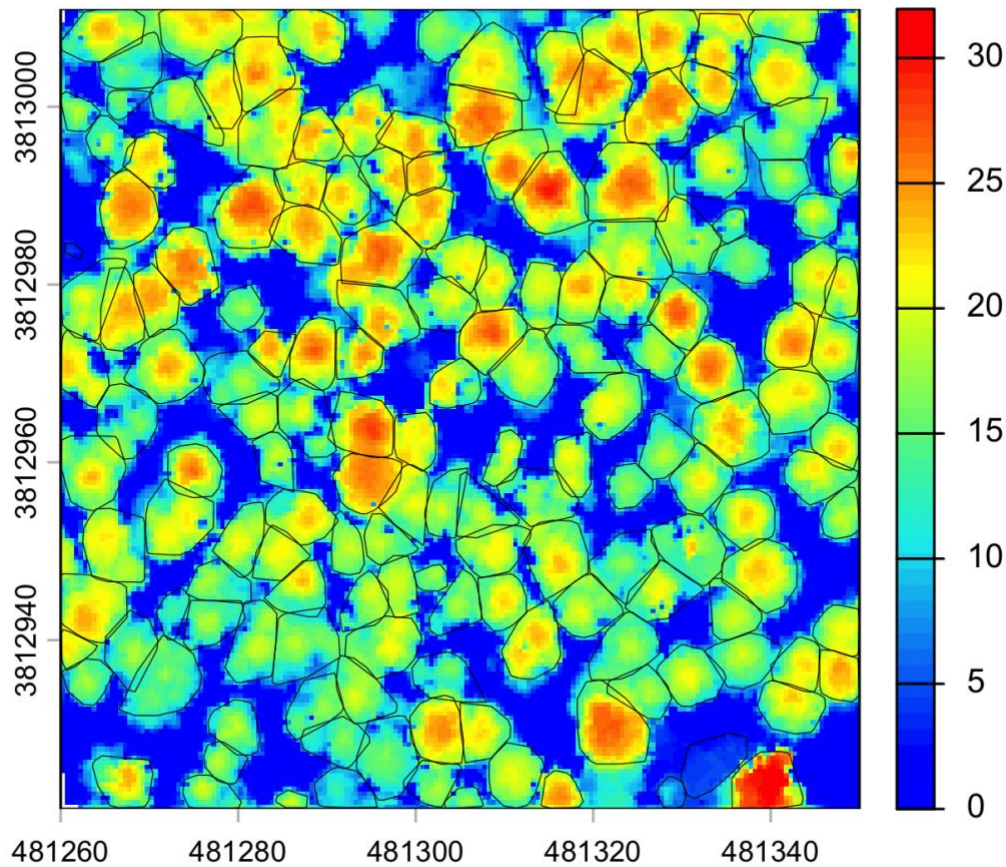
seg <- segment_trees(mc, dalponte2016(chm_pitfree, ttops_fixed))
crowns <- crown_metrics(seg, func = .stdtreemetrics, geom = "convex")

data.frame(
  crowns = nrow(crowns),
  median_area_m2 = round(median(crowns$convhull_area), 1),
  median_height_m = round(median(crowns$Z), 1)
)
```

crowns	median_area_m2	median_height_m
165	32.7	22.2

The `treeID` reset on the line above is not cosmetic. Identifiers produced by `uniqueness = "bitmerge"` are 64-bit integers derived from coordinates, and `dalponte2016()` uses the identifier to index its output raster. Passing bitmerge identifiers to it produces an error or a corrupted segmentation depending on version. Reset them to a simple sequence within a single tile.

```
plot(chm_pitfree, col = height.colors(50), mar = c(2, 2, 2, 4))
plot(st_geometry(crowns), add = TRUE, border = "black", lwd = 0.4)
```



Segmented crown polygons over the canopy height model.

5.9 Dominant height

Stand height is a mensurational variable with a specific definition, usually the mean height of the tallest trees per unit area rather than the mean of all trees. The raster equivalent is an upper percentile of canopy height aggregated to a coarser cell:

```
q95 <- function(x, ...) quantile(x, 0.95, na.rm = TRUE)
chm_dom <- aggregate(chm_pitfree, fact = 2, fun = q95)

v <- terra::extract(chm_dom, vect(tttops))[, 2]
ok <- !is.na(v)

data.frame(
  r = round(cor(tttops$Z[ok], v[ok]), 3),
  rmse = round(sqrt(mean((tttops$Z[ok] - v[ok])^2)), 2),
  bias = round(mean(tttops$Z[ok] - v[ok]), 2)
)
```

r	rmse	bias
0.884	3.26	-0.99

The raster surface tracks the detected tree heights closely but sits about a metre high, the negative bias, because a 95th percentile over a cell containing several crowns returns something near the tallest of them rather than the mean. Upper percentiles are preferred as biomass predictors for exactly this reason: they are dominated by the large trees that carry most of the biomass, and they are insensitive to the understorey that laser data sees poorly.

5.10 Area-based inventory

Individual-tree methods are appealing and rarely how operational inventory is done. The area-based approach instead summarises the point cloud into metrics on a grid matching the field plot size, regresses plot-measured biomass on those metrics, and applies the model wall to wall.

The metrics are distributional summaries of return heights in each cell:

```
mega <- readLAS(system.file("extdata", "Megaplot.laz", package = "lidR"),
  filter = "-drop_z_below 0")

mega <- normalize_height(classify_ground(mega, csf(sloop_smooth = TRUE, 0.5, 1)),
  knnidw())

m <- pixel_metrics(mega, ~list(
  zmax = max(Z),
  zmean = mean(Z),
  zsd = sd(Z),
  zq75 = quantile(Z, 0.75),
  zq95 = quantile(Z, 0.95),
  cover = sum(Z > 2) / length(Z)
), res = 20)

names(m)
#> [1] "zmax" "zmean" "zsd" "zq75" "zq95" "cover"
round(as.data.frame(global(m, "mean", na.rm = TRUE)), 2)
```

	mean
zmax	20.08
zmean	11.35
zsd	5.07
zq75	15.07
zq95	17.93
cover	0.74

Every cell now carries a description of its vertical structure, and this is the form in which laser data enters a biomass model. Height percentiles and cover fraction carry most of the signal; the mean carries least, because it is dragged down by ground and understorey returns.

The reason this matters for verification is coverage. A canopy metric exists for every cell in the survey, where field plots sample a fraction of a per cent of the area, so laser data turns a few

dozen plots into a wall-to-wall map. That leverage is also the risk: the model is fitted on the plots and applied everywhere, so its error applies everywhere too, and the uncertainty chapter treats how that error is estimated and carried into a reported number.

Documentation requirements

Laser-derived inventory is reproducible in principle and frequently not in practice. A verifier should require, in writing, the items this chapter has shown to change the answer.

The acquisition specification: sensor, date, pulse density, scan angle, and stated vertical accuracy. Density sets what can be resolved and the date sets what was standing.

The ground classification algorithm and its parameters. The two algorithms compared above differed by around ten per cent of returns on the same tile, and that difference moves the terrain surface and therefore every canopy height.

The normalisation order. A terrain model built after normalisation is flat, and nothing in the output announces it.

The detection method and its calibration. A window function is fitted to a stand structure. The one used above, transferred from another forest, overstated stem density by roughly an order of magnitude while producing no warning of any kind.

The plot-to-metric model and its validation. Predicted biomass is a model output, not a measurement, and a map without a cross-validated error statistic carries no defensible uncertainty.

5.11 Exercises

1. Rebuild the canopy height model at 1 m rather than 0.5 m resolution and recount the empty cells under both `p2r()` and `pitfree()`. Explain why coarsening the grid reduces pits and what it costs.
2. Run tree detection with fixed windows of 3, 5, 7 and 11 m and plot detected stems per hectare against window size. At what window does the count stabilise, and what does the shape of that curve tell you about mean crown diameter in this stand?
3. Refit the variable window function so that the detected stem density falls within a plausible range for a mixed conifer stand. State the coefficients you chose and the stand assumption that justifies them.
4. Compute the 25th, 50th, 75th and 95th height percentiles from the pixel metrics above. Explain why upper percentiles outperform the mean as biomass predictors, referring to where biomass sits in the height distribution.

5. You have laser coverage of 10,000 ha and field plots on 30 ha. Describe, in steps, how you would calibrate a metric-based biomass model and produce a wall-to-wall estimate, naming the two largest sources of error and where each enters.

6. Disturbance and Risk

6.1 Disturbance screening

A verifier's central remote-sensing task is to find where the forest changed between two dates, and to decide what caused it. The screening is not a substitute for plot remeasurement or the developer's monitoring report. It is an independent layer used to direct attention: to find the clearing nobody declared, and to test whether the loss a project did declare has the extent and the cause it claims.

The method is spectral index differencing. Take an index at two dates, subtract, and threshold the difference. Everything difficult is in the last step, and this chapter is mostly about the last step.

The working data are the 2022 Sentinel-2 land-cover map of Rondonia and one NDVI composite over the same window, both introduced in the raster chapter. The map carries labelled disturbance classes, which makes it possible to measure how well an index threshold actually separates cleared ground from forest rather than assuming it does:

```
library(terra)
library(sf)

sf_use_s2(FALSE)

cls <- rast("data/rondonia_class_2022.tif")
ndvi <- rast("data/rondonia_ndvi_2022-07-16.tif") / 1e4
leg <- read.csv("data/rondonia_legend.csv")

window <- project(as.polygons(ext(ndvi), crs = crs(ndvi)), crs(cls))
cw <- crop(cls, window) |> project(ndvi, method = "near")
names(cw) <- "class"; names(ndvi) <- "ndvi"
```

The NDVI rasters store the index multiplied by ten thousand, which is why the division appears above. Reading them raw yields indices in the thousands and every threshold in this chapter silently fails to fire.

6.2 Spectral indices

Three indices carry most operational disturbance screening, and they answer different questions.

$$\text{NDVI} = \frac{\text{NIR} - \text{Red}}{\text{NIR} + \text{Red}} \quad \text{NDMI} = \frac{\text{NIR} - \text{SWIR}_1}{\text{NIR} + \text{SWIR}_1} \quad \text{NBR} = \frac{\text{NIR} - \text{SWIR}_2}{\text{NIR} + \text{SWIR}_2}$$

1556 NDVI tracks greenness and responds to canopy removal. NDMI tracks canopy moisture and
 1557 responds to stress before the canopy is gone, which makes it the index for insect attack and
 1558 drought. NBR contrasts near-infrared against the longer shortwave band where char absorbs
 1559 strongly, so it responds to fire.

1560 The sign conventions differ and mixing them up inverts the map:

1561 $\Delta\text{NDVI} = \text{pre} - \text{post}$ $\Delta\text{NDMI} = \text{pre} - \text{post}$ $\Delta\text{NBR} = \text{post} - \text{pre}$

1562 Positive dNDVI and dNDMI mean loss. Positive dNBR means burned, following the convention of
 1563 the Monitoring Trends in Burn Severity programme. There is no deep reason for the
 1564 inconsistency; it is historical, and it has to be checked against the source of any published
 1565 threshold before that threshold is applied.

1566 6.3 Threshold selection

1567 Published severity breaks exist for each index. The ones in operational use, drawn from the burn
 1568 severity literature and from forest health screening practice, are these:

Index	Low	Moderate	High
dNDVI	0.10 to 0.20	0.20 to 0.35	above 0.35
dNDMI	0.15 to 0.30	0.30 to 0.45	above 0.45
dNBR	0.10 to 0.27	0.27 to 0.44	above 0.44

1569 These are defaults, not constants. Treating them as constants is the most common failure in
 1570 disturbance screening, and the way to see why is to measure what a threshold actually catches.

1571 6.3.1 Class separation

1572 The land-cover map labels each pixel, so the NDVI distribution of each class can be read directly.
 1573 This is the empirical basis a threshold ought to rest on:

```

1574 d <- as.data.frame(c(ndvi, cw), na.rm = TRUE)
1575
1576 tab <- do.call(rbind, lapply(leg$id, function(i) {
1577   v <- d$ndvi[d$class == i]
1578   if (!length(v)) return(NULL)
1579   data.frame(label = leg$label[leg$id == i],
1580             forest = leg$forest[leg$id == i],
1581             n      = length(v),
1582             q05    = round(quantile(v, 0.05), 3),
1583             median = round(median(v), 3),
1584             q95    = round(quantile(v, 0.95), 3))
1585   )))
1586 tab

```

label	forest	n	q05	median	q95
Clear_Cut_Bare_Soil	FALSE	201345	0.307	0.509	0.832
Clear_Cut_Burned_Area	FALSE	4516	0.218	0.580	0.811
Mountainside_Forest	TRUE	15	0.833	0.851	0.867
Forest	TRUE	710914	0.783	0.842	0.879
Riparian_Forest	TRUE	115764	0.696	0.851	0.884
Clear_Cut_Vegetation	FALSE	41805	0.650	0.791	0.873
Water	FALSE	163484	-0.456	-0.379	0.206
Seasonally_Flooded	FALSE	30256	-0.035	0.073	0.562
Wetland	FALSE	166377	0.720	0.844	0.882

1587 Three things in that table decide everything downstream.

1588 Intact forest is tight: a median around 0.84 with the fifth percentile still above 0.78. Cleared ground
1589 with bare soil sits far below it, around 0.51. Those two are separable and any sensible threshold
1590 finds them.

1591 Clear cut with regrowth is not separable. Its median sits near 0.79, inside the forest distribution,
1592 because two or three years after clearing the vegetation that returns is green. An index threshold
1593 cannot distinguish recovering clearing from forest, and a project whose deforestation is a few
1594 years old will not be caught by one.

1595 Wetland has a median around 0.84, indistinguishable from forest, which is why water masking is
1596 applied before thresholding rather than after.

1597 Measuring the trade-off directly makes the cost of each published break explicit:

```
1598 forest_ndvi <- d$ndvi[d$class %in% leg$id[leg$forest]]
1599 cleared    <- d$ndvi[d$class %in% c(1, 2, 6)]
1600 baseline   <- median(forest_ndvi)
1601
1602 sens <- do.call(rbind, lapply(c(0.10, 0.15, 0.20, 0.30), function(t) {
1603   cut <- baseline - t
1604   data.frame(dNDVI_threshold = t,
1605             ndvi_below      = round(cut, 3),
1606             pct_cleared_caught = round(100 * mean(cleared < cut), 1),
1607             pct_forest_flagged = round(100 * mean(forest_ndvi < cut), 1))
1608   }))
1609 sens
```

dNDVI_threshold	ndvi_below	pct_cleared_caught	pct_forest_flagged
0.10	0.743	75.7	2.5
0.15	0.693	68.2	1.4

dNDVI_threshold	ndvi_below	pct_cleared_caught	pct_forest_flagged
0.20	0.643	60.8	0.9
0.30	0.543	46.7	0.5

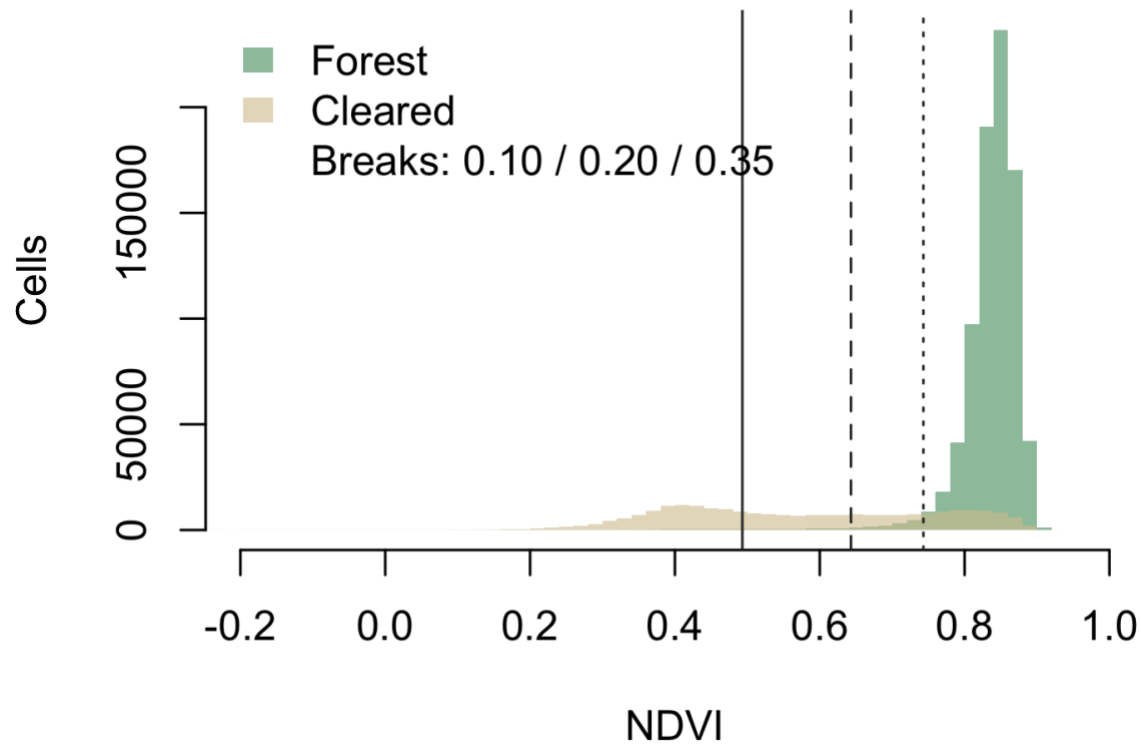
1610 Read the third column. The “Low severity” break of 0.10 catches about three quarters of genuinely
1611 cleared ground. The “Moderate” break of 0.20 catches three fifths. The “High severity” break of
1612 0.35, extrapolating past the last row, catches well under half. Every one of those misses is omitted
1613 deforestation, and omission understates emissions, which is the direction that concerns an auditor
1614 most.

1615 The fourth column is the other side of the trade. Tightening the threshold buys a falling false-
1616 positive rate on forest, from 2.5 per cent down to 0.5. That is a real gain, and it is why the tight
1617 thresholds are popular: they produce clean maps. Clean is not the same as correct.

1618 6.3.2 Histogram diagnostic

1619 Since defaults cannot be trusted, the threshold has to be read off the data. The diagnostic is the
1620 histogram of the difference image, and it supports three readings.

```
1621 hist(forest_ndvi, breaks = 60, xlim = c(-0.2, 1), col = "#1E844980",
1622      border = NA, main = "", xlab = "NDVI", ylab = "Cells")
1623 hist(cleared, breaks = 60, col = "#D7C49C99", border = NA, add = TRUE)
1624 abline(v = baseline - c(0.10, 0.20, 0.35), lty = c(3, 2, 1), col = "grey20")
1625 legend("topleft", bty = "n",
1626      legend = c("Forest", "Cleared", "Breaks: 0.10 / 0.20 / 0.35"),
1627      fill = c("#1E844980", "#D7C49C99", NA), border = NA)
```



1628
 1629 *NDVI distribution for forest and for cleared classes, with the published dNDVI breaks marked relative to*
 1630 *the forest median.*

1631 A unimodal distribution centred on zero with a narrow tail means noise and phenology only, and
 1632 the defaults can stand because nothing is there to find.

1633 A bimodal distribution with a gap above about 0.15 means a real disturbance signal, and the low
 1634 break belongs in the gap rather than on a default.

1635 A long unimodal right tail with no gap means the composite is contaminated, by cloud shadow,
 1636 senescence, or view-angle effects. The correct response is to rebuild the composite with a tighter
 1637 cloud filter or a shifted date window, not to classify it.

1638 **6.4 Composite construction**

1639 The threshold problem is downstream of a compositing problem, and three practical rules do more
 1640 for accuracy than any amount of threshold tuning.

Use enough scenes. A five-day revisit over a two-month window yields eight or more cloud-filtered observations per pixel, and a median composite is unstable below about four.

Choose the season for the question. Composites built in the dormant or senescent season carry a canopy that is naturally dry and naturally less green, which inflates dNDMI enormously. In one verification the default moisture breaks applied to an October to December composite flagged 38 per cent of the project area as disturbed; rebuilding the same analysis on a July to September window collapsed that to 4 per cent, all of it co-located with reported beetle survey polygons. The 34-point difference was seasonal phenology, not disturbance.

Watch the processing baseline. Sentinel-2 surface reflectance carries a digital number offset introduced in January 2022. Mixing scenes across that boundary without harmonising propagates a spurious index difference of around 0.04 across the entire project area, which is a third of the way to the low severity break before any real change has occurred.

6.5 Attribution

Detecting loss is half the task. Deciding its cause determines whether it is a reversal the project must account for, a natural disturbance that may be excluded, or a leakage signal on land the proponent controls elsewhere.

No single index answers it. The cross-check does:

Signal	Reading	Required corroboration
High dNDVI, dNBR clean	Harvest, blowdown or clearing	Edge geometry, harvest records
High dNDVI, high dNBR	Fire	Official burn perimeters
High dNDMI alone	Moisture stress	Drought index or pest survey

The third row is the one most often over-read. Moisture stress on its own is not disturbance, and classifying it as such without corroboration produces the false positives described above.

6.5.1 Edge geometry

The most reliable attribution cue is the shape of the patch. Anthropogenic clearing follows property and operational boundaries, so it has straight edges, right angles, and fills its bounding box. Natural disturbance follows wind, terrain and hydrology, so it is curvilinear and amorphous.

That is a testable claim, and the labelled map lets it be tested. Rectangularity, the ratio of a patch's area to the area of its bounding box, formalises it:

```

1666 rectangularity <- function(class_ids, min_cells = 40) {
1667   m <- ifel(cw %in% class_ids, 1, NA)
1668   p <- as.polygons(patches(m, directions = 8), dissolve = TRUE) |> st_as_sf()
1669   p$area <- as.numeric(st_area(p))
1670   p <- p[p$area >= min_cells * prod(res(cw)), ]
1671   bb <- vapply(seq_len(nrow(p)), function(i) {
1672     b <- st_bbox(p[i, ]); as.numeric((b$xmax - b$xmin) * (b$ymax - b$ymin))
1673   }, numeric(1))
1674   p$area / bb
1675 }
1676
1677 anthro <- rectangularity(c(1, 2))
1678 natural <- rectangularity(c(8, 9))
1679
1680 data.frame(
1681   origin      = c("Clear cut (bare, burned)", "Wetland, seasonally flooded"),
1682   patches     = c(length(anthro), length(natural)),
1683   median_rect = round(c(median(anthro), median(natural)), 3)
1684 )

```

origin	patches	median_rect
Clear cut (bare, burned)	110	0.560
Wetland, seasonally flooded	142	0.539

1685 The cleared patches sit marginally above the natural ones, and the difference is far too small to
 1686 separate anything. As a summary statistic over all patches, the edge geometry rule fails here.

1687 That result is worth more than a confirmation would have been, because it locates what the rule
 1688 actually is. Three things defeat the aggregate test. Patches delineated from a classified map
 1689 inherit the classifier's edges, which are already blocky at 20 m whatever produced them.
 1690 Amazonian wetlands follow river courses and are themselves linear and compact. And
 1691 rectangularity collapses a shape to one number, discarding the thing an analyst is actually
 1692 reading, which is whether the corners are square.

1693 The rule is a visual heuristic applied to individual polygons by someone looking at them, and it is
 1694 a good one in that role: a right-angled corner on a loss patch is strong evidence of machinery. It
 1695 is not a classifier, it does not survive being averaged, and a screening workflow that automates it
 1696 into a threshold will find nothing. Geometry raises or lowers a prior on a specific polygon. Records
 1697 settle it.

1698 6.6 Risk mapping

1699 Where disturbance is *likely* is a different question from where it happened, and it is the one that
 1700 drives a baseline. Under VM0048 and the associated allocation tooling, a risk map distributes an
 1701 expected quantity of future loss across the landscape in proportion to modelled pressure.

The covariates are distances. Pressure falls with distance from existing clearing, from roads, from settlements and from water, and rises on flat ground. Two of those are computable from the map itself:

```
forest_mask <- ifel(cw %in% leg$id[leg$forest], 1, NA)
edge <- boundaries(forest_mask, inner = FALSE)
dist_edge <- distance(edge) / 1000

cleared_mask <- ifel(cw %in% c(1, 2, 6), 1, NA)
dist_clear <- distance(cleared_mask) / 1000

normalise <- function(x) {
  r <- unlist(global(x, range, na.rm = TRUE))
  (x - r[1]) / (r[2] - r[1])
}

risk_edge <- 1 - normalise(dist_edge)
risk_clear <- 1 - normalise(dist_clear)
```

Both are inverted, because risk rises as distance falls. Forgetting the inversion produces a map that is exactly wrong and looks entirely plausible.

6.6.1 Index weights

The covariates are combined as a weighted sum. A published implementation uses these weights:

```
weights <- c(edge = 0.2, roads = 0.2, places = 0.2,
             urban = 0.1, water = 0.1, slope = 0.1)
sum(weights)
#> [1] 0.9
```

They sum to 0.9, not 1.0. The consequence is not that the map is wrong: the index is rescaled to zero-one afterwards, and the allocation step below normalises by the zonal sum, so a constant factor cancels. The consequence is that the number cannot be audited. Nobody reading the index can tell whether the missing tenth is a dropped covariate, a typing error, or an intentional choice, and a verifier confronted with weights that do not sum to unity is entitled to reject the whole construction until it is explained.

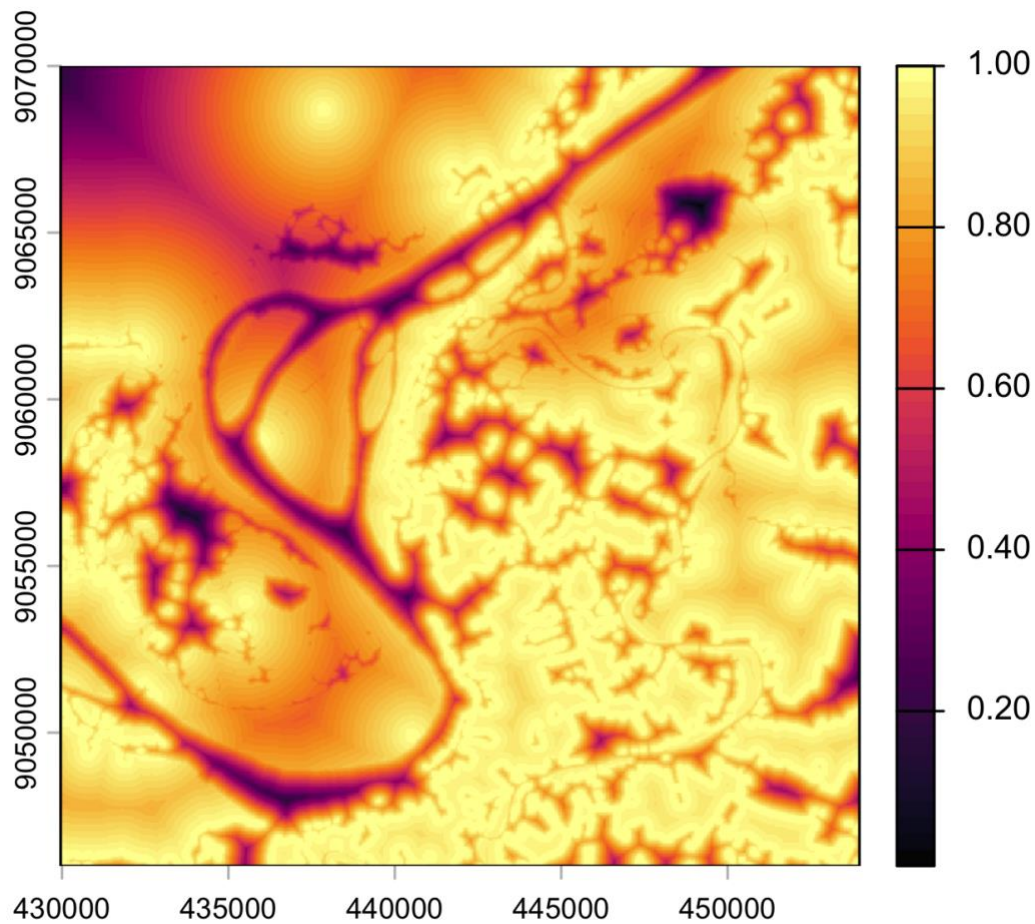
Weights should sum to one and each should be justified. With the two covariates available here:

```
risk <- 0.5 * risk_edge + 0.5 * risk_clear
risk <- normalise(risk)
names(risk) <- "risk"

global(risk, quantile, probs = c(0.05, 0.5, 0.95), na.rm = TRUE)
```

	X5.	X50.	X95.
risk	0.4986704	0.87953	0.9942256

```
plot(risk, col = hcl.colors(50, "Inferno"), mar = c(2, 2, 2, 4))
```



Deforestation risk index over the Rondonia window, with mapped forest outlined.

6.6.2 Risk allocation

The risk surface is relative, not absolute. It becomes a baseline only when a quantity of expected loss is distributed across it. The allocation rule assigns each pixel a share of its zone's expected annual loss in proportion to its own risk:

$$\text{AllocatedLoss}_{\text{pixel}} = \frac{\text{risk}_{\text{pixel}}}{\sum \text{risk}_{\text{zone}}} \times \text{AnnualLoss}_{\text{zone}}$$

In practice the fraction is computed as a per-zone factor and multiplied through, which is the same arithmetic in a form that rasterises:

```
annual_loss_ha <- 4654 # zone expected annual loss, hectares
risk_forest <- mask(risk, forest_mask)
risk_sum <- global(risk_forest, "sum", na.rm = TRUE)[1, 1]
loss_factor <- annual_loss_ha / risk_sum
allocated <- risk_forest * loss_factor
```

```
data.frame(
  risk_sum      = round(risk_sum),
  loss_factor   = signif(loss_factor, 3),
  allocated_total = round(global(allocated, "sum", na.rm = TRUE)[1, 1], 1)
)
```

risk_sum	loss_factor	allocated_total
719113	0.00647	4654

The allocated total reproduces the input by construction. That is the property to check first on any allocation: it is a redistribution, not a model of how much will be lost, and if the totals do not close the implementation is broken. Everything about *how much* comes from the historical rate, which the baseline chapter treats, and everything about *where* comes from the risk index above.

6.7 Leakage belts

Activity displaced outside a project boundary is leakage, and it is assessed on a belt drawn around the project. The rules are prescriptive: a belt of a stated width, then a sequence of exclusions for land where deforestation could not plausibly be displaced, or where it would be attributed elsewhere.

The exclusions typically remove ground within a set distance of existing road infrastructure, ground steeper than ten per cent, mapped wetlands, and protected areas in the stricter management categories. The slope rule is worth writing carefully, because it is a strict inequality:

```
slope_pct      <- tan(terrain(dem, "slope", unit = "degrees") * pi / 180) * 100
slope_invalid  <- slope_pct > 10      # strictly greater, not >=
```

The combination step is where implementations fail. Each mask produces a residual belt, the belt minus that exclusion, and the valid area is the ground that survives every exclusion. That is an intersection:

```
valid <- Reduce(st_intersection,
  list(belt_minus_roads, belt_minus_slope,
    belt_minus_wetland, belt_minus_protected))
```

Chaining `st_union()` across the four residuals instead, which is an easy slip because each residual is itself called a leakage area, readmits every excluded parcel: a pixel excluded by the slope mask survives in the roads residual, and the union restores it. The resulting belt is larger than it should be in a direction that suits the project. A verifier reproducing a leakage area should check the set operation before checking the arithmetic.

Standards divergence

Two families of requirement apply to the material in this chapter, and they rest on different philosophies. Treating them as interchangeable misrepresents both.

The Verra methodologies approach map quality through stratified area estimation. A map is accompanied by a sample drawn across strata, the sample yields an area-adjusted estimate with a confidence interval, and per-class accuracies must clear stated floors: for deforestation, user's and producer's accuracy of at least 70 per cent each, with forest above 90 and open canopy classes at 60 and 80. The map is not the estimate; the sample is. The uncertainty chapter develops the estimator.

The ACR improved forest management standards approach disturbance as screening rather than estimation. There is no confusion matrix requirement, because the output is not an area claim. Defensibility comes instead from a documented threshold justified by the histogram, corroboration across independent layers, and cross-reference to official fire perimeters and pest survey polygons. A finding directs a verifier to a location; the account of what happened there comes from records and from the field.

The practical consequence is that a threshold and a sample answer different questions. Do not report a screening result as an area estimate, and do not offer a stratified area estimate as evidence that a particular polygon was harvested.

6.8 Exercises

1. Recompute the threshold sensitivity table for breaks of 0.05 and 0.25. At what threshold does the proportion of cleared ground caught fall below half, and what does that imply for a project relying on the published high severity break?
2. Repeat the class separation table using only the clear cut with regrowth class against forest. Explain why no NDVI threshold separates them, and name the data source that would.
3. Compute rectangularity for the riparian forest class and compare it against both groups in the text. Riparian forest is natural but linear: what does that do to the edge geometry rule, and how would you amend the rule?
4. Rebuild the risk index weighting distance to existing clearing at 0.8 and distance to forest edge at 0.2. How much of the high risk area moves, and what assumption about deforestation behaviour does the reweighting encode?
5. (*Applied.*) A project reports 200 hectares of avoided deforestation, an overall map accuracy of 78 per cent, and a disturbance screen showing no unreported loss. State what each of those three claims does and does not support, and list the per-class accuracies you would require before accepting the area.

7. Allometry and Biomass

7.1 Indirect measurement

Every carbon number in this book rests on a step that is easy to miss. Nobody weighs a standing tree. Field crews measure diameter, sometimes height, and a statistical model converts those measurements into biomass. That model is an allometric equation, and it was fitted somewhere else, on other trees, by someone who cut them down and weighed them.

This is the largest single source of error in most forest carbon estimates, and it is the one least visible in a project report, because the equation appears as a line of code rather than a measurement with an error bar. A verifier who checks the field data and the arithmetic but never asks where the equation came from has checked the easy part.

The chapter is about that choice: what an allometric equation is, how to select one, what it costs to select badly, and how to carry the resulting uncertainty forward.

7.2 Equation form

Tree biomass scales with diameter as a power law. A tree twice as wide is far more than twice as heavy, because mass grows roughly with the square of diameter for cross-sectional area and again with height. The canonical form is

$$AGB = a D^b$$

with D the diameter at breast height and b typically between 2.3 and 2.6. Taking logarithms makes it linear and, more importantly, stabilises the variance, because absolute scatter grows with tree size while proportional scatter stays roughly constant:

$$\ln(AGB) = \ln(a) + b \ln(D) + \varepsilon$$

Fitting on that scale is standard practice, but it introduces a trap. Exponentiating a prediction from log space returns the median rather than the mean of the distribution, which systematically underestimates biomass. The correction is a multiplicative factor $\exp(\hat{\sigma}^2/2)$ (Baskerville, 1972), and it is used throughout this book. The uncertainty chapter shows what it does to the error statistics, and the [statistical primers appendix](#) shows the skew that makes the correction necessary.

7.3 Equation selection

There are thousands of published equations. Choosing among them is the analyst's judgement, and the metadata that governs the choice is more informative than the coefficients. The `eq_tab_acer` table holds published equations for maples, with the fields a careful selection actually needs:

```
eq <- read.csv("data/eq_tab_acer.csv")
c(equations = nrow(eq), taxa = length(unique(eq$equation_taxa)))
#> equations      taxa
#>      24         9
table(eq$allometry_specificity)
#>
#>   Genus Species
#>    1      23

knitr::kable(head(eq[, c("equation_taxa", "equation_allometry",
                        "dbh_min_cm", "dbh_max_cm", "sample_size")], 5))
```

equation_taxa	equation_allometry	dbh_min_cm	dbh_max_cm	sample_size
Acer saccharum	$\exp(-2.192 - 0.011 \text{dbh} + 2.67(\log(\text{dbh})))$	2.80	69.50	18
Acer rubrum	$2.02338 * (\text{dbh}^2)^{1.27612}$	12.95	27.18	18
Acer rubrum	$5.2879 * (\text{dbh}^2)^{1.07581}$	30.48	42.67	12
Acer pseudoplatanus	$\exp(-5.644074 + (2.5189(\log(\text{dbh}))))$	11.50	96.50	10
Acer mandshuricum	$0.0335 * (\text{dbh})^{1.606 + 0.0026(\text{dbh})^3.323 + 0.1222(\text{dbh})^2.310}$	7.80	35.90	10

Four questions decide it. Does the equation match the taxon, at species or at least genus level? Was it fitted in a comparable environment, since the same species allocates biomass differently in a dense stand than an open one? How many trees was it fitted on? And, most neglected, over what diameter range?

```
summary(eq$sample_size)
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
#>   9.00  10.25   13.00   25.95  35.50  150.00     2
```

The median equation here was fitted on thirteen trees. That is not unusual and not a scandal, because destructive harvest is expensive and slow, but it should calibrate how much confidence a single published equation deserves.

7.4 Selection cost

The abstract argument becomes concrete when you apply several defensible equations to the same trees. The `sobi_quercus` file holds 84 oak stems with aboveground biomass computed five

ways: a species-specific equation, a genus-level one, the pantropical models of Chave et al. (2014) and its 2005 predecessor, and an older generic equation.

```
q <- read.csv2("data/scbi_quercus.csv")
for (v in grep("^agb", names(q), value = TRUE)) q[[v]] <- as.numeric(q[[v]])

totals <- colSums(q[, grep("^agb", names(q))]) / 1000 # tonnes
round(totals, 2)
#>   agb_species   agb_genus agb_chave2014 agb_chave2005 agb_brown1997
#>    75.15      81.25      20.50      97.38      65.23
round(c(ratio = max(totals) / min(totals),
        spread_pct = 100 * (max(totals) - min(totals)) / min(totals)), 2)
#>   ratio spread_pct
#>    4.75    374.96
```

The same 84 trees, measured once, weigh between 20 and 97 tonnes depending on which published equation you apply. The largest estimate is **4.75 times** the smallest. No field error, no measurement problem, no disagreement about the data: the entire spread is equation choice.

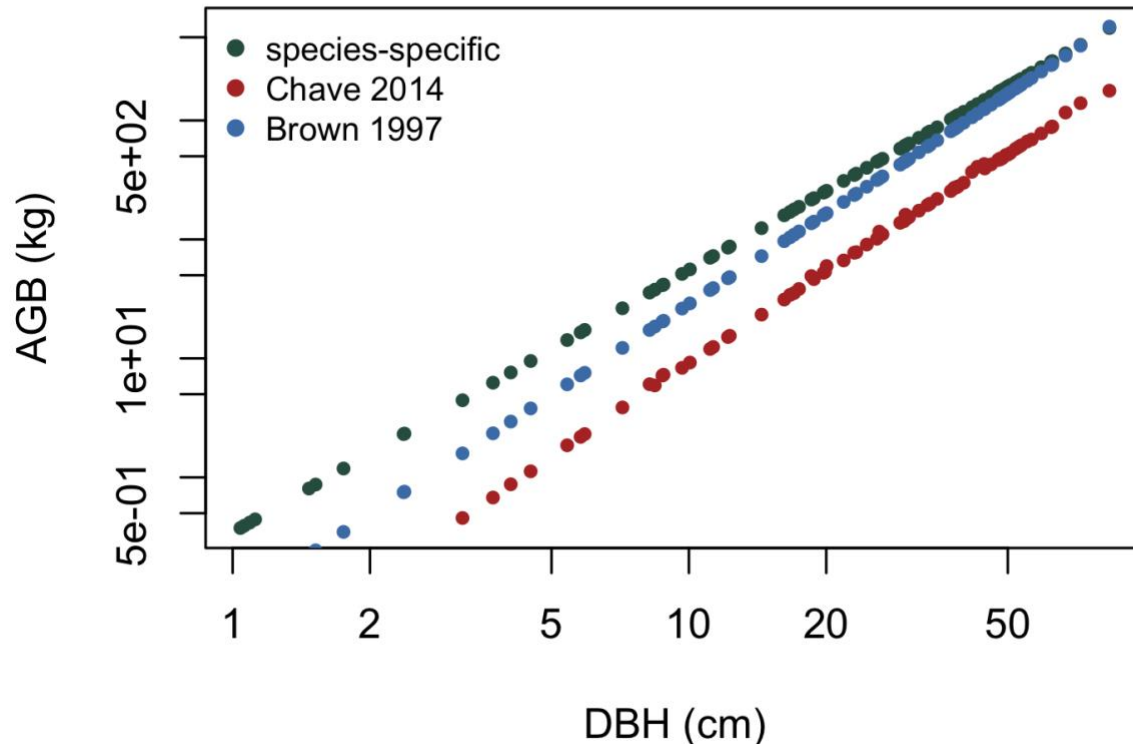
```
carbon_ha <- totals * 0.47 / 25.6
round(carbon_ha, 2)
#>   agb_species   agb_genus agb_chave2014 agb_chave2005 agb_brown1997
#>    1.38      1.49      0.38      1.79      1.20
```

Expressed as carbon density, the choice moves the answer from 0.38 to 1.79 MgC per hectare. If that were a crediting claim, four fifths of it would be a modelling decision.

Not all five are equally defensible, and that is the point rather than an objection. The pantropical equations were fitted on tropical moist forest and applied here to a temperate oak stand, which is precisely the kind of extension that looks harmless in code and is indefensible in review. Knowing that requires reading the equation's provenance, not its coefficients.

```
plot(q$dbh, q$agb_species, log = "xy", pch = 19, cex = 0.6, col = "#2f5d4e",
     xlab = "DBH (cm)", ylab = "AGB (kg)", main = "Five equations, 84 oaks")
points(q$dbh, q$agb_chave2014, pch = 19, cex = 0.6, col = "#b5342e")
points(q$dbh, q$agb_brown1997, pch = 19, cex = 0.6, col = "#4a7fb5")
legend("topleft", c("species-specific", "Chave 2014", "Brown 1997"),
     col = c("#2f5d4e", "#b5342e", "#4a7fb5"), pch = 19, bty = "n", cex = 0.8)
```

Five equations, 84 oaks



1906

1907 On log axes the equations are near-parallel lines separated by a constant factor. The divergence
 1908 is not a subtle disagreement about form; it is a straightforward difference in level that persists
 1909 across every tree in the stand and therefore does not average out.

1910 7.5 Extrapolation risk

1911 An equation is only evidence over the diameters it was fitted on. Outside that range it is an
 1912 extrapolated power law, and power laws extrapolate badly. Check the maple equations against
 1913 the maples actually in the plot:

```

1914 stems <- read.csv("data/scbi_stem1.csv")
1915 acer <- stems[stems$genus == "Acer" & !is.na(stems$dbh), ]
1916 c(stems = nrow(acer), min_dbh = min(acer$dbh), max_dbh = max(acer$dbh))
1917 #>   stems min_dbh max_dbh
1918 #>   438.0      0.9   70.0
1919
1920 has_range <- !is.na(eq$dbh_min_cm) & !is.na(eq$dbh_max_cm)
1921 in_range <- sapply(which(has_range), function(i)
1922   100 * mean(acer$dbh >= eq$dbh_min_cm[i] & acer$dbh <= eq$dbh_max_cm[i]))
1923 round(summary(in_range), 1)
  
```

```

1924 #>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
1925 #>   5.0    28.5    61.4    54.3    78.2    99.3

```

1926 The median published equation is calibrated over a range that contains about 61 per cent of the
 1927 stems it would be applied to, and the weakest of them covers five. That is already poor, and the
 1928 next calculation shows it is worse than it looks, because stems are not what matters. Weight each
 1929 stem by its approximate biomass and ask what share of the *carbon* sits inside the calibration
 1930 range:

```

1931 w <- exp(-2.48 + 2.4835 * log(acer$dbh)) # approximate stem biomass
1932 in_range_c <- sapply(which(has_range), function(i) {
1933   ins <- acer$dbh >= eq$dbh_min_cm[i] & acer$dbh <= eq$dbh_max_cm[i]
1934   100 * sum(w[ins]) / sum(w)
1935 })
1936 round(summary(in_range_c), 1)
1937 #>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
1938 #>   0.0    13.8    43.5    42.8    69.8    95.4

```

1939 The median falls from 61 per cent of stems to **44 per cent of biomass**, the weakest quarter of
 1940 the equations cover under 14 per cent of it, and the worst covers none at all. The reason is
 1941 structural rather than accidental. Destructive sampling is dominated by small trees because large
 1942 ones are expensive to fell and rare in the sample, so equations are best supported exactly where
 1943 the carbon is not. Almost every biomass estimate ever made is an extrapolation over the trees
 1944 that carry most of its mass.

```

1945 big <- max(acer$dbh)
1946 c(largest_stem_cm = round(big, 1),
1947   equations_valid_that_high = sum(eq$dbh_max_cm >= big, na.rm = TRUE),
1948   of_total = sum(has_range))
1949 #>           Largest_stem_cm equations_valid_that_high           of_total
1950 #>                70                1                23

```

1951 One equation out of twenty-three remains valid at the largest maple in the plot, a stem of 70
 1952 centimetres. A project that reports a single point estimate from a single equation is concealing
 1953 that.

1954 7.6 Generic equations

1955 When no local equation exists, the usual recourse is a generic model with wood density as a
 1956 covariate, on the reasoning that density carries much of the between-species variation in mass
 1957 per unit volume. The `scbi_quercus` file includes the density used:

```

1958 q$WD <- as.numeric(q$WD)
1959 knitr::kable(unique(q[, c("species", "WD")]), row.names = FALSE)

```

species	WD
alba	0.6300
michauxii	0.6515

species	WD
prinus	0.5700
rubra	0.5600
velutina	0.5600

Five oak species carry four distinct density values, spanning 0.56 to 0.65, a range of about sixteen per cent. Since biomass is roughly proportional to density in these models, getting the species wrong costs less than getting the equation wrong, which is the argument for generic models: they trade one kind of error for another, being applicable far more widely and less accurate anywhere in particular. Note too that the densities are themselves look-ups from a reference database rather than measurements from these trees, so the covariate that is supposed to absorb between-species variation carries its own error.

7.7 Stand aggregation

Scaling is arithmetic, and the arithmetic is where units go wrong. Biomass per stem sums to biomass per plot, divides by plot area for density, and multiplies by the carbon fraction to become carbon:

```
agb_kg <- exp(-2.48 + 2.4835 * log(stems$dbh[!is.na(stems$dbh)]))
plot_ha <- 25.6
carbon_density <- sum(agb_kg) / 1000 * 0.47 / plot_ha
round(c(stems = length(agb_kg),
        agb_t = sum(agb_kg) / 1000,
        MgC_per_ha = carbon_density), 2)
#>      stems      agb_t MgC_per_ha
#> 38517.00    6108.46    112.15
```

The carbon fraction of 0.47 is itself a default with its own uncertainty, and it multiplies everything above it. The uncertainty chapter propagates that.

Registry requirements

Standards do not usually mandate a particular equation. They constrain how you justify one and what you must report about it, which is a more demanding requirement than it first appears.

Expect to have to document, for each equation used: the source publication, the taxa and geography it was fitted on, its sample size, its diameter range, and an argument that the project's trees fall inside that range. Where they do not, the extrapolation must be acknowledged and its effect on uncertainty carried through.

The uncertainty attached to the equation then enters the deduction arithmetic set out at the end of the uncertainty chapter. Under ART-TREES the allometric contribution feeds the half-width of

the 90 per cent interval; under ACR's Improved Forest Management methodology it is one component of the total uncertainty combined in Equation 22 and compared against the ten per cent precision requirement. In both cases the practical consequence is the same, and it is the argument this chapter has been making with numbers: a better-matched equation is worth more than a larger field campaign, because it reduces a term that no amount of extra plot measurement will touch.

7.8 Exercises

1. Using `eq_tab_acer`, list the equations fitted on more than 30 trees. How many are there, and does the larger sample come at the cost of a narrower diameter range?
2. Apply the species-specific and genus-level columns of `schi_quercus` to compute total stand carbon each way. Express the difference as a percentage, and state which you would defend to a verifier and why.
3. Recompute the biomass-weighted coverage above using a weight of D^2 instead of the allometric weight. Does the conclusion about where the carbon sits change, and what does that tell you about how sensitive the argument is to the weighting?
4. Take the largest maple in the plot and compute its biomass under the three equations that remain valid at that diameter and under three that do not. How large is the spread, and how does it compare with the spread across the whole stand?
5. (*Applied.*) A project has no species-specific equation for its dominant species. Write the paragraph you would put in a monitoring report justifying a generic equation, naming the specific uncertainty it introduces and how you would quantify it.

8. Emission Factors

8.1 Factor definition

Almost every greenhouse gas estimate in land use reduces to one multiplication:

$$\text{Emissions} = \text{Activity Data} \times \text{Emission Factor}$$

Activity data is the extent of something happening: hectares deforested, hectares converted, hectares under a new management regime. The emission factor is the greenhouse gas consequence per unit of that activity. The previous chapters have been about the left-hand term, because that is where spatial analysis lives. This chapter is about the right-hand one, which is where most of the money and most of the uncertainty actually sit.

The distinction matters more than it looks. An analyst who maps deforestation to the pixel and then multiplies by a factor pulled from a default table has spent all their effort on the term they

measured and none on the term they assumed. The factor is usually the larger source of error, and unlike the map it cannot be improved by better imagery.

8.2 Tiers

The IPCC organises methods into three tiers, and the tier is a statement about where the numbers came from rather than about how sophisticated the arithmetic is (IPCC, 2006).

Tier 1 uses global or regional default factors published in the guidelines, applied through simple equations, with default assumptions about what happens after a land-use change. It requires no local measurement beyond the activity data. It is the fallback when nothing better exists, and its uncertainties are correspondingly wide.

Tier 2 keeps the Tier 1 equations but substitutes country-specific or region-specific factors derived from local measurement. This is where most credible national inventories sit.

Tier 3 replaces the equations themselves with models or direct measurement systems, calibrated and validated locally.

The tiers are not a quality ladder in the sense that Tier 1 is wrong. A well-applied Tier 1 estimate with an honest uncertainty is more defensible than a Tier 3 model whose calibration nobody can reproduce. What the tiers do determine is the size of the interval you must report, and therefore the deduction you must accept.

8.3 Default factors

The Tier 1 defaults are not conjured. They are meta-analyses of published soil and biomass measurements, and the guidelines publish their sample sizes and confidence limits alongside the means. Reading those columns is the single most useful habit in factor selection, because the stated uncertainty varies by more than an order of magnitude across the same table.

This is an extract from the reference soil organic carbon stocks of the 2019 Refinement (IPCC, 2019, Table 2.3), which are derived from a global soil profile compilation (Batjes, 2011). The full table covers ten climate zones by six soil classes; these are seven cells of it.

```
soc_ref <- data.frame(
  climate = c("Cool temperate moist", "Cool temperate moist", "Warm temperate dry",
              "Tropical moist", "Tropical moist", "Tropical dry", "Boreal moist/dry"),
  soil     = c("HAC", "Sandy", "HAC", "HAC", "LAC", "LAC", "Sandy"),
  socref   = c(81, 51, 24, 40, 38, 19, 10),      # tonnes C / ha, 0-30 cm
  unc_pct  = c(5, 13, 5, 7, 5, 10, 90),         # 95% Limit, % of mean
  n        = c(334, 126, 781, 226, 326, 135, NA))
knitr::kable(soc_ref)
```

climate	soil	socref	unc_pct	n
Cool temperate moist	HAC	81	5	334
Cool temperate moist	Sandy	51	13	126
Warm temperate dry	HAC	24	5	781
Tropical moist	HAC	40	7	226
Tropical moist	LAC	38	5	326
Tropical dry	LAC	19	10	135
Boreal moist/dry	Sandy	10	90	NA

Look at the last row against the third. Warm temperate dry soils with high activity clay are stated as 24 tonnes per hectare with a 95 per cent limit of five per cent, resting on 781 measured profiles. Boreal sandy soils are stated as 10 tonnes per hectare with a limit of ninety per cent, resting on no profiles at all: that figure is a nominal error assigned because no data were available, carried forward from the 1996 guidelines.

Those two entries look identical in a spreadsheet. They are epistemically unrelated. One is a well-estimated mean; the other is a placeholder with an honesty marker attached. A project whose stratification lands it in the second cell is in a different position from one that lands in the first, and no amount of downstream care recovers the difference.

```
c(worst_to_best_ratio = max(soc_ref$unc_pct) / min(soc_ref$unc_pct))
#> worst_to_best_ratio
#> 18
```

The stated uncertainty spans eighteenfold within a single table. Factor selection is therefore not a lookup; it is a decision about which of these positions you are willing to defend.

8.4 Fire factors

Combustion is the other place Tier 1 factors are used heavily, and it is structurally different from soil. Where soil carbon is a stock difference over two decades, a fire is an event, and the guidelines give it a single equation (IPCC, 2019, Eq. 2.27):

$$L_{fire} = A \times M_B \times C_f \times G_{ef} \times 10^{-3}$$

A is the area burnt in hectares, M_B the mass of fuel available in tonnes of dry matter per hectare, C_f the dimensionless fraction of that fuel actually consumed, and G_{ef} the emission factor in grams of gas per kilogram of dry matter burnt. The equation is a chain of four terms, so its uncertainty is the quadrature sum of four, and it can be no better than its weakest link.

8.4.1 Combustion parameters

The fuel load and combustion factor come from a single table, stratified by vegetation and climate, and for savanna additionally by season (IPCC, 2019, Table 2.4):

```
fuel <- data.frame(
  type = c("Humid tropical forest", "Other forest, boreal",
           "Other forest, temperate", "Other forest, tropical",
           "Savanna woodland, early", "Savanna woodland, late",
           "Savanna grassland, early", "Savanna grassland, late",
           "Other grassland", "Tropical peatland"),
  MB = c(119.6, 41.0, 50.4, 69.4, 9.0, 9.0, 6.8, 6.8, 13.4, 41.2),
  MB_sd = c(52.4, 25.3, 30.8, 44.3, 7.5, 7.5, 6.0, 6.0, 8.8, 1.4),
  Cf = c(0.36, 0.45, 0.47, 0.47, 0.29, 0.51, 0.31, 0.83, 0.52, 1.00),
  Cf_sd = c(0.09, 0.15, 0.14, 0.14, 0.09, 0.15, 0.09, 0.15, 0.15, NA))
fuel$consumed <- round(fuel$MB * fuel$Cf, 1)
knitr::kable(fuel)
```

type	MB	MB_sd	Cf	Cf_sd	consumed
Humid tropical forest	119.6	52.4	0.36	0.09	43.1
Other forest, boreal	41.0	25.3	0.45	0.15	18.4
Other forest, temperate	50.4	30.8	0.47	0.14	23.7
Other forest, tropical	69.4	44.3	0.47	0.14	32.6
Savanna woodland, early	9.0	7.5	0.29	0.09	2.6
Savanna woodland, late	9.0	7.5	0.51	0.15	4.6
Savanna grassland, early	6.8	6.0	0.31	0.09	2.1
Savanna grassland, late	6.8	6.0	0.83	0.15	5.6
Other grassland	13.4	8.8	0.52	0.15	7.0
Tropical peatland	41.2	1.4	1.00	NA	41.2

Two rows deserve attention. Savanna grassland burns at a combustion factor of 0.31 early in the dry season and 0.83 late in it, so the same fuel load yields nearly three times the consumption depending on a date. Season is not a detail in savanna accounting; it is the dominant term. And tropical peatland carries a combustion factor of exactly 1.00, meaning the affected layer is assumed to burn completely, which is why peat fires dominate any inventory that contains them.

The emission factors are gas-specific (IPCC, 2019, Table 2.5):

```
ef <- data.frame(
  vegetation = c("Tropical forest", "Extratropical forest",
                "Savanna and grassland", "Tropical peatland"),
  CO2 = c(1580, 1569, 1613, 1703),
  CH4 = c(6.8, 4.7, 2.3, 5.7),
  N2O = c(0.20, 0.26, 0.21, NA))
knitr::kable(ef)
```

vegetation	CO2	CH4	N2O
Tropical forest	1580	6.8	0.20
Extratropical forest	1569	4.7	0.26
Savanna and grassland	1613	2.3	0.21
Tropical peatland	1703	5.7	NA

The missing entry is real. The guidelines provide no default nitrous oxide factor for fires in organic soils, for want of data, and national inventories consequently report that cell as zero. Zero and unknown are different things, and a table that renders them identically has lost information a verifier needs.

8.4.2 Unit conversion

The 10^{-3} in the equation is the whole unit chain, and dropping it is the most consequential arithmetic error in fire accounting. Working through the dimensions makes it explicit:

```
biomass_Gg <- 1682.71      # dry matter burnt, gigagrams
ef_ch4     <- 6.8          # grams CH4 per kilogram dry matter

# Gg -> kg, multiply by EF (g/kg) to get grams, then g -> Gg
correct <- biomass_Gg * 1e6 * ef_ch4 / 1e9
wrong  <- biomass_Gg * ef_ch4      # the shortcut, units unchecked

data.frame(correct_Gg_CH4 = round(correct, 2),
            wrong_Gg_CH4  = round(wrong, 2),
            ratio          = wrong / correct)
```

correct_Gg_CH4	wrong_Gg_CH4	ratio
11.44	11442.43	1000

A factor of a thousand, and it does not announce itself. The shortcut is tempting because both quantities are already expressed in gigagrams and grams per kilogram look dimensionless at a glance. Applying a global warming potential then compounds it:

```
gwp_ch4 <- 28      # AR5, 100-year, without feedbacks

data.frame(correct_Gg_CO2e = round(correct * gwp_ch4, 1),
            wrong_Gg_CO2e  = round(wrong * gwp_ch4, 0))
```

correct_Gg_CO2e	wrong_Gg_CO2e
320.4	320388

The erroneous figure is around 320,000 gigagrams of carbon dioxide equivalent from a single gas in a single fire category. The whole national inventory of the country this example is drawn from is of the order of 20,000. An error of this kind is invisible inside the pipeline and obvious the

2132 moment the output is compared against an independent total, which is the argument for always
2133 making that comparison.

2134 Two conventions have to be stated alongside any such number. The global warming potentials
2135 used here are the hundred-year AR5 values without climate-carbon feedbacks, 28 for methane
2136 and 265 for nitrous oxide, which is what national inventory reporting has used through recent
2137 submission cycles. The AR6 values are 27.9 and 273. Mixing assessment reports across a time
2138 series produces a discontinuity that looks like a trend.

2139 **8.4.3 Carbon dioxide**

2140 Whether carbon dioxide from a fire is reported at all depends on the land category, and the rule
2141 is easy to get backwards:

Fire category	CO2 reported	Accounting route
Forest land	Yes	Carbon stock change, not this equation
Savanna and grassland	No, at Tier 1	Assumed synchronous with regrowth
Organic soil	Yes	This equation, with its own factor

2142 The logic is regrowth. Grass burnt this year regrows next year, so the emission and the removal
2143 cancel over the inventory cycle and reporting only the net, which is zero, is defensible. Woody
2144 biomass killed by fire represents several years of accumulation and does not regrow within the
2145 cycle, so forest fire emissions are real and are captured through the stock-change accounting
2146 rather than through this equation, which would double count them. Peat is the extreme case: the
2147 carbon accumulated over centuries to millennia, consumption runs to hundreds of millimetres in
2148 one event against an accumulation rate below a millimetre a year, and nothing regrows on any
2149 policy-relevant horizon.

2150 The trap is applying the equation’s carbon dioxide factor to forest fires and adding the result to a
2151 stock-change inventory that already contains it. The factors appear in Table 2.5 for forest, which
2152 invites exactly this, and they are there as a cross-check rather than as an input.

2153 **8.5 Stock-difference method**

2154 Soil carbon is the pool where Tier 1 methods have changed most. The 2006 guidelines assumed
2155 that converting land oxidised its soil carbon immediately. The 2019 Refinement replaces that with
2156 a linear transition to a new equilibrium spread over a default twenty years, computed as a
2157 difference between two stock estimates (IPCC, 2019, Eq. 2.25):

$$\Delta C_{Mineral} = \frac{SOC_0 - SOC_{(0-T)}}{D}$$

where SOC_0 is the stock in the final year of the inventory period, $SOC_{(0-T)}$ the stock at its beginning, and D the time dependence of the stock change factors, twenty years under Tier 1. Each stock is itself built from a reference value and three dimensionless factors:

$$SOC = SOC_{REF} \times F_{LU} \times F_{MG} \times F_I$$

F_{LU} is the land use factor, F_{MG} the management factor, and F_I the input factor, each a multiplier relative to the native reference condition. All are defined for a 30 cm depth, because that is the depth the reference stocks were compiled at, and mixing a factor from one depth convention with a stock from another is a common and invisible error.

```

2167 socref <- 38      # tropical moist, LAC
2168 f_lu   <- 0.83    # forest to long-term cultivated
2169 f_mg   <- 1.00    # full tillage
2170 f_i    <- 0.94    # low input
2171
2172 soc_new <- socref * f_lu * f_mg * f_i
2173 annual_c <- (soc_new - socref) / 20
2174 annual_co2 <- annual_c * 44 / 12
2175
2176 round(c(soc_before = socref, soc_after = soc_new,
2177         tC_ha_yr = annual_c, tC02e_ha_yr = annual_co2), 3)
2178 #> soc_before soc_after tC_ha_yr tC02e_ha_yr
2179 #> 38.000 29.648 -0.418 -1.531

```

The chain matters more than any single link. Three multiplicative factors mean three opportunities for a stratification error, and because they multiply, a mistake in any one of them scales the whole result rather than adding to it.

2183 8.6 Gain-loss asymmetry

2184 Tier 1 does not treat gains and losses alike. Carbon *increases* are spread over the twenty-year
 2185 transition; carbon *losses* from dead organic matter on conversion are assigned to a single year
 2186 (IPCC, 2019, p. s.2.3.2). That asymmetry is deliberate and conservative: it credits slowly and
 2187 debits fast.

2188 It also has a practical consequence for anyone comparing a project's numbers against an
 2189 inventory. A project reporting a gain over five years captures a quarter of the eventual stock
 2190 change, not all of it, and a project reporting a conversion loss books the whole thing at once. Two
 2191 projects with identical biophysical outcomes and different timings will report very different annual
 2192 figures, entirely legitimately.

8.7 Uncertainty propagation

Each factor in the chain carries its own uncertainty, and they combine. For independent terms entering a product, the proportional uncertainties combine in quadrature:

$$U_{total} = \sqrt{U_1^2 + U_2^2 + \dots + U_n^2}$$

Take a silvopastoral project on a thousand hectares, with the uncertainty on each component taken from the guidelines and from the project's own measurement:

```
u <- c(SOC_REF = 0.05, # reference stock, well-sampled cell
      F_MG    = 0.09, # management factor, improved tropical grassland
      F_I     = 0.30, # input factor, high organic input
      G       = 0.63, # biomass accumulation rate, agroforestry
      Area    = 0.10) # activity data

u_total <- sqrt(sum(u^2))
round(100 * u_total, 1)
#> [1] 71.2
```

Seventy-one per cent. That figure is not unusual for a Tier 1 agroforestry estimate, and it is worth sitting with, because it means the honest interval on the claim is wider than the claim itself in one direction.

8.8 Dominant uncertainty

Quadrature has a property that governs everything about how a project should spend its money, and it is not intuitive. Because the terms are squared before summing, a term's contribution to the total is proportional to its square, so large terms do not merely dominate, they annihilate the small ones.

```
round(100 * u^2 / sum(u^2), 1) # share of total variance
#> SOC_REF F_MG F_I G Area
#> 0.5 1.6 17.7 78.2 2.0
```

The biomass accumulation rate contributes seventy-eight per cent of the variance. The reference soil stock contributes half of one per cent. Now ask what improving each one would buy:

```
sapply(names(u), function(k) {
  v <- u; v[k] <- v[k] / 2
  round(100 * sqrt(sum(v^2)), 1)
})
#> SOC_REF F_MG F_I G Area
#> 71.1 70.8 66.3 45.8 70.7
```

Halving the uncertainty on the growth rate takes the project from 71 to 46 per cent. Halving the uncertainty on the reference soil stock takes it from 71.2 to 71.1. The comparison becomes starker if you imagine perfect knowledge:

```

2230 sapply(names(u), function(k) {
2231   v <- u; v[k] <- 0
2232   round(100 * sqrt(sum(v^2)), 1)
2233 })
2234 #> SOC_REF    F_MG    F_I    G    Area
2235 #>    71.1    70.7    64.6   33.3   70.5

```

2236 Eliminating the area uncertainty *entirely*, surveying every boundary to the centimetre and knowing
 2237 the hectares exactly, moves total uncertainty from 71.2 per cent to 70.5. Eliminating the soil
 2238 reference stock uncertainty moves it to 71.1.

2239 This is the most actionable result in the chapter, and it cuts against how carbon projects usually
 2240 allocate effort. Spatial precision is visible, satisfying, and easy to procure. It is also, in a project of
 2241 this shape, nearly worthless. The money belongs in the growth-rate measurement, which is
 2242 unglamorous fieldwork on permanent plots, and which no improvement in imagery will ever
 2243 substitute for.

2244 The general rule follows from the algebra: rank the terms by U_i^2 , and work down the list. A term
 2245 contributing under one per cent of the variance cannot be worth measuring, however cheap,
 2246 because the total will not move.

2247 8.9 Cost of imprecision

2248 Convert it to credits, which is the language the decision is actually made in:

```

2249 point_estimate <- 18.5 * 1000 # tCO2e/yr over 1,000 ha
2250 conservative   <- point_estimate * (1 - u_total)
2251
2252 round(c(point_estimate = point_estimate,
2253   conservative   = conservative,
2254   forgone         = point_estimate - conservative,
2255   forgone_pct     = 100 * u_total))
2256 #> point_estimate conservative forgone forgone_pct
2257 #>    18500         5321    13179         71

```

2258 Reporting against the lower bound, the project claims about 5,300 tonnes where its central
 2259 estimate is 18,500. Thirteen thousand tonnes a year are forgone, not because the carbon is not
 2260 there, but because the project cannot demonstrate that it is. Under the deduction arithmetic of the
 2261 uncertainty chapter the loss is smaller and the logic identical.

2262 That number is also the budget available for reducing it. If halving the growth-rate uncertainty
 2263 costs less than the value of the credits it recovers, the measurement pays for itself, and this
 2264 calculation is how you find out before commissioning the fieldwork rather than after.

2265 *Double counting*

2266 The twenty-year transition creates a bookkeeping hazard that catches experienced practitioners.
2267 A parcel that changes management in year one is still accruing its stock change in year fifteen. If
2268 a second intervention is credited on the same parcel in year eight, part of the soil carbon response
2269 may be claimed twice: once against the first transition and once against the second.

2270 The rule is that stock change for a given parcel is computed against a single transition timeline,
2271 not per intervention. Where interventions overlap, the parcel carries one trajectory and the later
2272 activity is credited only for the increment beyond what the first would have delivered anyway.

2273 Tracking this in practice means the parcel, not the project, is the accounting unit, and each parcel
2274 needs the date its transition clock started. Where activity data comes from wall-to-wall mapping
2275 rather than parcel records, that date may simply not exist, which is one of the strongest arguments
2276 for the transition matrices of IPCC Approach 3 over the aggregate statistics of Approach 1.

2277 *Registry acceptance*

2278 Standards differ on how much a project may lean on defaults. In general, a Tier 1 default is
2279 acceptable where it is conservative for the claim being made, and not otherwise. A default that
2280 inflates a crediting claim will be challenged; the same default used to inflate a *baseline* against
2281 which the project is credited is usually accepted without argument, because it works against the
2282 proponent.

2283 That asymmetry is worth stating plainly because it is the fastest way to predict which of a project's
2284 factor choices a verifier will scrutinise. Work out, for each factor, whether an error in the direction
2285 of the default increases or decreases the credits issued. The ones that increase them are where
2286 the questions come.

2287 The uncertainty from this chapter then feeds the deduction machinery compared at the end of the
2288 uncertainty chapter. Under ACR it joins the other pool uncertainties in the weighted combination
2289 of Equation 22, and is measured against the ten per cent precision requirement. Under ART-
2290 TREES and Verra it enters the half-width of the 90 per cent interval directly. A 71 per cent
2291 uncertainty is catastrophic under any of them, which is the practical reason Tier 1 agroforestry
2292 projects tend to move to Tier 2 growth rates as soon as they can afford to.

8.10 Exercises

1. Recompute the stock change using a cool temperate moist HAC reference stock (81 tC/ha) instead of the tropical moist LAC value, holding the three factors constant. How much does the annual figure move, and what does that say about the cost of misclassifying climate zone?
2. Using the extract of Table 2.3 above, identify which cells you would be comfortable using in a crediting claim and which you would not. Write one sentence per cell justifying the decision on the basis of the stated uncertainty and sample size.
3. In the propagation example, suppose better fieldwork reduces the growth-rate uncertainty from 63 to 25 per cent, and that improving the input factor from 30 to 20 per cent costs the same amount. Which would you buy, and by how much does each move the total?
4. Add a sixth uncertainty term of 15 per cent to the propagation, representing a carbon fraction. Does it change the ranking of where to spend money? Explain in terms of the squared contributions rather than the raw percentages.
5. The transition period is twenty years for gains and one year for losses. Construct a numerical example of two projects with identical biophysical outcomes but different timings, and show how their reported annual figures differ.
6. (*Applied.*) A project reports 71 per cent uncertainty and a point estimate of 18,500 tCO₂e per year. Using the three registry deduction rules from the uncertainty chapter, calculate the credits issued under each. Then state what you would advise the project to measure first.

9. Activity Data

9.1 Extent term

The emission factors chapter dealt with the greenhouse gas consequence per hectare. This chapter deals with the hectares. Activity data is the extent and location of whatever the factor is being applied to: land deforested, land converted, land brought under a new management regime. It is the term spatial analysis is actually for.

It is also the term most likely to be produced by a classification, and therefore the term that inherits a classifier's errors. The disturbance chapter built a two-class change map and validated it. Real activity data is rarely two classes, the classes of interest are usually rare, and both of those facts change the problem in ways that a binary example conceals.

9.2 Three approaches

The IPCC distinguishes three ways of collecting activity data, and the distinction is about spatial explicitness rather than accuracy of measurement (IPCC, 2006).

Approach 1 uses aggregate area statistics: how much cropland, how much forest, at two points in time. It gives net change and cannot say what became what.

Approach 2 adds transition information: how much forest became cropland, tracked as a matrix, but without locating the transitions.

Approach 3 is spatially explicit: every parcel or pixel carries its own land-use trajectory, so transitions are located as well as counted.

Only Approach 3 supports the parcel-level transition clocks that the emission factors chapter showed are necessary to avoid double counting across overlapping interventions, and only Approach 3 lets you stratify emission factors by anything you can map. It is also the only one that produces a classification, and therefore the only one with classification error to worry about. That is not a drawback. Approaches 1 and 2 have the same errors; they simply have no mechanism for measuring them.

9.3 Scene classification

We build a four-class scene over the plot footprint: forest, a cleared block, a patch of regrowth, and water along the southern edge. Four spectral bands are simulated with deliberate overlap between forest and regrowth, because that is where real classifiers fail and a scene with separable classes teaches nothing.

```
library(terra)
library(sf)
library(ranger)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
  coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
r <- rast(ext(vect(stems)), resolution = 10, crs = "EPSG:32617")
xy <- xyFromCell(r, 1:ncell(r))

set.seed(42)
cls <- rep("forest", ncell(r))
cls[xy[,1] > 747760 & xy[,1] < 747900 & xy[,2] > 4309050 & xy[,2] < 4309200] <- "cleared"
cls[xy[,1] > 747620 & xy[,1] < 747720 & xy[,2] > 4308950 & xy[,2] < 4309080] <- "regrowth"
cls[xy[,2] < 4308880] <- "water"

# simulated reflectance: forest and regrowth deliberately overlap
mu <- list(forest = c(0.04, 0.06, 0.32, 0.18),
  regrowth = c(0.05, 0.08, 0.28, 0.22),
  cleared = c(0.12, 0.16, 0.20, 0.30),
  water = c(0.03, 0.04, 0.02, 0.01))
sd_ <- c(0.012, 0.014, 0.030, 0.025)

bands <- sapply(1:4, function(b) {
  v <- numeric(ncell(r))
  for (k in names(mu)) { i <- cls == k; v[i] <- rnorm(sum(i), mu[[k]][b], sd_[b]) }
  v
})
img <- rast(r, nlyrs = 4); values(img) <- bands
names(img) <- c("blue", "green", "nir", "swir")

truth <- rast(r); values(truth) <- as.integer(factor(cls))
round(100 * table(cls) / ncell(r), 1)
#> cls
```

```
2376 #> cleared forest regrowth water
2377 #> 7.7 79.9 4.8 7.7
```

2378 Regrowth covers under five per cent of the scene. That is the situation activity data is almost
2379 always in: the class you are being paid to detect is the rare one, and the class that dominates the
2380 accuracy statistics is the one nobody is asking about.

2381 **9.4 Rare-class trap**

2382 The obvious way to collect training data is to sample the scene at random, which gives each class
2383 a share proportional to its area.

```
2384 fit_and_assess <- function(idx, label) {
2385   tr <- data.frame(cover = factor(cls[idx]), bands[idx, ])
2386   names(tr)[-1] <- names(img)
2387   f <- ranger(cover ~ ., data = tr, probability = TRUE, num.trees = 300, seed = 1)
2388   pp <- terra::predict(img, f, na.rm = TRUE,
2389                        fun = function(m, d) predict(m, d)$predictions)
2390   hard <- max.col(as.matrix(pp))
2391   cm <- table(map = levels(factor(cls))[hard], ref = cls)
2392   ua <- diag(cm) / rowSums(cm); pa <- diag(cm) / colSums(cm)
2393   data.frame(design = label,
2394              overall = round(100 * mean(hard == values(truth)), 1),
2395              regrowth_UA = round(ua[["regrowth"]], 3),
2396              regrowth_PA = round(pa[["regrowth"]], 3),
2397              VM0048 = ifelse(ua[["regrowth"]] >= 0.7 & pa[["regrowth"]] >= 0.7,
2398                             "pass", "fail"))
2399 }
2400
2401 set.seed(7)
2402 prop300 <- fit_and_assess(sample(ncell(r), 300), "proportional, n = 300")
2403 prop300
```

design	overall	regrowth_UA	regrowth_PA	VM0048
proportional, n = 300	96.9	0.871	0.415	fail

2404 Ninety-seven per cent overall accuracy, which in a project report would look like a finished job.
2405 The class-level figures say otherwise: the map catches under half of the real regrowth. Overall
2406 accuracy is dominated by forest, which is 81 per cent of the scene and easy, so it tells you almost
2407 nothing about the class you care about.

2408 This is why the standards require class-level accuracies rather than an overall figure, and why
2409 they require *both* user's and producer's accuracy to clear the threshold. A single number can
2410 always be gamed by a class that is common and easy.

2411 **9.5 Reweighting limits**

2412 The standard response is to stratify: sample equally from each class so the rare one is properly
2413 represented. Compare four designs, holding the classifier fixed.

```

2414 strat <- function(n) unlist(lapply(split(seq_len(ncell(r)), cls),
2415                                   function(i) sample(i, min(length(i), n))))
2416 set.seed(7)
2417 results <- rbind(
2418   prop300,
2419   fit_and_assess(strat(75),           "stratified, 75 per class"),
2420   fit_and_assess(strat(300),         "stratified, 300 per class"),
2421   fit_and_assess(sample(ncell(r), 1200), "proportional, n = 1200"))
2422 knitr::kable(results, row.names = FALSE)

```

design	overall	regrowth_UA	regrowth_PA	VM0048
proportional, n = 300	96.9	0.871	0.415	fail
stratified, 75 per class	89.8	0.308	0.923	fail
stratified, 300 per class	96.2	0.556	0.962	fail
proportional, n = 1200	97.9	0.860	0.662	fail

2423 Stratifying does exactly what it promises and not what was wanted. Producer's accuracy for
 2424 regrowth climbs above 0.9, because the classifier now finds the class. User's accuracy collapses,
 2425 because it now finds it everywhere, including where it is not. Omission has been traded for
 2426 commission at close to a one-for-one rate, and overall accuracy has fallen.

2427 Only the last row passes. What fixed it was not a cleverer allocation of the sample but four times
 2428 as much of it. That is an unwelcome result, because reweighting is free and fieldwork is not, and
 2429 it is worth stating plainly: when a rare class is spectrally confusable, sampling design redistributes
 2430 the error rather than reducing it. Reducing it takes more information, either more training data or
 2431 better features.

2432 The dual threshold in the standards exists precisely to prevent the trade. A project may not buy
 2433 producer's accuracy with user's accuracy, because a map that over-calls deforestation is as
 2434 unusable as one that misses it, just in the opposite fiscal direction.

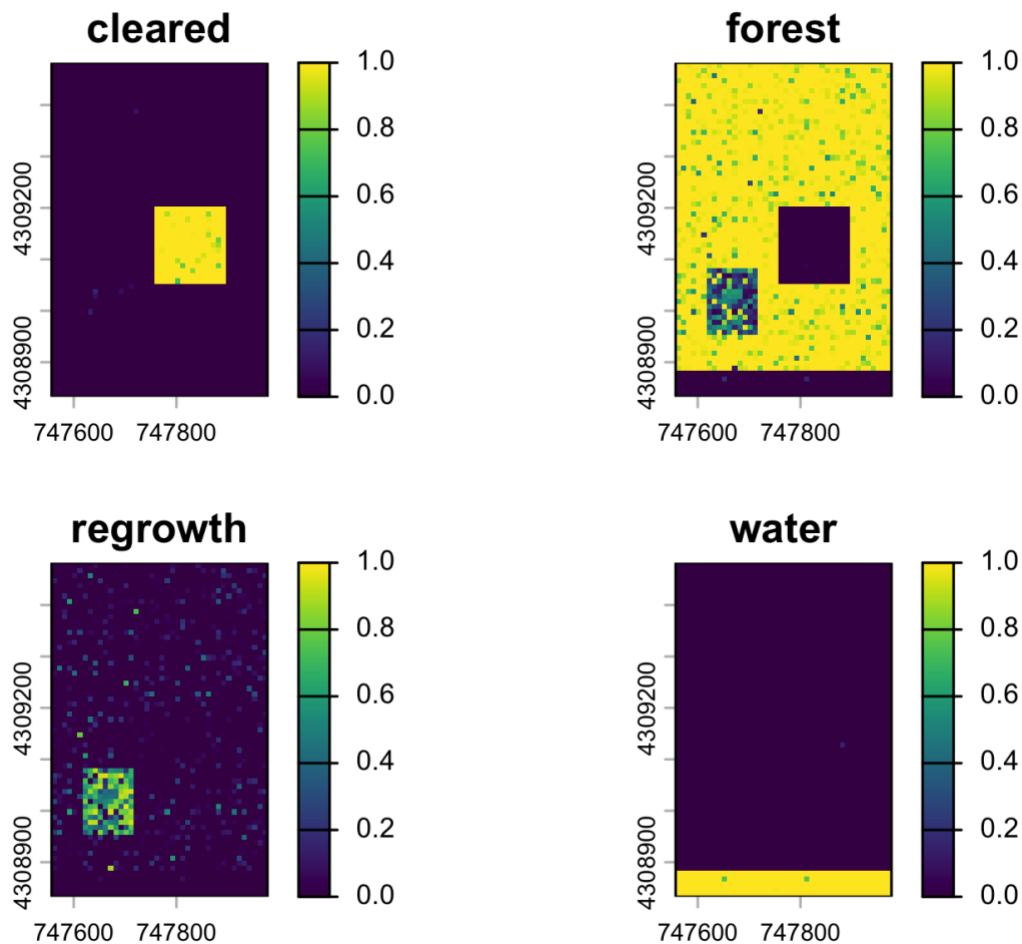
2435 9.6 Probability surfaces

2436 A classifier that returns only a label discards most of what it knows. Asking for probabilities gives
 2437 a surface per class, and the shape of those surfaces is where the useful diagnostics live.

```

2438 set.seed(7)
2439 idx <- sample(ncell(r), 1200)
2440 tr <- data.frame(cover = factor(cls[idx]), bands[idx, ])
2441 names(tr)[-1] <- names(img)
2442 fit <- ranger(cover ~ ., data = tr, probability = TRUE, num.trees = 300, seed = 1)
2443
2444 prob <- terra::predict(img, fit, na.rm = TRUE,
2445                        fun = function(m, d) predict(m, d)$predictions)
2446 names(prob) <- levels(tr$cover)
2447 plot(prob, range = c(0, 1))

```



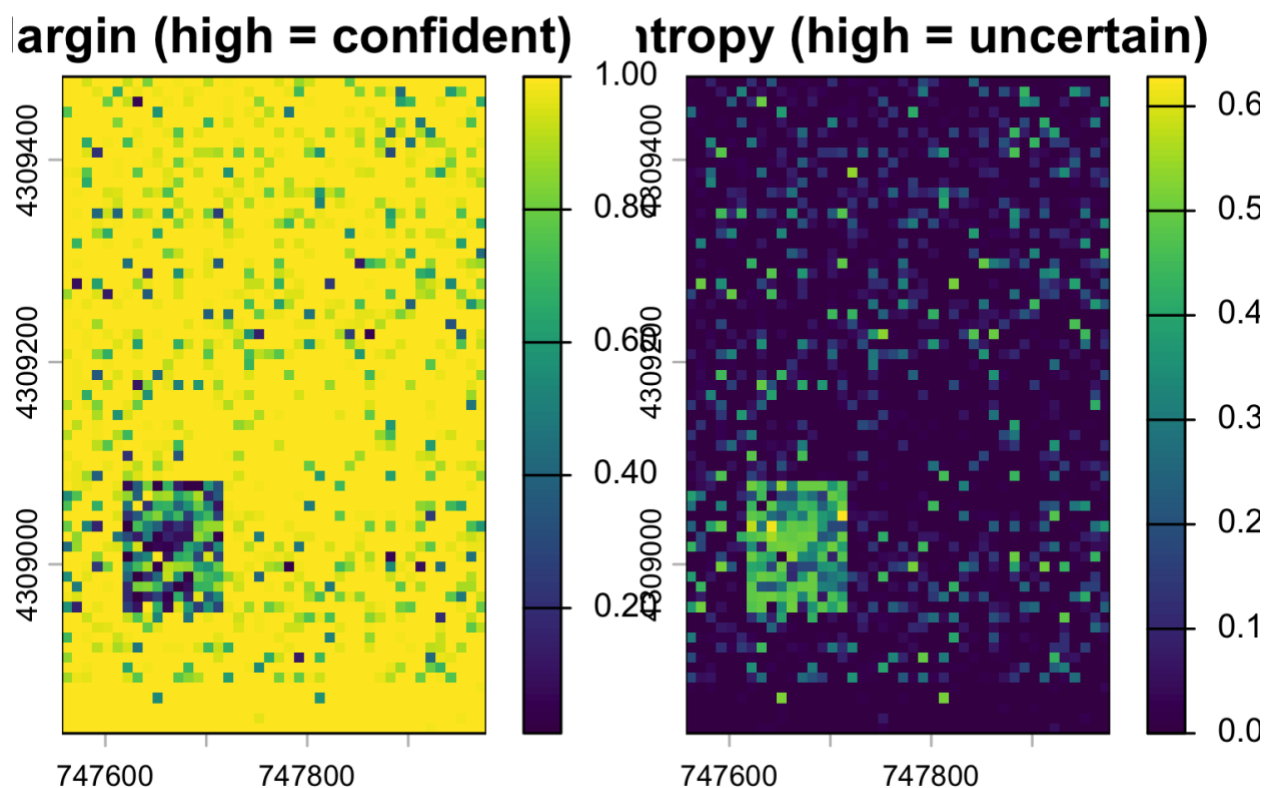
2448

2449 Two summaries turn the four surfaces into one uncertainty layer. The **margin** is the difference
 2450 between the highest and second-highest class probability: it is small where two classes compete.
 2451 **Entropy**, normalised to a maximum of one, is small where the classifier is confident in any single
 2452 class and large where probability is spread.

```

2453 pm <- as.matrix(prob)
2454 top2 <- t(apply(pm, 1, function(x) sort(x, decreasing = TRUE)[1:2]))
2455 margin <- top2[, 1] - top2[, 2]
2456 entropy <- -rowSums(pm * log(pmax(pm, 1e-12))) / log(ncol(pm))
2457
2458 unc <- rast(r, nlyrs = 2); values(unc) <- cbind(margin, entropy)
2459 names(unc) <- c("margin", "entropy")
2460 plot(unc, main = c("Margin (high = confident)", "Entropy (high = uncertain)"))

```



2461

2462 Uncertainty is not spread evenly. It concentrates along class boundaries, where a pixel genuinely
 2463 contains two covers, and across the regrowth patch, where the spectral overlap with forest was
 2464 built in. That spatial structure is the point: it tells you *where* the map is weak, which a single
 2465 accuracy figure never can.

```
2466   hard    <- max.col(pm)
2467   correct <- hard == values(truth)
2468   round(c(margin_when_correct = mean(margin[correct]),
2469         margin_when_wrong    = mean(margin[!correct])), 3)
2470   #> margin_when_correct  margin_when_wrong
2471   #>           0.944           0.387
2472   round(100 * sum(!correct & margin <= quantile(margin, 0.1)) / sum(!correct), 1)
2473   #> [1] 78
```

2474 Correct pixels carry a mean margin close to twice that of incorrect ones, and more than three
 2475 quarters of all errors fall in the lowest-margin tenth of the scene. The margin is therefore a usable
 2476 predictor of error without any reference data at all, which is what makes the next step possible.

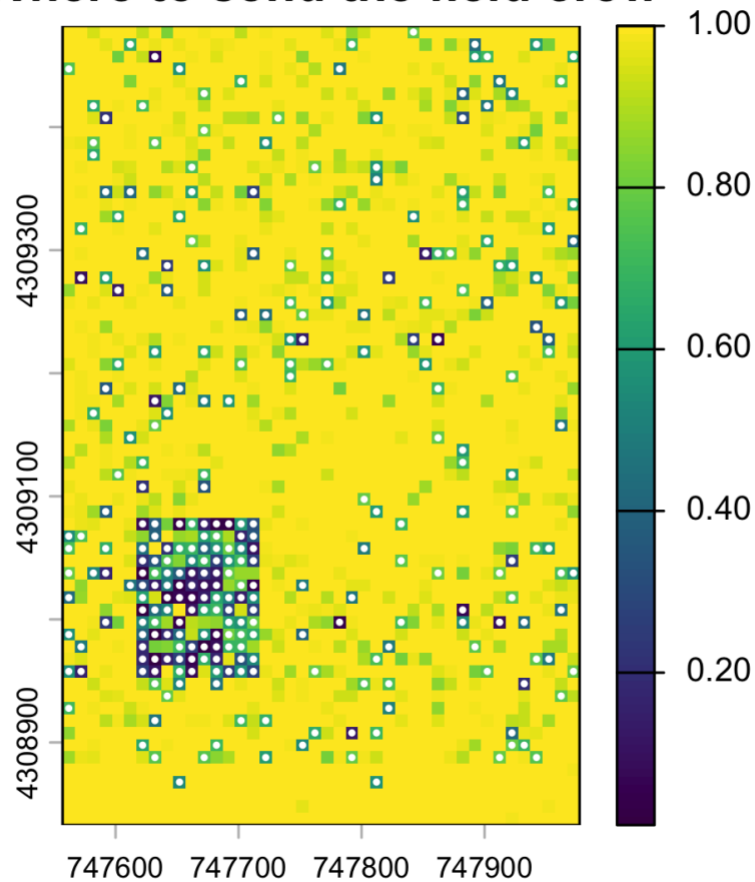
9.7 Sample allocation

If errors concentrate where the margin is low, then the most informative place to collect another training point is where the classifier is least sure. This is active learning, and in an operational setting it is the difference between a validation campaign that improves the map and one that merely measures it.

```
candidates <- order(margin)[1:300]
cand_xy    <- xy[candidates, ]

plot(rast(r, vals = margin), main = "Where to send the field crew")
points(cand_xy, pch = 19, cex = 0.25, col = "white")
```

Where to send the field crew



```
round(100 * table(cls[candidates]) / length(candidates), 1)
#>
#>   cleared forest regrowth   water
#>    3.0    61.3    35.0    0.7
```

The three hundred least-certain pixels are heavily enriched in regrowth relative to its share of the scene. A crew sent to those locations returns labels for the class the map is failing on, which is

exactly what a proportional random sample would not have delivered. Note that these points are useful for *training* and are not a valid basis for *accuracy assessment*: a sample chosen because it is difficult is not representative, and using it to estimate accuracy would understate the map's quality. Accuracy needs a probability sample, which is the subject of the uncertainty chapter.

VM0048 map requirements

Verra's activity data module sets class-level thresholds rather than an overall accuracy figure, and the reasoning is the one demonstrated above.

For a jurisdictional forest cover benchmark map, user's and producer's accuracy must **each** be 70 per cent or greater for areas mapped as deforestation over the historical reference period, and **each** greater than 90 per cent for areas mapped as forest at the end of that period. Where it is demonstrated that half or more of the jurisdiction's forest carries under 50 per cent canopy cover, those thresholds relax to 60 and 80 per cent respectively.

A minimum of 100 sample observations should be made of the target and non-target classes, so at least 200 per estimate, following the same interpretation protocol used to develop the activity data in the first place.

Resolution is prescribed too, and asymmetrically: the wall-to-wall benchmark map must be at 30 m or finer and must not exceed 30 m, while imagery interpreted inside sample plots must be 10 m or finer. The split follows from the map not being the estimator. Under VM0048 the area comes from the probability sample and the map only stratifies it, which is the same argument the uncertainty chapter makes when it adjusts a mapped area against a reference sample.

Sources: VMD0055 v1.1, accuracy assessment of the jurisdictional FCBM.

Operational stack

Production activity data work is rarely done with a raster and a random forest in memory. The `sits` package organises satellite time series into regularised data cubes and runs classification over them, with the training-sample tooling that large multi-class problems need (Simões et al., 2021). The code below is the shape of that workflow. It is not executed here, because the book renders offline and the dependency stack is heavy, but it maps one-to-one onto the steps above.

```
library(sits)
# a regularised cube, rather than a single-date image
cube <- sits_cube(source = "MPC", collection = "SENTINEL-2-L2A",
  roi = roi, start_date = "2022-01-01", end_date = "2022-12-31")
reg <- sits_regularize(cube, period = "P16D", res = 10, output_dir = tempdir())
```

```

2528 # spectral indices as additional cube bands
2529 reg <- sits_apply(reg, NDVI = (B08 - B04) / (B08 + B04), output_dir = tempdir())
2530
2531 # training samples carry a full time series per point, not one observation
2532 samples <- sits_get_data(reg, samples = training_points)
2533
2534 # self-organising map to find and drop mislabelled samples
2535 som <- sits_som_map(samples)
2536 clean <- sits_som_clean_samples(som)
2537
2538 # class imbalance handled explicitly rather than by resampling pixels
2539 bal <- sits_reduce_imbalance(clean, n_samples_over = 200, n_samples_under = 400)
2540
2541 model <- sits_train(bal, sits_rfor())
2542 probs <- sits_classify(reg, model, output_dir = tempdir())
2543
2544 # Bayesian smoothing over the probability cube before labelling
2545 sm <- sits_smooth(probs, output_dir = tempdir())
2546 map <- sits_label_classification(sm, output_dir = tempdir())
2547
2548 # per-pixel uncertainty, then active learning on the worst pixels
2549 unc <- sits_uncertainty(sm, type = "margin", output_dir = tempdir())
2550 new <- sits_uncertainty_sampling(unc, n = 300, min_uncert = 0.4, sampling_window = 10)

```

2551 Three things in that sequence do not appear in the in-memory version above and are worth
 2552 knowing about. `sits_som_clean_samples()` finds training points whose spectra are inconsistent with
 2553 their label, which catches interpretation errors before they poison the model. `sits_reduce_imbalance()`
 2554 addresses the rare-class problem at the sample level with explicit over- and under-sampling
 2555 targets. And `sits_smooth()` applies Bayesian smoothing to the probability cube before labelling,
 2556 which suppresses isolated misclassified pixels while preserving real edges, a step with no
 2557 equivalent in the hard-label workflow.

2558 9.8 Exercises

- 2559 1. Change the regrowth spectral means to be more distinct from forest, for instance by raising the near-infrared
 2560 value, and rerun the four training designs. At what point does the proportional design at $n = 300$ pass the
 2561 VM0048 thresholds, and what does that tell you about the relative value of better features against more
 2562 samples?
- 2563 2. Compute the confusion matrix for the $n = 1200$ proportional design and report all four classes' user's and
 2564 producer's accuracies. Which class pair accounts for nearly all of the remaining error?
- 2565 3. Replace the margin with entropy as the active-learning criterion and compare the class composition of the 300
 2566 selected pixels. Do the two criteria disagree, and where?
- 2567 4. Using the area-adjusted estimator from the uncertainty chapter, compute the adjusted area of regrowth and
 2568 its confidence interval under both the proportional and stratified designs. Which design gives the tighter
 2569 interval, and is it the one with the higher overall accuracy?
- 2570 5. The active-learning sample is explicitly not valid for accuracy assessment. Design a sampling scheme that
 2571 serves both purposes, or explain in one paragraph why one cannot.
- 2572 6. (*Applied.*) A jurisdiction reports a benchmark map with 94 per cent overall accuracy, deforestation user's
 2573 accuracy of 0.81 and producer's accuracy of 0.62. State whether it meets VM0048, what the failure implies
 2574 about the direction of bias in the reported area, and what you would ask the proponent to do next.

10. Baseline Modelling

10.1 Counterfactual accounting

Every chapter so far has measured something. This one does not, and that is the difficulty. A carbon credit is issued for the difference between what happened and what would have happened otherwise, and the second term is a counterfactual: a quantity that by construction cannot be observed, and against which the project's performance is scored.

The baseline is that counterfactual, expressed as a rate. It is the single most consequential number in a carbon project, because it multiplies through everything: a baseline set ten per cent too high issues ten per cent more credits for the same forest, indefinitely. It is also the number most exposed to the proponent's discretion, since the proponent usually proposes it.

That combination, high leverage and unfalsifiability, explains almost every feature of how the standards handle baselines. Read the rules as a forecasting method and they look crude. Read them as anti-gaming devices and they are carefully built.

10.2 Baseline versus forecast

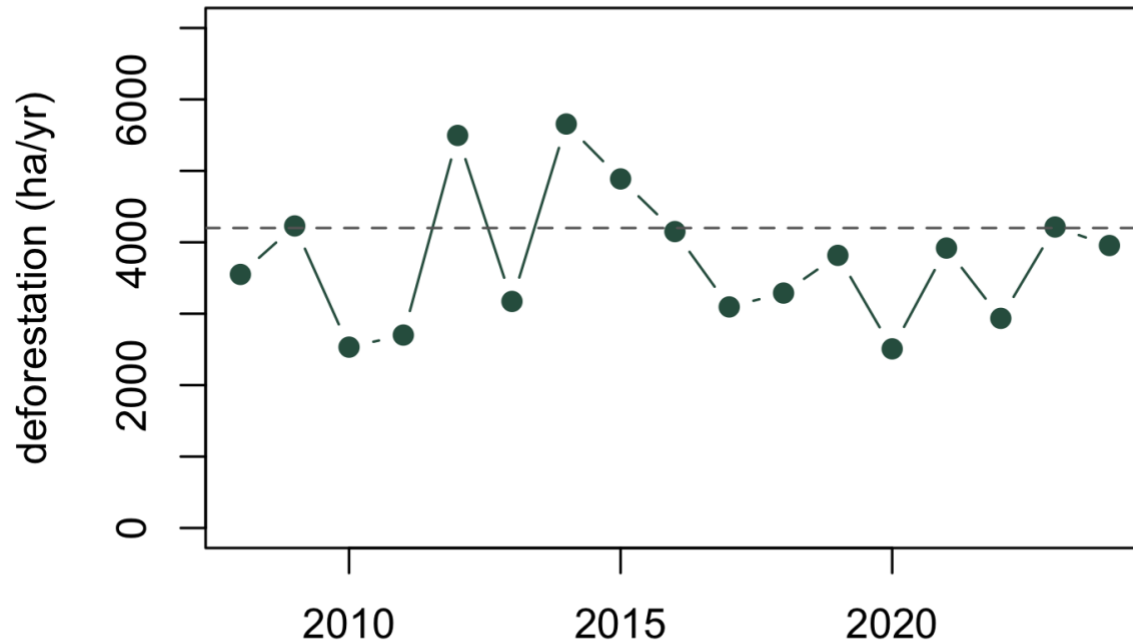
The temptation is to treat baseline construction as a prediction problem: model historical deforestation, extrapolate, and call the projection the baseline. Every major standard declines to do this, and the reason is worth demonstrating rather than asserting.

Take a jurisdiction whose deforestation rate is genuinely flat at 4,200 hectares a year, observed for seventeen years with the year-to-year variability any real record has.

```
set.seed(11)
years <- 2008:2024
true_rate <- 4200 # ha/yr, genuinely constant
obs <- round(rnorm(length(years), true_rate, 1100))
names(obs) <- years

plot(years, obs, type = "b", pch = 19, col = "#2f5d4e", ylim = c(0, 7000),
      xlab = "", ylab = "deforestation (ha/yr)",
      main = "Seventeen years, no underlying trend")
abline(h = true_rate, lty = 2, col = "grey40")
```

Seventeen years, no underlying trend



2604

2605 There is no trend here. Now build a baseline two ways from every possible five-year reference
 2606 window: as the mean of the window, and as a fitted linear trend projected forward over a six-year
 2607 validity period.

```

2608 wins <- 1:(length(years) - 4)
2609 res <- as.data.frame(t(sapply(wins, function(i) {
2610   k <- i:(i + 4); y <- years[k]; v <- obs[k]
2611   fit <- lm(v ~ y)
2612   proj <- predict(fit, data.frame(y = max(y) + 1:6))
2613   c(start = min(y), mean_baseline = mean(v), trend_baseline = mean(proj))
2614 })))
2615 knitr::kable(round(res, 0), row.names = FALSE)
  
```

start	mean_baseline	trend_baseline
2008	3702	5002
2009	3626	4094
2010	3911	7607
2011	4382	6875

start	mean_baseline	trend_baseline
2012	4672	4135
2013	4192	3279
2014	4215	626
2015	3848	2197
2016	3372	1963
2017	3326	3802
2018	3294	2960
2019	3479	4150
2020	3506	5260

```

2616 round(c(mean_sd = sd(res$mean_baseline),
2617         trend_sd = sd(res$trend_baseline),
2618         mean_range = diff(range(res$mean_baseline)),
2619         trend_range = diff(range(res$trend_baseline)),
2620         ratio = diff(range(res$trend_baseline)) /
2621               diff(range(res$mean_baseline))), 1)
2622 #>      mean_sd  trend_sd mean_range trend_range      ratio
2623 #>      441.0    1922.2    1378.6    6980.8      5.1

```

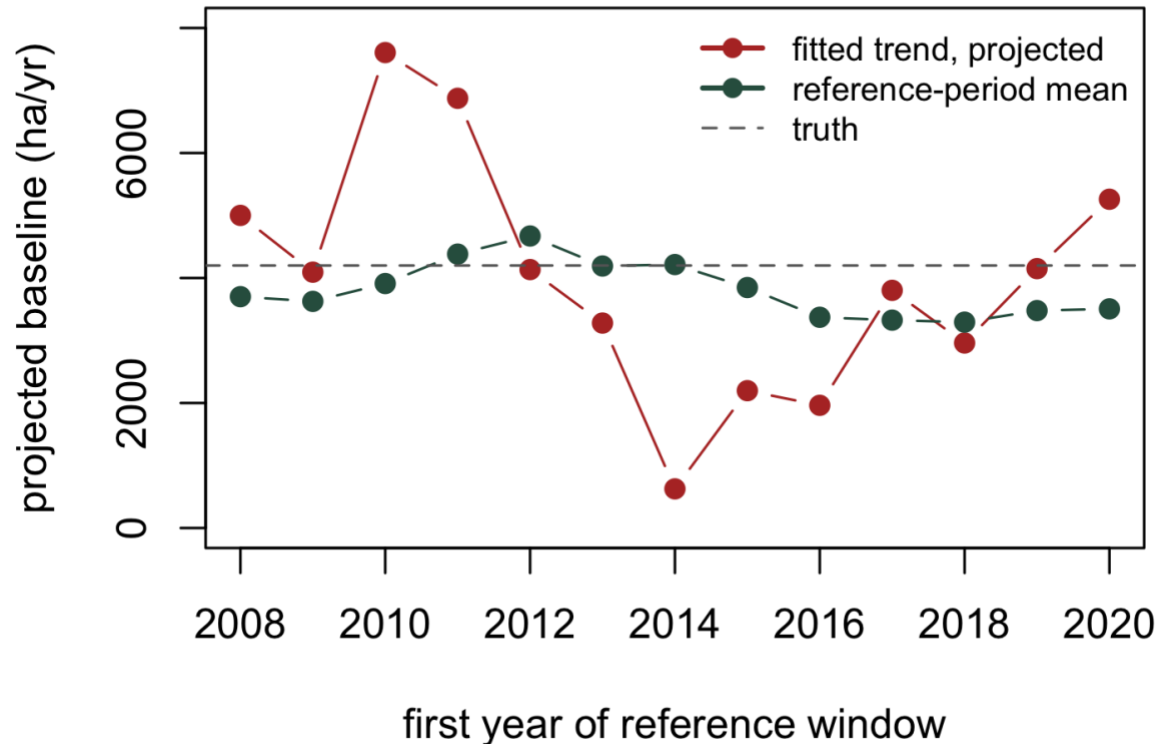
2624 The mean baseline moves by about 1,400 hectares a year across every choice of window. The
2625 trend baseline moves by nearly 7,000, from a low of about 600 to a high of over 7,600. That is a
2626 twelvefold spread in the crediting level, produced from a single dataset with no underlying trend
2627 at all, by nothing more than the choice of which five years to call the reference period.

```

2628 plot(res$start, res$trend_baseline, type = "b", pch = 19, col = "#b5342e",
2629      ylim = c(0, 8000), xlab = "first year of reference window",
2630      ylab = "projected baseline (ha/yr)",
2631      main = "The same data, two baseline methods")
2632 lines(res$start, res$mean_baseline, type = "b", pch = 19, col = "#2f5d4e")
2633 abline(h = true_rate, lty = 2, col = "grey40")
2634 legend("topright", c("fitted trend, projected", "reference-period mean", "truth"),
2635      col = c("#b5342e", "#2f5d4e", "grey40"), lwd = c(2, 2, 1),
2636      lty = c(1, 1, 2), pch = c(19, 19, NA), bty = "n", cex = 0.8)

```

The same data, two baseline methods



2637

2638 A proponent free to fit a trend and choose the window has been handed a dial. Nothing in the
 2639 fitted model is dishonest; each regression is a correct summary of its five years. The problem is
 2640 that a short noisy series does not contain enough information to identify a trend, so the fit reports
 2641 sampling noise as signal, and the proponent picks the noise they prefer.

2642 Mandating an average removes the dial. It is a worse forecast and a far better rule, and the
 2643 distinction between those two things is the whole of baseline design.

2644 10.3 Standard requirements

2645 Two of the major programmes, arrived at independently, converge on almost the same structure.

2646 Under ART-TREES, the crediting level is the mean of a five-year historical reference period,
 2647 updated every five years, with the reference period barred from overlapping the crediting period
 2648 (Architecture for REDD+ Transactions, 2021, Eq. 1). Under Verra's jurisdictional module for
 2649 unplanned deforestation, historical activity data over a historical reference period is discounted

for uncertainty, annualised, and projected flat across a six-year validity period, after which it must be rebuilt (Verra, 2024).

Neither fits a curve. Both fix the window length in advance so it cannot be chosen after the fact. Both require the reference period to end before crediting begins.

10.4 Ratchet mechanism

ART adds a device that is easy to miss and does a great deal of work. An updated crediting level may not be higher than the one it replaces. If the recomputed five-year mean comes out above the previous level, the previous level stands.

```
crediting_levels <- numeric(0)
for (p in 1:3) {
  k <- (p - 1) * 5 + 1:5
  raw <- mean(obs[k])
  crediting_levels[p] <- if (p == 1) raw else min(raw, crediting_levels[p - 1])
  cat(sprintf("period %d (%d-%d): five-year mean %.0f -> crediting level %.0f%s\n",
    p, years[min(k)], years[max(k)], raw, crediting_levels[p],
    if (p > 1 && raw > crediting_levels[p - 1]) " capped" else ""))
}
#> period 1 (2008-2012): five-year mean 3702 -> crediting level 3702
#> period 2 (2013-2017): five-year mean 4192 -> crediting level 3702 capped
#> period 3 (2018-2022): five-year mean 3294 -> crediting level 3294
```

In the second period the recomputed mean rises to 4,192 and the ratchet holds the crediting level at 3,702. In the third it falls and the lower value is adopted. The baseline can therefore only ever decline, regardless of what the forest does.

This is not a statistical adjustment and does not pretend to be. It is a one-way valve, and its purpose is to prevent a jurisdiction from being rewarded for a bad five years. Without it, a period of high deforestation raises the counterfactual and every subsequent year of ordinary performance looks like an achievement.

10.5 Discounting for uncertainty

Verra's module requires the historical estimate to be conservatively discounted for its own statistical uncertainty before it becomes a baseline. This is a different use of uncertainty from the crediting deduction of the uncertainty chapter, and it points the same way: the less precisely the historical rate is known, the lower the baseline a project may claim against.

```
ad_estimate <- 4200
u <- c(0.05, 0.10, 0.20, 0.35)
data.frame(uncertainty_pct = 100 * u,
  discounted_baseline = round(ad_estimate * (1 - u)),
  pct_of_estimate = round(100 * (1 - u)))
```

uncertainty_pct	discounted_baseline	pct_of_estimate
5	3990	95
10	3780	90
20	3360	80
35	2730	65

The incentive this creates runs the opposite way from the crediting deduction, and the two together are what make the system coherent. Sloppy historical work lowers the baseline, so it costs credits. Sloppy monitoring work raises the deduction, so it costs credits. There is no measurement corner a proponent can cut that pays.

Note also which quantity is discounted. The uncertainty here is on the *activity data*, the hectares, which is the term the classification chapter was about. A jurisdiction whose historical map fails the accuracy thresholds does not merely have a caveat attached to its baseline; it has a materially lower one.

10.6 Jurisdictional allocation

A jurisdictional baseline is a rate for a whole territory, and a project occupies a fraction of it. Allocating the jurisdictional rate to project areas cannot be done by area alone, because deforestation is not spatially uniform: a project on a road frontage faces very different pressure from one in the interior.

The standard device is a risk surface. Each pixel receives a share of the zonal rate proportional to its modelled risk, so the allocation sums back to the jurisdictional total by construction:

$$\text{Allocated}_{\text{pixel}} = \frac{\text{risk}_{\text{pixel}}}{\sum_{\text{zone}} \text{risk}} \times \text{Rate}_{\text{zone}}$$

```
library(terra)
library(sf)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
  coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
r <- rast(ext(vect(stems)), resolution = 10, crs = "EPSG:32617")
xy <- xyFromCell(r, 1:ncell(r))

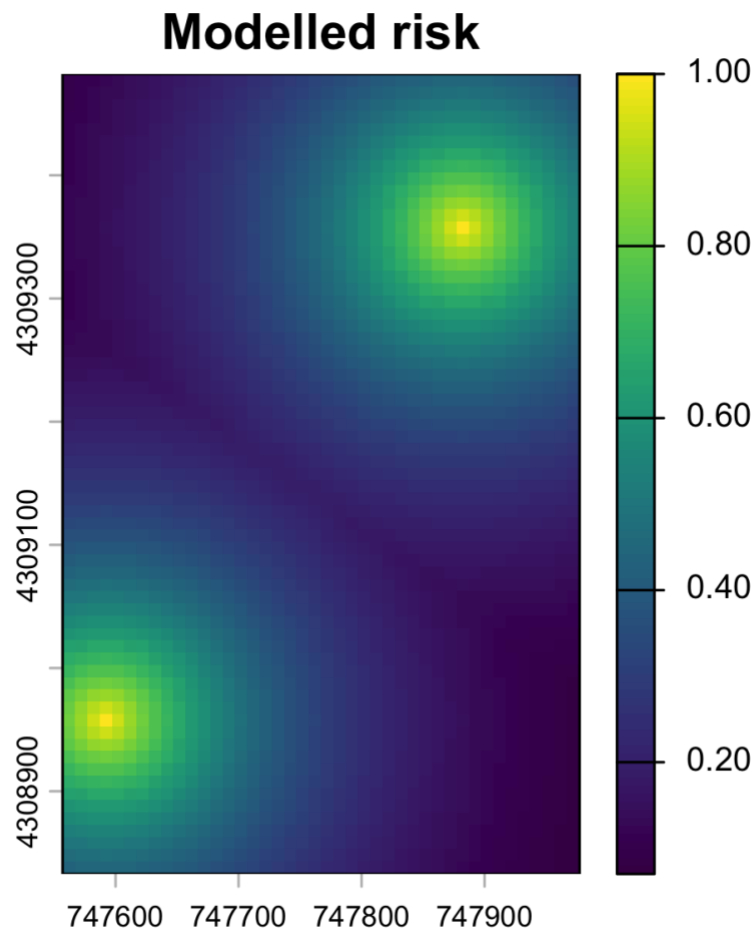
# a risk surface: pressure decays with distance from two access points
e <- ext(r)
access <- vect(cbind(c(e$xmin + 40, e$xmin + 330),
  c(e$ymin + 120, e$ymin + 520)), crs = crs(r))
d_acc <- distance(rasterize(access, r))
risk_r <- exp(-d_acc / 150); names(risk_r) <- "risk"
risk <- values(risk_r)[, 1]

zone_rate <- 40 # ha/yr across the whole area
```

```

2721 alloc <- risk / sum(risk) * zone_rate
2722
2723 plot(risk_r, main = "Modelled risk")

```



```

2724
2725 round(c(zone_rate = zone_rate, allocated_total = sum(alloc)), 3)
2726 #>      zone_rate allocated_total
2727 #>      40          40

```

Only the risk surface is drawn, because the allocated surface is the same map multiplied by a constant: dividing by the sum and scaling to the zone rate changes the units, not the pattern. That is precisely what makes the allocation conserve the total, and the conservation is what makes it auditable. A verifier can sum the project allocations and check they do not exceed the jurisdictional rate. Without that constraint, projects in aggregate can be credited against more deforestation than the jurisdiction has.

```

2734 project <- xy[, 1] < quantile(xy[, 1], 0.25) # a project on the high-risk edge
2735 round(c(project_share_of_area = 100 * mean(project),
2736         project_share_of_baseline = 100 * sum(alloc[project]) / sum(alloc)), 1)
2737 #>      project_share_of_area project_share_of_baseline
2738 #>      23.8              25.4

```

2739 The project occupies under a quarter of the area and carries just over half the baseline, because
2740 it sits where the pressure is. That asymmetry is the entire reason risk allocation exists, and it is
2741 also where the arguments happen: the risk model's covariates and decay parameters are chosen
2742 by someone, and they determine how many credits the project may claim. A verifier who accepts
2743 a jurisdictional rate without interrogating the allocation has checked the easy half.

2744 *Reference periods*

2745 The mechanics differ between programmes and the differences matter for anyone reproducing a
2746 baseline.

2747 **ART-TREES.** The crediting level is the mean of a five-year reference period, recomputed every
2748 five calendar years starting with the first year of crediting. The reference period may not overlap
2749 the crediting period and there may be no gap between them. An updated crediting level may not
2750 exceed the previous one; the sole exception is where a new pool or activity is added, which
2751 requires the whole five-year reference to be recalculated with it included. Interpolation is allowed
2752 where data do not fall on the calendar years, provided no bias is introduced in selecting the data
2753 used.

2754 **Verra, jurisdictional unplanned deforestation.** Historical activity data are estimated over a
2755 historical reference period, conservatively discounted for statistical uncertainty, annualised, and
2756 used to project forward. The validity period is six years and the baseline must be updated at the
2757 end of each one. Four activity data categories must be estimated in every application: historical
2758 deforestation, stable forest, stable non-forest and forest regrowth.

2759 One provision is worth emphasising because it contradicts a widespread assumption. Wall-to-wall
2760 mapping is **not required** to estimate the activity data. The estimate comes from a probability
2761 sample; maps may be used to stratify that sample and reduce the effort needed to hit precision
2762 targets, but the map is not the estimator. This is the same architecture the classification chapter
2763 described and the uncertainty chapter's area adjustment implements.

2764 Source note: the jurisdictional module consulted here is issued as a draft for public consultation,
2765 so check its status before relying on the detail. The ART provisions are from the published
2766 Standard.

2767 *Verifier questions*

2768 Four questions, in the order they usually expose problems.

2769 *Was the reference period fixed before the data were seen?* A window chosen after inspecting the
2770 series is a chosen result. Check the dates against the programme's rules and against the project's
2771 own earlier documents.

2772 *Is the method the mandated one?* If a proponent has fitted a trend, modelled drivers, or applied a
2773 development adjustment, that is a departure requiring explicit permission, not an improvement.

2774 *What uncertainty was carried, and which way did it push?* The historical estimate should be
2775 discounted, not merely reported with a caveat. Ask to see the sample design behind it.

2776 *Does the allocation conserve the jurisdictional total?* Sum the allocations across all projects and
2777 leakage belts in the jurisdiction. If they exceed the jurisdictional rate, credits are being issued
2778 twice against the same deforestation.

2779 The recurring theme is that baseline review is mostly documentary rather than computational. The
2780 arithmetic is deliberately simple; what is under examination is whether the choices around it were
2781 constrained in advance.

2782 **10.7 Exercises**

- 2783 1. Rerun the window comparison with the observation noise halved ($s_d = 550$). By how much does the range of
2784 trend baselines shrink, and does it become acceptable? State the sample size or precision at which you would
2785 be willing to fit a trend.
- 2786 2. Introduce a genuine downward trend of 100 ha/yr into the simulated series and repeat the comparison. Does
2787 the mean baseline now systematically over-credit the project, and by how much over a six-year validity period?
- 2788 3. Apply the ART ratchet to a series with a genuine downward trend. Does the ratchet ever bind, and what does
2789 that tell you about which kind of jurisdiction it is designed to constrain?
- 2790 4. Change the risk decay length from 150 m to 400 m and recompute the project's share of the baseline. How
2791 sensitive is the project's crediting entitlement to a parameter that no field measurement determines?
- 2792 5. A jurisdiction reports historical activity data with 30 per cent uncertainty. Compute the discounted baseline,
2793 then compute what the project would have claimed at 10 per cent uncertainty. Express the difference as credits
2794 over a six-year validity period at 4,200 ha/yr and a nominal 150 tCO₂e per hectare.
- 2795 6. (*Applied.*) You are reviewing a project whose allocated baseline is 62 per cent of the jurisdictional rate on 25
2796 per cent of the area. Write the paragraph you would send the proponent, naming the specific evidence you
2797 require to accept the allocation.

11. Uncertainty

11.1 Uncertainty rationale

Every estimate in this book is wrong by some amount, and the discipline of verification is knowing by how much. A carbon figure without an uncertainty is not conservative, it is simply incomplete, and the crediting standards, ART-TREES, VCS, ACR, all deduct credits in proportion to how uncertain the estimate is. Reducing uncertainty is therefore worth real money, and quantifying it honestly is the auditor's core duty. This chapter builds the estimate, then its error, on the running stem data.

11.2 Carbon estimate

We convert diameters to aboveground biomass with a published generic hardwood equation (Jenkins et al., 2003), apply the IPCC default carbon fraction of 0.47 (IPCC, 2006), and scale to the plot area:

```
library(dplyr)
stems <- read.csv("data/scbi_stems.csv")

stems <- stems |>
  mutate(agb_kg = exp(-2.48 + 2.4835 * log(dbh)), # Jenkins generic hardwood
         carbon_Mg = 0.47 * agb_kg / 1000)

area_ha <- 25.6
total_C <- sum(stems$carbon_Mg)
C_per_ha <- total_C / area_ha
round(c(total_MgC = total_C, MgC_per_ha = C_per_ha), 1)
#> total_MgC MgC_per_ha
#> 2871.0 112.1
```

That point estimate is the easy part. Three distinct sources of error sit beneath it: sampling error from measuring plots rather than the whole population, model error in the allometric equation, and parameter error in constants like the carbon fraction.

11.3 Sampling error

Divide the plot into quadrats and each becomes a sample of carbon density. The variation among them is the sampling uncertainty of the plot mean, summarised by the standard error and a confidence interval. If the distinction between a standard deviation and a standard error is not immediate, the [statistical primers appendix](#) covers it before you go further:

```
stems$quadrat <- paste(cut(stems$gx, 8), cut(stems$gy, 12))
quadrat_area <- 25.6 / (8 * 12) # ha per quadrat (equal-area cut)

qd <- stems |>
  group_by(quadrat) |>
  summarise(carbon_Mg = sum(carbon_Mg), .groups = "drop") |>
  mutate(carbon_ha = carbon_Mg / quadrat_area)
```

```

2839 m <- mean(qd$carbon_ha); se <- sd(qd$carbon_ha) / sqrt(nrow(qd))
2840 ci <- m + c(-1.96, 1.96) * se
2841 round(c(mean = m, se = se, lower = ci[1], upper = ci[2]), 1)
2842 #> mean      se lower upper
2843 #> 112.1    2.4 107.4 116.9

```

2844 The half-width relative to the mean is the number the standards care about: the wider it is, the
 2845 deeper the deduction.

2846 11.4 Measuring error

2847 Sampling error assumed the per-tree biomass was right and asked only how well the plot
 2848 represents the stand. The other half of the problem is the model itself, and to quantify that you
 2849 need trees whose biomass was actually measured rather than predicted. The `tree_measurements` file
 2850 holds 6,681 harvested trees across six coastal species, each with diameter, height and observed
 2851 aboveground biomass, so predictions can be checked against truth.

2852 No single number describes an error well, and the common metrics answer genuinely different
 2853 questions. Written on the residual $e_i = \hat{y}_i - y_i$:

$$2854 \quad \text{bias} = \frac{1}{n} \sum e_i \quad \text{MAE} = \frac{1}{n} \sum |e_i| \quad \text{MSE} = \frac{1}{n} \sum e_i^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

$$2855 \quad \text{MAPE} = \frac{100}{n} \sum \left| \frac{e_i}{y_i} \right| \quad R^2 = 1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2}$$

2856 Bias is signed and measures systematic error: it can be near zero while every individual prediction
 2857 is badly wrong, because overestimates cancel underestimates. MAE is the typical error magnitude
 2858 in the units of the data. MSE squares the residuals, so it is dominated by the largest ones, and
 2859 RMSE is its square root, returned to the original units and therefore directly readable. Whether
 2860 you prefer MAE or RMSE is a statement about how much you care about occasional large errors
 2861 relative to many small ones, and in biomass work you should care a great deal, because a handful
 2862 of large trees carry most of the carbon. Relative RMSE, the RMSE as a percentage of the
 2863 observed mean, is what makes results comparable across species and datasets, and it is the
 2864 quantity the crediting standards consume.

2865 Two definitions of R^2 circulate. The one above, one minus the residual sum of squares over the
 2866 total, is the proportion of variance explained and can go negative for a bad model. The alternative,
 2867 the squared correlation between observed and predicted, cannot go negative and will flatter a
 2868 model with a systematic offset. This book uses the first throughout.

```

2869 tm <- read.csv("data/tree_measurements.csv")
2870

```

```

2871 metrics <- function(obs, prd) {
2872   e <- prd - obs
2873   c(bias = mean(e),
2874     rel_bias = 100 * mean(e) / mean(obs),
2875     mae = mean(abs(e)),
2876     rmse = sqrt(mean(e^2)),
2877     rel_rmse = 100 * sqrt(mean(e^2)) / mean(obs),
2878     mape = 100 * mean(abs(e / obs)),
2879     r2 = 1 - sum(e^2) / sum((obs - mean(obs))^2))
2880 }

```

2881 Fit the standard log-log allometry. Fitting on logs stabilises the variance, but exponentiating the
 2882 prediction returns the median rather than the mean and biases the result low, so the fit carries the
 2883 Baskerville correction factor $\exp(\hat{\sigma}^2/2)$ (Baskerville, 1972):

```

2884 fit <- lm(log(agb_kg) ~ log(dbh_cm), data = tm)
2885 cf <- exp(sigma(fit)^2 / 2) # Baskerville correction
2886 pred <- exp(predict(fit, tm)) * cf
2887
2888 round(metrics(tm$agb_kg, pred), 3)
2889 #>      bias rel_bias      mae      rmse rel_rmse      mape      r2
2890 #>  1.959   0.271  68.948  160.516   22.216   10.358   0.983
2891 round(metrics(tm$agb_kg, exp(predict(fit, tm))), 3) # uncorrected
2892 #>      bias rel_bias      mae      rmse rel_rmse      mape      r2
2893 #> -3.429  -0.475  69.633  160.938   22.274   10.387   0.983

```

2894 The correction moves the relative bias from about -0.5 per cent to $+0.3$ per cent. That is small
 2895 here and much larger in datasets with more residual scatter, and it is free, so there is no reason
 2896 to omit it.

2897 Now the reason to compute more than one metric. Split the same predictions by diameter class:

```

2898 tm$ccls <- cut(tm$dbh_cm, c(0, 20, 40, Inf), labels = c("<20", "20-40", ">40"))
2899 round(t(sapply(split(seq_len(nrow(tm)), tm$ccls),
2900               function(i) metrics(tm$agb_kg[i], pred[i]))), 2)
2901 #>      bias rel_bias      mae      rmse rel_rmse      mape      r2
2902 #> <20   -0.44   -0.25  17.67  22.51   12.89  10.68  0.91
2903 #> 20-40 -0.81   -0.13  59.13  75.80   12.53  10.26  0.91
2904 #> >40   18.33    0.67 252.24 409.16   15.02   9.78  0.97

```

2905 MAPE is almost flat across the three classes, near ten per cent everywhere, while RMSE climbs
 2906 from about 23 kg to over 400. Both are correct. The model is proportionally about as good on a
 2907 large tree as on a small one, and in absolute terms it is eighteen times worse, which is what
 2908 matters when those large trees hold most of the stand's carbon. A report quoting only the
 2909 percentage error conceals where the tonnes actually go.

2910 MAPE carries a specific hazard worth stating plainly: it divides by the observation, so it explodes
 2911 as y_i approaches zero and is undefined at zero. It is safe here because every harvested tree has
 2912 substantial biomass. Applied to a variable that legitimately reaches zero, such as annual forest
 2913 loss in a quiet year, it is meaningless, and that is the usual reason it is absent from carbon work.

Finally, the single most effective way to reduce model error is usually not a cleverer equation but a narrower one:

```
pred_sp <- numeric(nrow(tm))
for (s in unique(tm$species)) {
  i <- tm$species == s
  m <- lm(log(agb_kg) ~ log(dbh_cm), data = tm[i, ])
  pred_sp[i] <- exp(predict(m, tm[i, ])) * exp(sigma(m)^2 / 2)
}
round(metrics(tm$agb_kg, pred_sp), 3)
#>      bias rel_bias      mae      rmse rel_rmse      mape      r2
#>    0.166    0.023   36.341   81.161   11.233    5.069    0.996
```

Fitting one equation per species halves the relative RMSE, from about 22 per cent to about 11. Under the crediting arithmetic below, that difference is worth real money, and it required no new field data, only refusing to pool species that scale differently.

11.5 Cross-validation

Every number in the previous section is optimistic, because the model was scored on the same trees that determined its coefficients. Out-of-sample error is what a prediction on a new tree will actually cost, and cross-validation estimates it by repeatedly holding data back (Hastie et al., 2009).

The standard device is k -fold: split the data into k parts, fit on $k - 1$ and predict the one held out, rotating until every observation has been predicted by a model that never saw it.

```
set.seed(1)
k <- 10
fold <- sample(rep(1:k, length.out = nrow(tm)))
p_cv <- numeric(nrow(tm))
for (i in 1:k) {
  m <- lm(log(agb_kg) ~ log(dbh_cm), data = tm[fold != i, ])
  p_cv[fold == i] <- exp(predict(m, tm[fold == i, ])) * exp(sigma(m)^2 / 2)
}
round(metrics(tm$agb_kg, p_cv), 3)
#>      bias rel_bias      mae      rmse rel_rmse      mape      r2
#>    1.944    0.269   69.050   161.257   22.318   10.362    0.982
round(metrics(tm$agb_kg, pred)[ "rmse" ] / metrics(tm$agb_kg, p_cv)[ "rmse" ], 4)
#>      rmse
#>    0.9954
```

The cross-validated error is essentially identical to the in-sample error, a ratio of 0.995. This is the honest result and it is worth dwelling on, because the reflex that cross-validation always exposes a worse number is wrong. Two parameters fitted to 6,681 observations cannot overfit; there is nothing to discover. Optimism is a function of model flexibility relative to sample size, not a tax on having been evaluated.

Where it does bite is where forest data usually sits: a small sample and a model free enough to chase it. Take twenty-five trees and fit polynomials of rising degree, scoring each both in-sample and by leave-one-out:

```

2957 set.seed(7)
2958 s <- tm[sample(nrow(tm), 25), ]
2959 rmse <- function(a, b) sqrt(mean((a - b)^2))
2960
2961 cv_poly <- t(sapply(1:8, function(d) {
2962   f <- lm(log(agb_kg) ~ poly(log(dbh_cm), d), data = s)
2963   loo <- sapply(seq_len(nrow(s)), function(i) {
2964     m <- lm(log(agb_kg) ~ poly(log(dbh_cm), d), data = s[-i, ])
2965     exp(predict(m, s[i, ]))
2966   })
2967   c(degree = d, in_sample = rmse(s$agb_kg, exp(fitted(f))),
2968     loocv = rmse(s$agb_kg, loo))
2969 }))
2970 round(cv_poly, 1)
2971 #>      degree in_sample loocv
2972 #> [1,]      1      100.0 114.1
2973 #> [2,]      2       90.0 114.5
2974 #> [3,]      3       94.2 130.5
2975 #> [4,]      4       82.5 124.9
2976 #> [5,]      5       84.8 168.4
2977 #> [6,]      6       89.9 160.0
2978 #> [7,]      7       72.6 181.2
2979 #> [8,]      8      100.1 398.7

```

The two columns move in opposite directions. In-sample RMSE falls as the curve bends to meet the points; leave-one-out RMSE rises, and by degree eight it is more than three times the honest error of a straight line. Chosen on fit alone, the worst model looks the best. This is the entire argument for validating out of sample, and it is why the straight log-log line has survived a century of allometry.

Which regime to use is a real choice. Leave-one-out fits n models, is nearly unbiased, but has high variance and costs n fits, which is fine at 25 trees and absurd at 6,681. Ten-fold costs ten fits and trades a little bias for much lower variance, and is the usual default. Monte Carlo cross-validation, also called repeated leave-group-out, draws a fresh random split many times, which decouples the number of repetitions from the sample size and, because every repetition gives a complete error estimate, yields a distribution rather than a point. For biomass work the split should be stratified by size class, so that a random draw cannot under-represent the large trees that dominate the total:

```

2993 set.seed(8787)
2994 mc <- t(replicate(100, {
2995   tr <- unlist(lapply(split(seq_len(nrow(tm)), tm$cls),
2996     function(i) sample(i, round(0.8 * length(i)))))
2997   m <- lm(log(agb_kg) ~ log(dbh_cm), data = tm[tr, ])
2998   te <- setdiff(seq_len(nrow(tm)), tr)
2999   metrics(tm$agb_kg[te], exp(predict(m, tm[te, ])) * exp(sigma(m)^2 / 2))
3000 })
3001 round(rbind(mean = colMeans(mc), sd = apply(mc, 2, sd)), 3)

```

```

3002 #>      bias rel_bias      mae      rmse rel_rmse      mape      r2
3003 #> mean 2.488      0.344 68.913 158.667 21.912 10.351 0.982
3004 #> sd 4.260      0.588 3.156 18.946 2.372 0.168 0.004
3005 round(quantile(mc[, "rel_rmse"], c(0.025, 0.975)), 2)
3006 #> 2.5% 97.5%
3007 #> 17.97 26.75

```

3008 The mean relative RMSE agrees with the ten-fold figure, but the spread across repetitions is the
3009 extra information: the estimate is itself uncertain, and quoting it to three significant figures from a
3010 single split would overstate what is known. That distribution is the honest input to a deduction,
3011 and it is what the standards are asking for when they require an uncertainty estimate rather than
3012 a residual statistic.

3013 *Metric to deduction*

3014 The crediting standards each convert a relative error into forgone credits, and they do it differently.
3015 ART-TREES scales the half-width of a 90 per cent interval:

```

3016 art_deduction <- function(rel_rmse_pct) {
3017   100 * 0.524417 * (rel_rmse_pct / 100) / 1.645006
3018 }
3019 round(c(in_sample = art_deduction(metrics(tm$agb_kg, pred)["rel_rmse"]),
3020       cross_val = art_deduction(mean(mc[, "rel_rmse"])),
3021       species = art_deduction(metrics(tm$agb_kg, pred_sp)["rel_rmse"])), 2)
3022 #> in_sample.rel_rmse      cross_val      species.rel_rmse
3023 #>      7.08      6.99      3.58

```

3024 The deduction is roughly proportional to the relative RMSE, so halving the error by fitting per
3025 species roughly halves the credits forgone. Note also that the in-sample and cross-validated
3026 figures barely differ here, for the reason given above, and that a project quoting the in-sample
3027 number on a small dataset would be understating its own deduction.

3028 That is the loop this chapter has been building: a metric, estimated honestly out of sample,
3029 converted into a quantity of carbon that the project does not get paid for. The arithmetic differs
3030 between registries but the logic does not, and the comparison is drawn out at the end of the
3031 chapter.

3032 *Model eligibility*

3033 Where a carbon estimate comes from a remotely sensed model rather than a field tally, ACR's
3034 Framework for Remotely Sensed Quantification of Forest Carbon (v1.0, 2026) gates the model
3035 before any credit is calculated. Validation plots are installed independently of the calibration data,
3036 and the model must clear two tests: uncertainty computed from RMSE below 20 per cent, and
3037 uncertainty at the 90 per cent confidence interval at or below 10 per cent.

```

3038 acr_eligibility <- function(obs, prd) {
3039   n <- length(obs); cbar <- mean(obs)
3040   rmse <- sqrt(mean((obs - prd)^2))

```

```

3041 ci90 <- rmse / sqrt(n) * 1.645
3042 c(n = n, rmse = rmse,
3043   unc_rmse = 100 * rmse / cbar,      # threshold < 20
3044   unc_ci    = 100 * ci90 / cbar)     # threshold <= 10
3045 }
3046
3047 fp <- read.csv("data/forest_plots.csv")
3048 set.seed(12)
3049 fp$C_pred <- fp$carbon * exp(rnorm(nrow(fp), 0, 0.17)) # a model with real error
3050 round(acr_eligibility(fp$carbon, fp$C_pred), 2)
3051 #>      n      rmse unc_rmse  unc_ci
3052 #> 140.00  28.25  17.43    2.42

```

3053 The two tests are not independent, and seeing why matters more than running them. Since the
3054 confidence interval is the RMSE divided by the root of the plot count, the second statistic is the
3055 first one rescaled:

$$3056 \quad \text{UNC}_{CI} = \text{UNC}_{RMSE} \times \frac{1.645}{\sqrt{N}}$$

3057 The thresholds therefore bind equally when $1.645/\sqrt{N} = 10/20$, which is $N \approx 11$ plots. Above
3058 about eleven validation plots the confidence-interval test is satisfied automatically whenever the
3059 RMSE test passes, so only one test is doing any work.

```

3060 n_needed <- function(unc_rmse) ceiling((unc_rmse * 1.645 / 10)^2)
3061 sapply(c(10, 14, 18, 20, 22), n_needed) # plots needed to clear the CI test
3062 #> [1] 3 6 9 11 14

```

3063 There is a second, less comfortable property. Run the same model through different draws of
3064 validation plots and eligibility itself moves:

```

3065 sizes <- c(10, 15, 20, 30)
3066 t(sapply(sizes, function(n) {
3067   set.seed(n); i <- sample(nrow(fp), n)
3068   a <- acr_eligibility(fp$carbon[i], fp$C_pred[i])
3069   c(N = n, unc_rmse = round(unname(a["unc_rmse"]), 1),
3070     eligible = unname(a["unc_rmse"] < 20 & a["unc_ci"] <= 10))
3071 })))
3072 #>      N unc_rmse eligible
3073 #> [1,] 10    14.5        1
3074 #> [2,] 15    20.9        0
3075 #> [3,] 20    17.0        1
3076 #> [4,] 30    14.0        1

```

3077 At fifteen plots this model fails and at twenty it passes, on nothing but which plots were drawn.
3078 Eligibility is a random variable, and a project that installs the minimum viable number of plots is
3079 buying a coin toss.

3080 11.6 Monte Carlo propagation

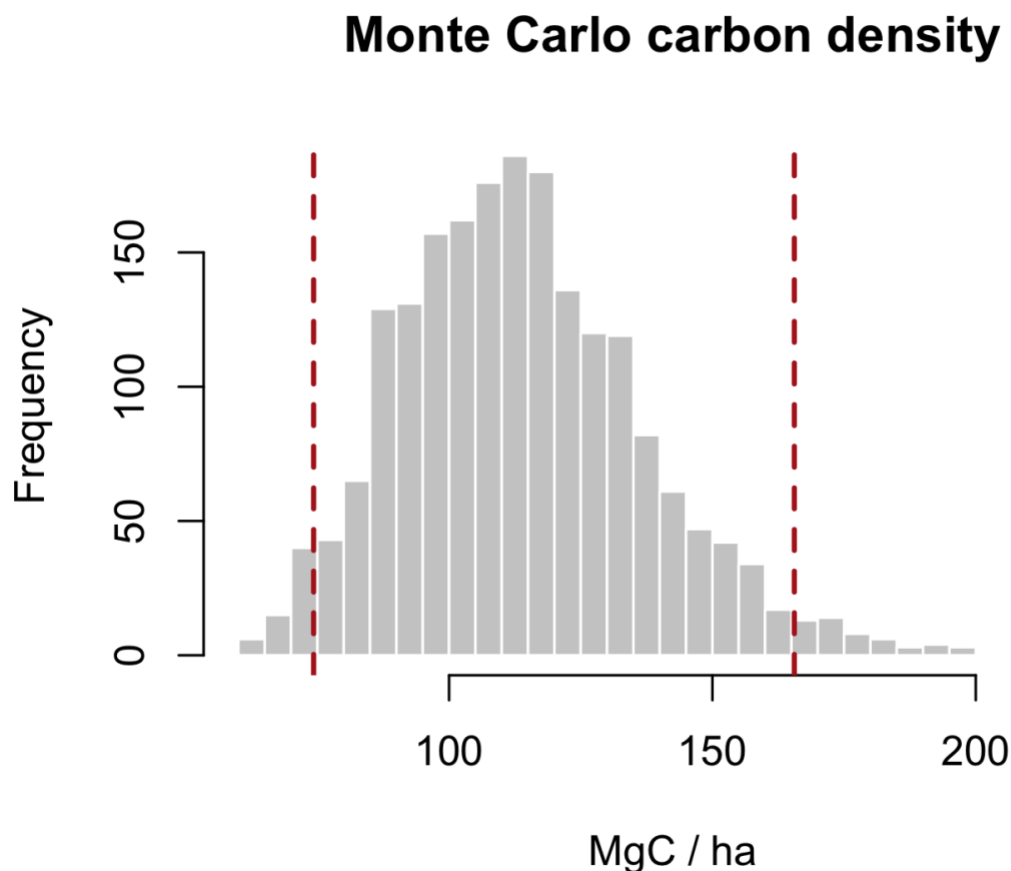
3081 Sampling error is only part of the story; the allometric model and the carbon fraction are uncertain
3082 too, and their errors compound. When error propagates through a non-linear function like aD^b ,
3083 the clean way to combine sources is Monte Carlo simulation: draw the uncertain inputs from their

3084 distributions many times, recompute the whole estimate each time, and read the spread of results
 3085 (IPCC, 2006). The [statistical primers appendix](#) sets out why the algebraic alternative, combining
 3086 relative errors in quadrature, is the special case rather than the general rule, and how many
 3087 iterations the simulation needs.

```

3088 set.seed(42)
3089 n_sim <- 2000
3090 sim <- numeric(n_sim)
3091 for (i in seq_len(n_sim)) {
3092   b <- rnorm(1, 2.4835, 0.05)      # allometric exponent uncertainty
3093   cf <- rnorm(1, 0.47, 0.02)      # carbon-fraction uncertainty
3094   agb <- exp(-2.48 + b * log(stems$dbh))
3095   sim[i] <- sum(cf * agb / 1000) / area_ha
3096 }
3097 mc_ci <- quantile(sim, c(0.025, 0.975))
3098 round(c(mean = mean(sim), lower = mc_ci[1], upper = mc_ci[2]), 1)
3099 #>      mean  Lower.2.5% upper.97.5%
3100 #>    113.9      74.3     165.6
3101
3102 hist(sim, breaks = 40, col = "grey80", border = "white",
3103       main = "Monte Carlo carbon density", xlab = "MgC / ha")
3104 abline(v = mc_ci, col = "firebrick", lwd = 2, lty = 2)

```



3104

11.7 Conservative estimates

The standards translate uncertainty into a deduction. A common rule is that when the half-width of the 90% or 95% interval exceeds a threshold (often 15% of the mean), credits are issued against the conservative lower bound rather than the mean. Computing the uncertainty percentage and the conservative estimate is a few lines:

```
half_width <- (mc_ci[2] - mc_ci[1]) / 2
unc_pct    <- 100 * half_width / mean(sim)
conservative <- mean(sim) - half_width      # Lower-bound crediting basis
round(c(uncertainty_pct = unc_pct,
        mean = mean(sim), conservative = conservative), 1)
#> uncertainty_pct.97.5%      mean      conservative.97.5%
#>                40.1          113.9          68.2
```

If the uncertainty percentage clears the threshold, the gap between the mean and the conservative figure is credit the project forgoes for being imprecise, the direct financial incentive to measure better.

11.8 Area-adjusted accuracy

The Bolivia land-cover work raises the same question for areas rather than carbon: a class area read straight off a map inherits the map's classification errors, and good practice adjusts it and attaches a confidence interval (Olofsson et al., 2014). This is the LULC counterpart of the carbon deduction, and it uses the confusion matrix from the disturbance chapter. We take mapped forest and non-forest areas and an error matrix, and compute the area-adjusted estimate:

```
A <- c(forest = 18000, nonforest = 7000)      # mapped area, ha
Atot <- sum(A); W <- A / Atot

# error matrix: rows = map class, cols = reference class
n <- matrix(c(95, 5, 12, 88), nrow = 2, byrow = TRUE,
            dimnames = list(map = names(A), ref = names(A)))
ni <- rowSums(n)

# Olofsson stratified estimator of the reference area proportion of "forest"
p_forest <- sum(W * n[, "forest"] / ni)
se_p <- sqrt(sum(W^2 * (n[, "forest"]/ni) * (1 - n[, "forest"]/ni) / (ni - 1)))

adj_forest_ha <- Atot * p_forest
ci_ha <- Atot * 1.96 * se_p
round(c(mapped_forest_ha = A["forest"],
        adjusted_forest_ha = adj_forest_ha,
        ci_halfwidth_ha = ci_ha), 0)
#> mapped_forest_ha.forest      adjusted_forest_ha      ci_halfwidth_ha
#>                18000          17940          893
```

The adjusted area differs from the mapped area because the map commits and omits forest at rates the reference sample reveals, and the confidence interval is the honest precision of the figure. A project that reports only the mapped area, with no adjustment and no interval, has

reported a number no auditor can accept. The full Bolivia workflow applies this class by class across the national legend; the estimator is the one above.

11.9 Reported against adjusted

One year in isolation understates what this correction does. A crediting period runs for a decade, the adjustment is applied every year, and the interesting quantity is the gap between what a project reports and what it can defend. We run the same estimator across an eight-year record of mapped forest loss. Each year has its own reference sample and its own error rates, and both degrade towards the present, because recent imagery has had less interpretation effort and the map is harder to validate close to the reporting date.

```

years <- 2017:2024
Atot <- 250000 # ha in the accounting area
mapped <- c(1200, 2400, 4100, 3300, 3500, 3700, 6900, 9200) # ha mapped as Loss

# reference sample sizes thin out in recent years
n_chg <- c(150, 150, 150, 150, 140, 130, 90, 60)
n_stab <- c(900, 900, 880, 880, 850, 800, 520, 340)
# user's accuracy of the loss class, and omission from the stable stratum
ua_chg <- c(0.79, 0.80, 0.78, 0.80, 0.79, 0.77, 0.72, 0.68)
om_stab <- c(0.0022, 0.0022, 0.0026, 0.0022, 0.0026, 0.0026, 0.0030, 0.0034)

est <- t(sapply(seq_along(years), function(k) {
  W <- unname(c(mapped[k], Atot - mapped[k]) / Atot)
  p <- W[1] * ua_chg[k] + W[2] * om_stab[k]
  se <- sqrt(W[1]^2 * ua_chg[k] * (1 - ua_chg[k]) / (n_chg[k] - 1) +
    W[2]^2 * om_stab[k] * (1 - om_stab[k]) / (n_stab[k] - 1))
  c(adjusted = Atot * p, ci = 1.96 * Atot * se)
}))

adj <- est[, "adjusted"]; ci <- est[, "ci"]
data.frame(year = years, mapped = mapped, adjusted = round(adj),
  lower = round(adj - ci), upper = round(adj + ci),
  bias_pct = round(100 * (mapped - adj) / adj, 1))

```

year	mapped	adjusted	lower	upper	bias_pct
2017	1200	1495	729	2261	-19.8
2018	2400	2465	1691	3239	-2.6
2019	4100	3837	2966	4709	6.8
2020	3300	3183	2390	3976	3.7
2021	3500	3406	2529	4283	2.8
2022	3700	3489	2579	4400	6.0
2023	6900	5697	4385	7010	21.1
2024	9200	7075	5224	8926	30.0

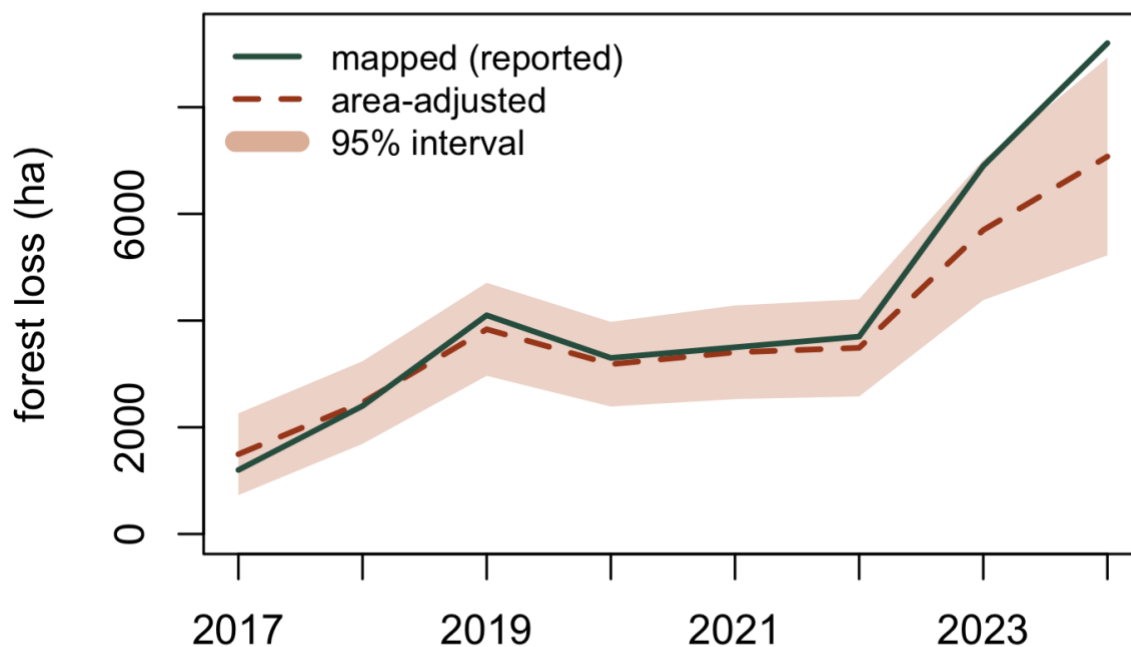
Plotting the two series together with the interval around the adjusted estimate is the single most useful figure a monitoring report can carry:

```

3182 plot(years, mapped, type = "n", ylim = c(0, max(adj + ci) * 1.05),
3183       xlab = "", ylab = "forest loss (ha)",
3184       main = "Mapped against area-adjusted loss")
3185 polygon(c(years, rev(years)), c(adj - ci, rev(adj + ci)),
3186        col = adjustcolor("#c4703f", alpha.f = 0.28), border = NA)
3187 lines(years, adj, col = "#a8481f", lwd = 2.2, lty = 2)
3188 lines(years, mapped, col = "#2f5d4e", lwd = 2.2)
3189 legend("topleft", c("mapped (reported)", "area-adjusted", "95% interval"),
3190       col = c("#2f5d4e", "#a8481f", adjustcolor("#c4703f", alpha.f = 0.5)),
3191       lwd = c(2.2, 2.2, 8), lty = c(1, 2, 1), bty = "n", cex = 0.85)

```

Mapped against area-adjusted loss



3192

3193 Read the two lines against each other. Early in the record the adjustment pushes the estimate
3194 *up*: the mapped loss area is small, so omitted change scattered through the vast stable stratum
3195 outweighs anything the map over-called, and the defensible figure exceeds the reported one. Late
3196 in the record the sign reverses. As the map's user accuracy falls from 0.79 to 0.68, commission
3197 error dominates, and the reported figure overstates what the reference data support by thirty per
3198 cent in the final year.

3199 The direction of the correction is therefore not something you can guess, and the common
3200 assumption that adjustment always shrinks a claim is wrong. It depends on the balance between

3201 what the map missed and what it invented, weighted by stratum area, and only a reference sample
3202 can tell you which way it falls.

```
3203 round(c(mapped_total = sum(mapped),  
3204         adjusted_total = sum(adj),  
3205         overstatement_pct = 100 * (sum(mapped) - sum(adj)) / sum(adj),  
3206         conservative_total = sum(adj - ci),  
3207         forgone_pct = 100 * sum(ci) / sum(adj)), 1)  
3208 #> mapped_total adjusted_total overstatement_pct conservative_total  
3209 #> 34300.0 30647.5 11.9 22492.4  
3210 #> forgone_pct  
3211 #> 26.6
```

3212 Over the whole period the reported total overstates the defensible total by around twelve per cent.
3213 That gap is the project’s exposure: it is the portion of the claim that would not survive a reference-
3214 based check. The conservative basis, summing the lower bounds, is lower again by about a
3215 quarter, and that second gap is the price of imprecision rather than of bias.

3216 Notice which of the two is easier to fix. The bias is corrected by the estimator itself, at the cost of
3217 running a reference sample, and once corrected it is gone. The interval width is set by how many
3218 reference points sit in each stratum, and it narrows only as the square root of that effort. The
3219 widening band in the last two years is not a modelling failure; it is a direct readout of thin reference
3220 data, and the only remedy is more interpretation. This is the whole argument of the chapter in one
3221 picture. Uncertainty is not a statistical afterthought appended to a result. It is a budget, it has line
3222 items, and each line has a known price.

3223 **11.10 Registry deductions**

3224 You now have an uncertainty percentage estimated honestly. What it costs depends entirely on
3225 which registry the project is going to, and the differences are larger than most practitioners expect.
3226 Three rules cover most of the market.

```
3227 acr_ifm <- function(u) pmax(0, u - 10) # excess over threshold  
3228 art_trees <- function(u) u * (0.524417 / 1.645006) # risk-tolerance discount  
3229 vm0048 <- function(u) ifelse(u <= 10, 0, u * (0.4307 / 1.6449))  
3230  
3231 u <- c(5, 10, 15, 20, 30, 50)  
3232 data.frame(uncertainty_pct = u,  
3233            ACR_IFM = round(acr_ifm(u), 2),  
3234            ART_TREES = round(art_trees(u), 2),  
3235            VM0048 = round(vm0048(u), 2))
```

uncertainty_pct	ACR_IFM	ART_TREES	VM0048
5	0	1.59	0.00
10	0	3.19	0.00
15	5	4.78	3.93
20	10	6.38	5.24

uncertainty_pct	ACR_IFM	ART_TREES	VM0048
30	20	9.56	7.86
50	40	15.94	13.09

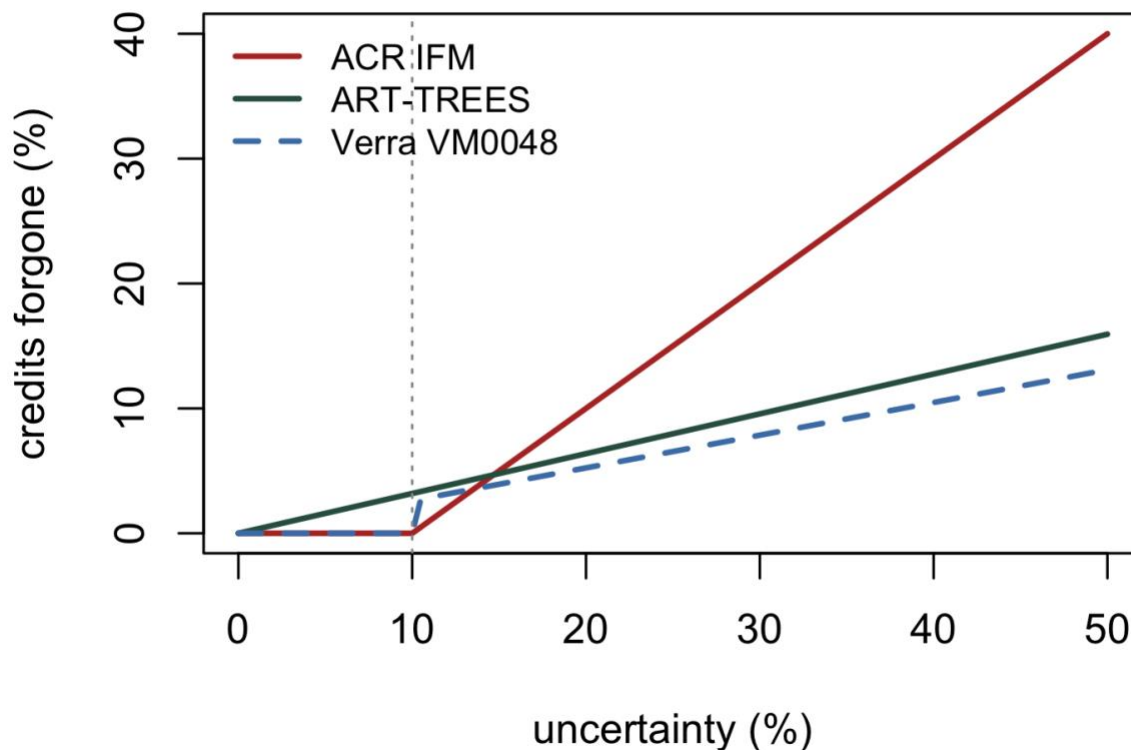
3236 Read the 30 per cent row. The same nominal statistic, at the same nominal 90 per cent
3237 confidence, costs a project twenty per cent of its credits under ACR and under eight per cent
3238 under VM0048. “Ninety per cent confidence” is not a comparable unit across registries, and a
3239 number lifted from one project’s report into another’s is close to meaningless.

```

3240 uu <- seq(0, 50, 0.5)
3241 plot(uu, acr_ifm(uu), type = "l", lwd = 2.2, col = "#b5342e",
3242      xlab = "uncertainty (%)", ylab = "credits forgone (%)",
3243      main = "Three registries, one statistic")
3244 lines(uu, art_trees(uu), lwd = 2.2, col = "#2f5d4e")
3245 lines(uu, vm0048(uu), lwd = 2.2, col = "#4a7fb5", lty = 2)
3246 abline(v = 10, col = "grey60", lty = 3)
3247 legend("topleft", c("ACR IFM", "ART-TREES", "Verra VM0048"),
3248      col = c("#b5342e", "#2f5d4e", "#4a7fb5"),
3249      lwd = 2.2, lty = c(1, 1, 2), bty = "n", cex = 0.85)

```

Three registries, one statistic



3250

3251 The shapes differ because the designs differ, and the split is not the one the market talk suggests.
3252 ART-TREES and modern Verra share a paradigm: multiply the uncertainty by a fixed discount
3253 that encodes how much risk of over-crediting the standard will tolerate, an approach traceable to
3254 Neeff (2021). Both compute the same thing, a one-sided over a two-sided t value, and differ only
3255 in where they set the tolerance. ART is the stricter of the two and, unusually, has no threshold at
3256 all: a project with five per cent uncertainty still forfeits 1.6 per cent of its credits.

3257 ACR belongs to an older design, in which nothing is deducted below a threshold and the excess
3258 above it is forfeited one for one. That produces the crossing you can see in the figure:

```
3259 uu <- seq(10, 40, 0.01)
3260 round(uu[which(acr_ifm(uu) > art_trees(uu))[1]], 2) # where ACR becomes the harsher rule
3261 #> [1] 14.68
```

3262 Below about fifteen per cent uncertainty ACR is the most forgiving of the three and ART the least;
3263 above it the ordering reverses and ACR becomes much the harshest. A project choosing a registry
3264 on the strength of its uncertainty deduction needs to know which side of that line it sits on, and a
3265 verifier reading a comparison written by a project developer should check which side the
3266 developer assumed.

3267 *Rule sources*

3268 Check the version before relying on any of this. Registry documents move, and several of these
3269 were revised in the last two years.

	ACR	ART-TREES	Verra
Document	IFM on Non-Federal U.S. Forestlands v2.1, s.7.5	TREES Standard, s.8	VMD0055 v1.1, s.5.3.2.3 Step 4
Equations	22 (total uncertainty), 23 (deduction)	11 (uncertainty adjustment factor)	10 and 11 (discount factor)
Quantity	Magnitude-weighted quadratic mean of baseline and project uncertainty	Half-width of 90% CI as % of the mean	Half-width of two-sided 90% CI as % of the estimate
Threshold	10%	none	10%
Deduction	excess over 10%, one for one	$0.524417 \times CI_{90} / 1.645006$	$U\% \times t_{66.67\%} / (100 \times t_{10\%})$
t values	precision requirement is $\pm 10\%$ at 90%	0.524417 and 1.645006	0.4307 and 1.6449

	ACR			ART-TREES			Verra		
Cap	none	stated	in	the	none		uncertainty	must	stay
	methodology						below 100%;	beyond that,	resample

3270 ACR's Equation 22 is worth reading closely, because it is not simple quadrature. Each term is a
3271 carbon stock change multiplied by its own percentage uncertainty, the squares of those products
3272 are summed under a root, and the whole is divided by the sum of the unweighted magnitudes. A
3273 pool that barely moves contributes almost nothing however uncertain it is. Note also that the
3274 equation includes harvested wood products terms alongside the stock changes, which an
3275 implementation that only carries baseline and project stock change will omit.

3276 The ART constants are unchanged between Standard 2.0 (August 2021) and 3.0 (July 2025), so
3277 the arithmetic here is stable across that revision even though much else in TREES is not.

3278 Two things that catch people out. Verra's uncertainty rules are not in VM0048 itself, which is a
3279 short framework document; they are in module VMD0055. And VM0007 has not been superseded
3280 by VM0048, so two different uncertainty regimes remain live inside Verra at the same time.

3281 *Sampling versus counting*

3282 VM0048 sets a resolution rule that looks odd until you see the reasoning. The wall-to-wall forest
3283 cover benchmark map must be at 30 m or finer and *must not exceed* 30 m, with the minimum
3284 mapping unit set equal to the minimum area in the forest definition, while imagery inside sample
3285 plots must be 10 m or finer.

3286 The split exists because the map is not the estimator. VM0048 derives activity data from a
3287 probability sample; the map only stratifies that sample. This is the same argument the area-
3288 adjusted accuracy section above makes from the other direction, and it is the cleanest statement
3289 in any standard of why counting classified pixels is a biased way to estimate area.

3290 The accuracy requirements follow from that: over the historical reference period, deforestation
3291 needs user's and producer's accuracy at 70 per cent or better, and forest at the end of the period
3292 better than 90 per cent, with at least 100 sample observations per class and 200 per estimate.
3293 Open-canopy jurisdictions, where at least half the forest carries under 50 per cent canopy cover,
3294 may use 60 and 80 per cent instead.

3295 ACR's Improved Forest Management methodology, by contrast, sets no minimum mapping unit
3296 and requires no accuracy assessment at all, because its uncertainty is field-inventory sampling

error rather than map classification error. The two standards are not measuring the same thing, which is worth remembering before comparing their uncertainty figures.

11.11 Exercises

1. Recompute the sampling confidence interval using a 4-by-6 quadrat grid instead of 8-by-12. Does the mean change? Does the standard error? Explain the difference.
2. In the Monte Carlo, double the uncertainty on the carbon fraction (sd 0.04) and rerun. How much does the uncertainty percentage grow, and would it now trigger a 15% deduction?
3. Report the conservative (lower-bound) carbon density and express the forgone credit as a percentage of the mean. In one sentence, state the incentive this creates.
4. (*Applied.*) Change the error matrix so the map omits more forest (change the second row to 25, 75). Recompute the adjusted forest area and its interval, and explain which way omission error pushes the adjustment.

12. Time Series

12.1 Data cubes

Everything so far has been a snapshot. Monitoring is about change, and change lives in time, so the capstone data structure is the space-time cube: a stack of raster layers sharing one grid, indexed by date. A pixel is no longer a single value but a time series, and the questions become temporal, is this pixel trending down, when did it drop, how fast is it recovering.

The word cube is meant literally. The data have three axes and you must hold all three in mind at once. Two are spatial, x and y , the familiar grid of the earlier chapters. The third is time, t , and it is the one that changes how everything must be handled, because unlike x and y it is not sampled on a regular grid unless you force it to be. A satellite revisits on a fixed cycle but returns a usable observation only when the sky is clear, so the raw time axis is ragged. Operational cubes carry a fourth axis as well, the spectral band, which is why real archives are four-dimensional arrays and why the software built for them, `gdalcubes` and `sits` in R, is organised around regularising all four at once (Appel & Pebesma, 2019; Simões et al., 2021).

We build a monthly index cube over the plot spanning nine years. It carries three things a real optical record carries: a slow downward trend, an annual seasonal swing from leaf-on to leaf-off, and observation noise. A fire clears a block in mid-2021 and holds it low.

```
library(terra)
library(sf)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
                  coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
```

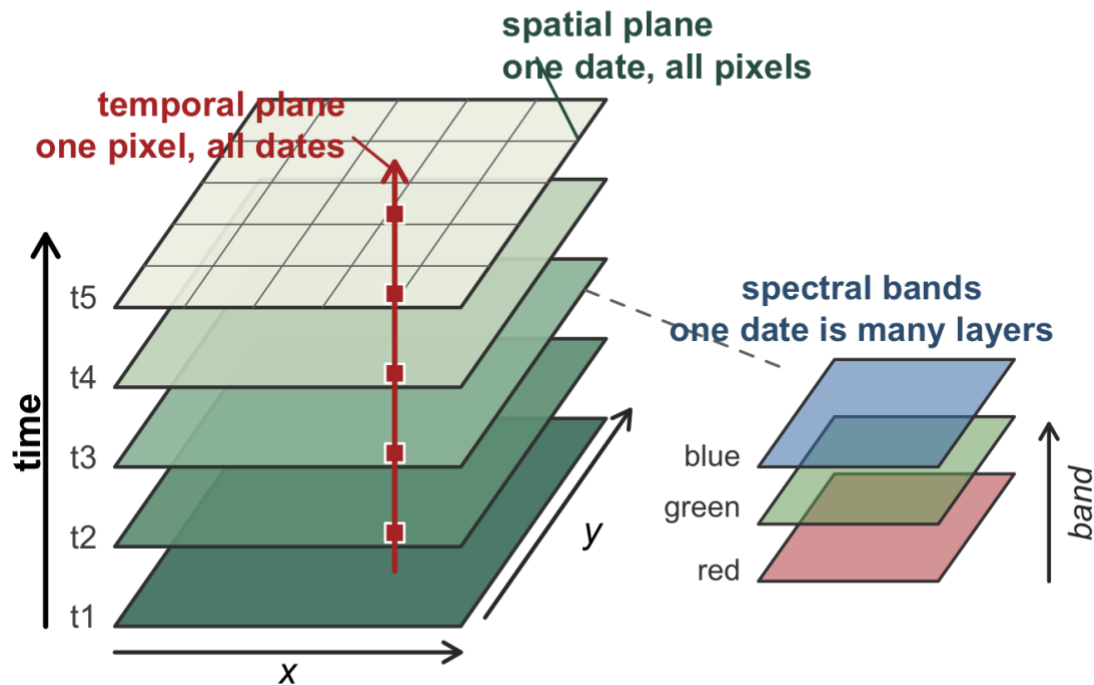
```

3332 r <- rast(ext(vect(stems)), resolution = 10, crs = "EPSG:32617")
3333
3334 dates <- seq(as.Date("2016-01-15"), as.Date("2024-12-15"), by = "month")
3335 doy <- as.numeric(format(dates, "%j"))
3336 yfrac <- as.numeric(format(dates, "%Y")) + (doy - 1) / 365
3337
3338 set.seed(3)
3339 xy <- xyFromCell(r, 1:ncell(r))
3340 burn <- xy[, 1] > 747700 & xy[, 1] < 747850 &
3341         xy[, 2] > 4309000 & xy[, 2] < 4309150
3342
3343 # a standing pattern of greenness, and a decline that is steeper in the north
3344 base <- 0.62 + 0.06 * sin((xy[, 1] - min(xy[, 1])) / 90) *
3345         cos((xy[, 2] - min(xy[, 2])) / 120)
3346 decline <- 0.004 + 0.010 * (xy[, 2] - min(xy[, 2])) / diff(range(xy[, 2]))
3347
3348 # simulated: trend + seasonal cycle + noise, with a 2021 fire in the burn box
3349 mcube <- rast(lapply(seq_along(dates), function(k) {
3350   seas <- 0.25 * sin(2 * pi * (doy[k] - 100) / 365)
3351   v <- base - decline * (yfrac[k] - 2016) + seas + rnorm(ncell(r), 0, 0.02)
3352   if (yfrac[k] >= 2021.5) {
3353     v[burn] <- 0.20 + 0.06 * sin(2 * pi * (doy[k] - 100) / 365) +
3354       rnorm(sum(burn), 0, 0.02)
3355   }
3356   setValues(r, v)
3357 }))
3358 names(mcube) <- format(dates, "%Y-%m")
3359 terra::time(mcube) <- dates
3360 c(layers = nlyr(mcube), cells = ncell(mcube))
3361 #> layers cells
3362 #> 108 2730

```

12.2 Two planes

A cube can only be looked at one slice at a time, and there are two ways to cut it. Fix a date and you get the spatial plane: a map, every pixel at one instant, the view every earlier chapter of this book has used. Fix a pixel and you get the temporal plane: a profile, one location across every date, the view that makes change visible. Neither is complete. A map cannot tell you whether a low value is a burn scar or simply February, and a profile cannot tell you whether a decline is one pixel or the whole stand.

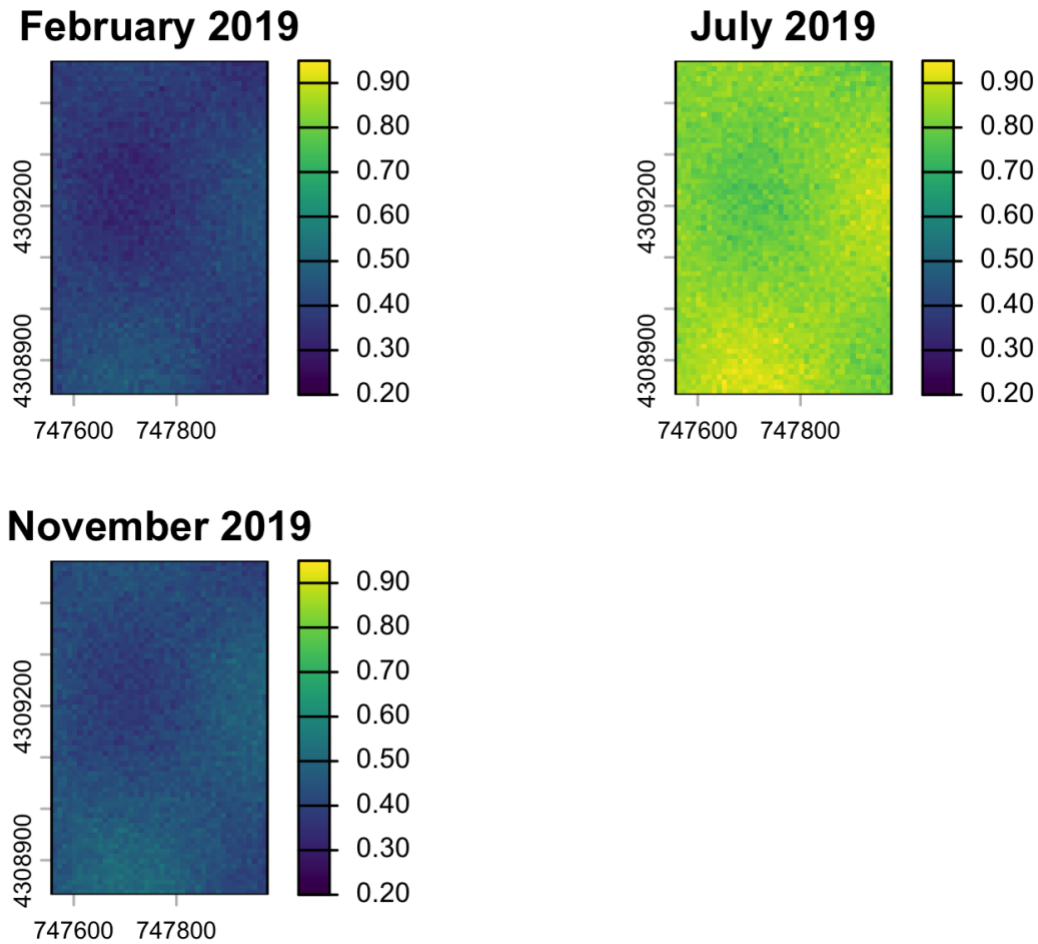


3370

3371 *Geometry of a space-time cube. Each horizontal sheet is one date. Cutting horizontally gives the spatial*
 3372 *plane, a map; cutting vertically through a single pixel gives the temporal plane, a profile. The detail at*
 3373 *right shows the fourth axis: every date is itself a stack of spectral bands.*

3374 Cut the cube both ways on the same data. Three dates from 2019, an undisturbed year, show the
 3375 spatial plane swinging with the season while the map itself barely changes:

```
3376 plot(mcube[[c("2019-02", "2019-07", "2019-11")]],
3377       range = c(0.2, 0.95),
3378       main = c("February 2019", "July 2019", "November 2019"))
```



3379

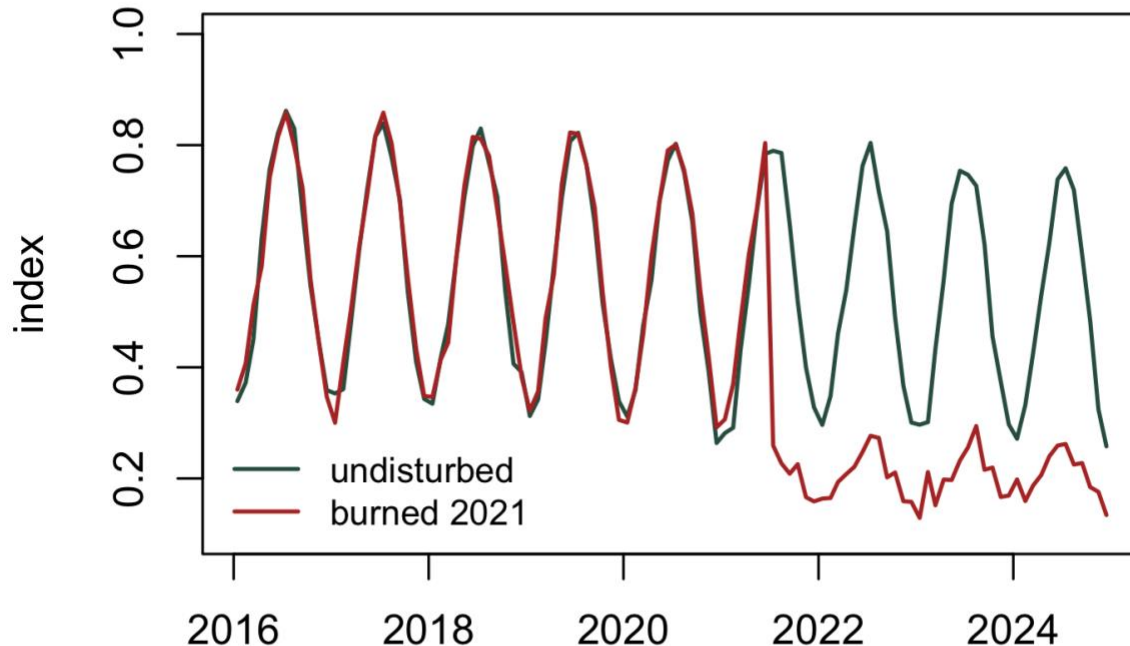
3380 The same three maps differ by more than half an index unit, and nothing on the ground has
 3381 changed. Now the temporal plane, two pixels drawn across all 108 dates, one inside the burn box
 3382 and one outside it:

```

3383 inside <- cellFromXY(r, cbind(747775, 4309075)) # in the 2021 burn
3384 outside <- cellFromXY(r, cbind(747600, 4309300)) # undisturbed
3385
3386 p_in <- as.numeric(mcube[inside])
3387 p_out <- as.numeric(mcube[outside])
3388
3389 plot(dates, p_out, type = "l", col = "#2f5d4e", lwd = 1.6,
3390      ylim = c(0.1, 1.0), xlab = "", ylab = "index",
3391      main = "Temporal plane: two pixels")
3392 lines(dates, p_in, col = "#b5342e", lwd = 1.6)
3393 legend("bottomleft", c("undisturbed", "burned 2021"),
3394      col = c("#2f5d4e", "#b5342e"), lwd = 1.6, bty = "n", cex = 0.85)

```

Temporal plane: two pixels



3395

3396 The saw-tooth is the seasonal cycle, present in both pixels throughout. The step down in the red
3397 line during 2021 is the fire. Note how much larger the annual swing is than the trend it rides on:
3398 the disturbance is obvious because it is abrupt, but the slow decline underneath is nearly invisible
3399 against the seasonality.

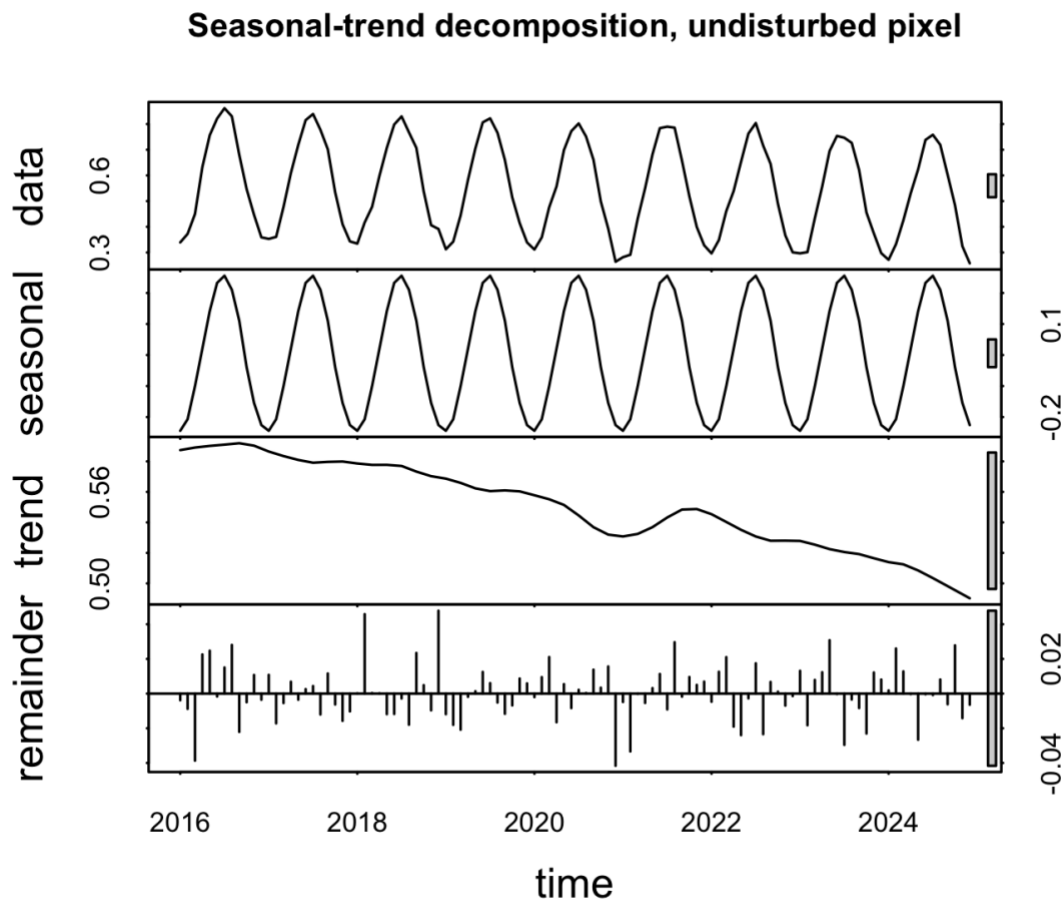
3400 **12.3 Seasonality and randomness**

3401 That last observation is the central problem of the temporal plane, and it is worth quantifying rather
3402 than asserting. A time series of this shape is conventionally treated as three additive parts: a
3403 trend, the slow directional movement; a seasonal component, the oscillation that repeats each
3404 year; and a remainder, what is left when both have been removed. Seasonal-trend decomposition
3405 by loess separates them without assuming the seasonal shape is a fixed sine (Cleveland et al.,
3406 1990). Base R implements it in `stl()`, which needs the profile declared as a regular series with
3407 twelve observations per cycle:

```

3408 s <- ts(p_out, start = c(2016, 1), frequency = 12)
3409 dec <- stl(s, s.window = "periodic")
3410 plot(dec, main = "Seasonal-trend decomposition, undisturbed pixel")

```



3411

3412 The three panels beneath the data are the decomposition. Partitioning the variance among them

3413 says which one governs what an analyst sees:

```

3414 v <- apply(dec$time.series, 2, var)
3415 round(100 * v / sum(v), 1) # per cent of variance
3416 #> seasonal trend remainder
3417 #> 96.9 2.3 0.7
3418 round(c(remainder_sd = sd(dec$time.series[, "remainder"])), 4)
3419 #> remainder_sd
3420 #> 0.0158

```

3421 Seasonality accounts for almost all of the variance in the record. The trend, the thing carbon

3422 monitoring exists to measure, is a percent or two. The remainder is smaller still, and its standard

3423 deviation recovers the observation noise that was put into the simulation, which is the check that

3424 the decomposition is doing what it claims.

This is the distinction the whole chapter turns on. The seasonal component is nature's oscillation: deterministic, repeating, and therefore removable, either by decomposition or simply by always comparing like with like. The remainder is nature's randomness together with sensor error: irreducible, and the honest floor on how precisely any single observation can be known. Confusing the two is expensive. Seasonality mistaken for randomness inflates the error bars you report and costs credits to the uncertainty deduction. Seasonality mistaken for trend invents change that never happened.

12.4 Sampling and aliasing

The practical consequence appears the moment you reduce a monthly record to an annual one, which every monitoring programme does. If each year is represented by an observation taken at the same point in the season, the seasonal component is a constant offset and cancels out of the comparison. If the dates drift, the seasonal component leaks into the difference between years and is indistinguishable from real change. Signal processing calls this aliasing. In forest monitoring it is the reason cloud cover biases change estimates, because the dates you can see are not a random sample of the dates you wanted (Hansen et al., 2013).

Estimate the trend both ways on the same pixel. First from a consistent July observation each year, then from one observation per year taken at whatever month happened to be clear:

```

yrs <- 2016:2024
julys <- which(as.numeric(format(dates, "%m")) == 7)
fixed_slope <- unname(coef(lm(p_out[julys] ~ yrs))[2])

set.seed(11)
drift_slope <- replicate(2000, {
  idx <- sapply(yrs, function(y) sample(which(format(dates, "%Y") == as.character(y)), 1))
  coef(lm(p_out[idx] ~ yrs))[2]
})

round(c(true = -0.008,
  fixed_july = fixed_slope,
  drift_mean = mean(drift_slope),
  drift_sd = sd(drift_slope)), 5)
#>      true fixed_july drift_mean  drift_sd
#> -0.00800 -0.01293 -0.00989  0.02339
round(quantile(drift_slope, c(0.025, 0.975)), 4)
#>    2.5%  97.5%
#> -0.0537  0.0364

```

Consistent sampling recovers the true rate of decline closely. Drifting sampling recovers it on average, because the seasonal error is centred rather than systematic, but the spread around that average is several times the size of the trend being measured. The interval spans zero comfortably in both directions:

```

round(100 * mean(drift_slope > 0), 1)      # per cent of draws with the wrong sign
#> [1] 34.7

```

More than a third of the time, an analyst sampling this way would report a pixel as greening when it is in fact declining. Nothing is wrong with the sensor, the algorithm, or the arithmetic. The error is entirely a consequence of sampling a periodic signal at inconsistent phase, and no amount of downstream statistics recovers what the sampling design threw away.

This is where the chapter connects back to the previous one. The uncertainty chapter propagated error through an equation, taking the input distributions as given. Here the input distribution is itself a product of a design decision. Compositing to a fixed seasonal window before any comparison is not data cleaning, it is uncertainty reduction, and it is the cheapest available: it costs nothing but discipline and it removes a variance component that dwarfs the quantity being estimated.

12.5 Annual compositing

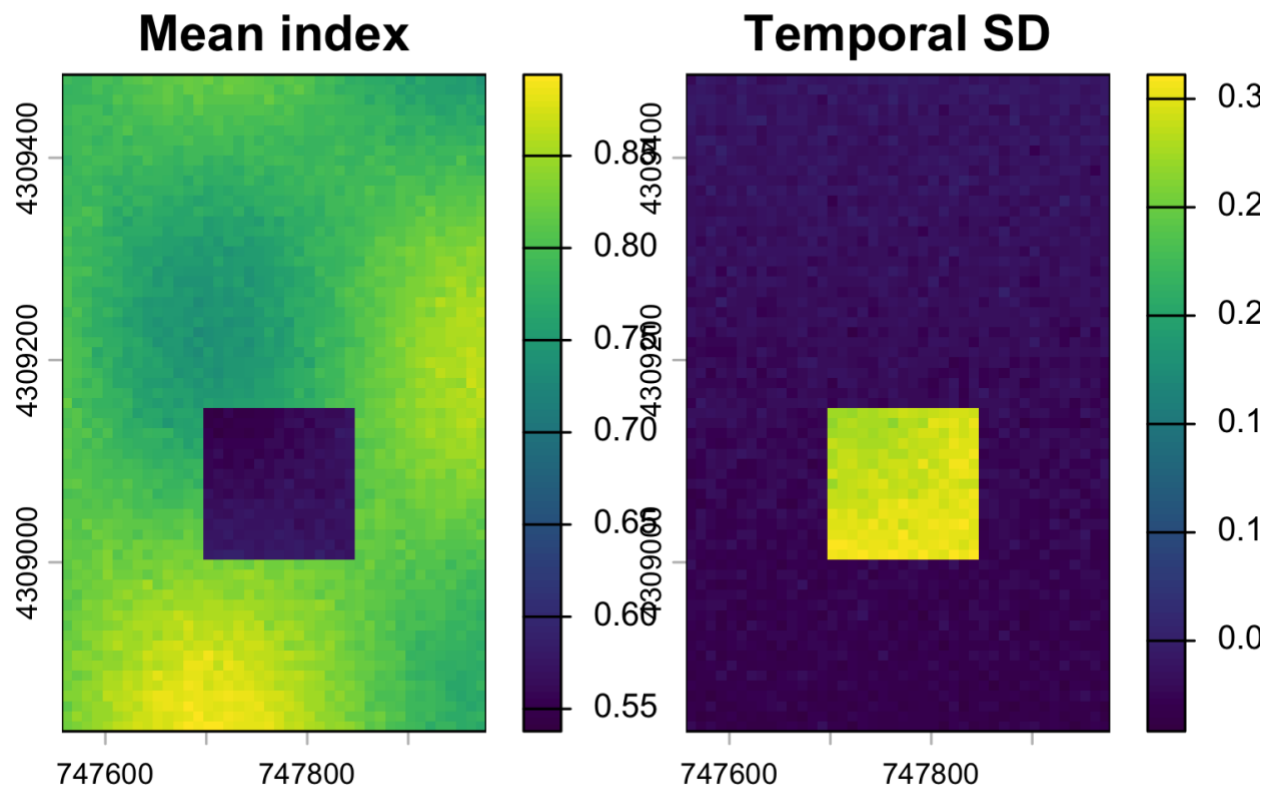
So we reduce the monthly cube to one layer per year, taking a consistent mid-summer window rather than an annual mean, which would average leaf-on against leaf-off and discard the contrast that makes disturbance visible. This is the temporal normalisation step that operational cube software performs with cloud-ranked compositing:

```
years <- 2016:2024
cube <- rast(lapply(years, function(y) {
  sel <- which(format(dates, "%Y") == as.character(y) &
    as.numeric(format(dates, "%m")) %in% 6:8)
  app(mcube[[sel]], median)
}))
names(cube) <- paste0("y", years)
nlyr(cube)
#> [1] 9
```

12.6 Temporal reduction

A cube is reduced along time the way a single raster is summarised along space. `app()` applies a function to each pixel's time series. The mean over years is the baseline surface; the standard deviation flags where the record is unstable:

```
mean_ndvi <- app(cube, mean)
sd_ndvi <- app(cube, sd)
plot(c(mean_ndvi, sd_ndvi), main = c("Mean index", "Temporal SD"))
```



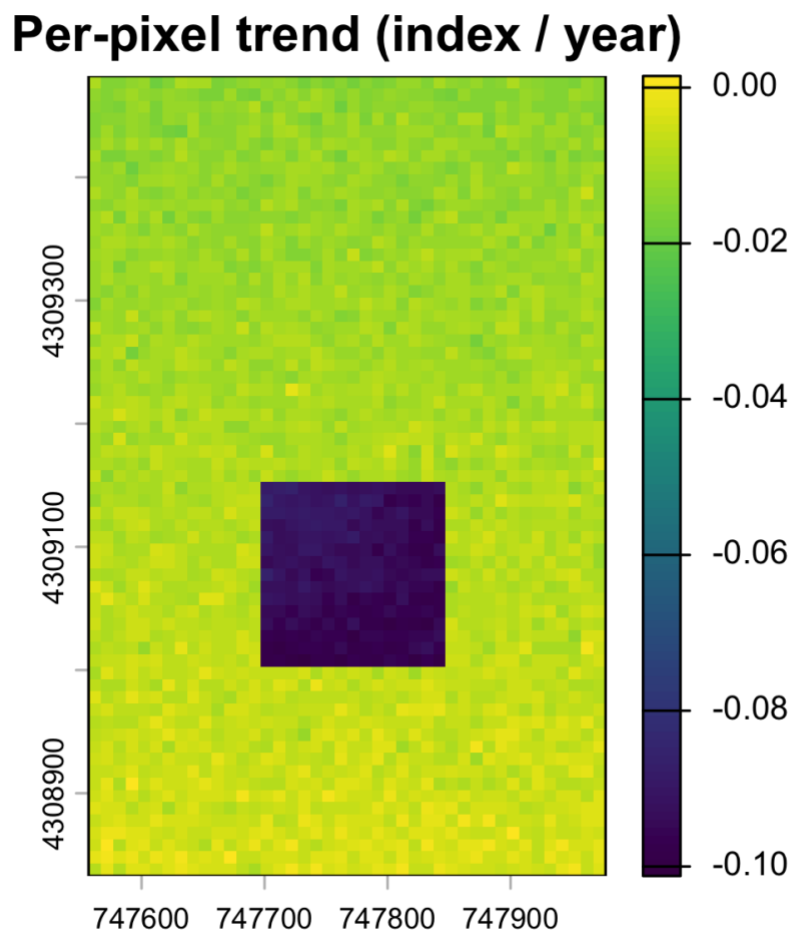
3498

3499 High temporal SD is the signature of disturbance: stable forest barely varies year to year once
 3500 the season is controlled, so the burned block lights up.

3501 12.7 Trends

3502 The direction and rate of change per pixel is the slope of index against year, a linear trend fitted
 3503 independently at every location. `app()` carries the regression across the cube:

```
3504 slope <- app(cube, function(v) {
3505   if (all(is.na(v))) return(NA)
3506   coef(lm(v ~ years))[2]
3507 })
3508 names(slope) <- "trend_per_year"
3509 plot(slope, main = "Per-pixel trend (index / year)")
```



3510

3511 Strong negative slopes mark sustained loss. This is the engine of trend-based monitoring: a whole
 3512 landscape of time series reduced to one map of who is gaining and who is losing.

3513 12.8 Event detection

3514 A trend says a pixel declined; monitoring needs to know *when*. Differencing consecutive years
 3515 isolates the step. The fire year is the layer where the largest drop occurs, and thresholding the
 3516 year-on-year change delineates the burned area:

```
3517 d2021 <- cube[["y2021"]] - cube[["y2020"]]
3518 burned <- d2021 < -0.3
3519 cell_ha <- prod(res(r)) / 1e4
3520 burned_ha <- global(burned, "sum", na.rm = TRUE)[1, 1] * cell_ha
3521 round(burned_ha, 2) # hectares burned in 2021
3522 #> [1] 2.25
```

3523 12.9 Tier 1 emissions

3524 The capstone joins the whole book: a time series detects a fire, the burned area is measured, and
 3525 IPCC Tier 1 methods turn that area into an emission. The IPCC-wildfire-emissions workflow runs

this chain from burn-area mapping through to emission factors, and the governing equation is the same one every fire inventory uses (IPCC, 2006):

$$L_{fire} = A \times M_B \times C_f \times G_{ef} \times 10^{-3}$$

where A is burned area, M_B the fuel biomass available, C_f the combustion factor, and G_{ef} the emission factor for the gas. With the burned area from the cube and IPCC default factors for temperate forest:

The plot is temperate forest, so every factor must be drawn from the temperate rows of the guidelines. Mixing a tropical fuel load with an extratropical emission factor is a common and silent error, and it inflates the answer by more than a factor of two:

```
A <- burned_ha      # ha, from the time-series detection
MB <- 50.4           # t dry matter per ha, other forest temperate (Table 2.4)
Cf <- 0.47           # combustion factor, same row
Gef <- 1569          # g CO2 per kg dry matter, extratropical (Table 2.5)

CO2_tonnes <- A * MB * Cf * Gef * 1e-3
round(c(burned_ha = A, CO2_tonnes = CO2_tonnes), 1)
#> burned_ha CO2_tonnes
#>      2.2      83.6
```

That single number, tonnes of CO2 from the fire, is the end of the pipeline this course has built toward: geospatial data, cleaned and projected in the early chapters, turned into a detected, measured, and quantified emission with every step reproducible. In a real inventory each factor carries its own uncertainty, so the honest final move is to wrap this equation in the Monte Carlo of the previous chapter and report the emission with a confidence interval. The methods are all now in hand.

```
set.seed(9)
sim <- replicate(2000, {
  mb <- rnorm(1, 50.4, 30.8)      # Table 2.4 states the spread alongside the mean
  cf <- rnorm(1, 0.47, 0.14)
  gef <- rnorm(1, 1569, 131)      # Table 2.5
  A * max(mb, 0) * min(max(cf, 0), 1) * max(gef, 0) * 1e-3
})
round(quantile(sim, c(0.025, 0.5, 0.975)), 0)      # CO2 tonnes: Lower, median, upper
#> 2.5% 50% 97.5%
#> 0 80 219
```

Note what that interval does and does not include. It propagates the uncertainty in the three emission factors while treating the burned area A as known exactly, which it is not: it came from a threshold applied to a differenced cube, and both the threshold and the dates feeding it are choices. The area-adjusted accuracy of the uncertainty chapter is how that term is estimated properly, and a complete Tier 2 statement would carry it into this same simulation alongside the factors.

12.10 Exercises

1. Reduce the cube with `app()` to the maximum index per pixel instead of the mean. Where does the maximum surface differ most from the mean, and why there?
2. Fit the per-pixel trend on the pre-fire years only (2016–2020). How does the burned block's slope compare with the full-record slope, and what does that say about mixing a step change into a linear trend?
3. Recompute burned area with a -0.2 threshold on the 2021 difference. How does the emission estimate move, and which term in the IPCC equation is it multiplying?
4. Run `st1()` on the burned pixel `p_in` rather than the undisturbed one. Which component absorbs the 2021 step, and what does that tell you about applying seasonal decomposition across a disturbance?
5. Rebuild the annual cube using an annual mean over all twelve months instead of a summer median. How do the temporal SD map and the detected burned area change, and why does averaging across the season blur the event?
6. Repeat the drifting-date experiment restricting the sampled months to June through September, as a cloud-screened archive in a seasonal climate might allow. How much of the variance inflation survives, and what does that imply about how tight a compositing window needs to be?
7. (*Applied.*) Using the Monte Carlo output, report the median CO₂ and its 95% interval, then state which of the three factors (fuel load, combustion factor, emission factor) you would most want to measure locally to narrow the interval.

13. Spatial Patterns

13.1 Point patterns

Every earlier chapter analysed values measured at locations: carbon at a plot, elevation at a pixel, an index through time. Point-pattern analysis asks a different question, about the locations themselves. Are the events, tree stems, seedlings, detected clearings, spread evenly, scattered at random, or clustered together? The answer characterises the process that generated them, and it is the second capstone of this course: a whole branch of spatial statistics with its own tools. We use `spatstat`, whose `ppp` object holds a point pattern inside an observation window.

The running example here is a real regeneration survey: conifer seedlings mapped after the 2015 Mt Midgeley wildfire in the Darkwoods Conservation Area of the southern Selkirk Mountains, British Columbia (Murphy et al., 2026). We load the seedlings and build a `ppp` from one four-hectare plot, its bounding box serving as the window:

```
library(spatstat)
seed <- read.csv("data/darkwoods_seedlings.csv")
p3 <- subset(seed, plot == 3)
win <- owin(range(p3$x), range(p3$y))
pp <- ppp(p3$x, p3$y, window = win)
```

```
3603 #> Planar point pattern: 1607 points
3604 #> window: rectangle = [524164.7, 524362.6] x [5446884, 5447083] units
```

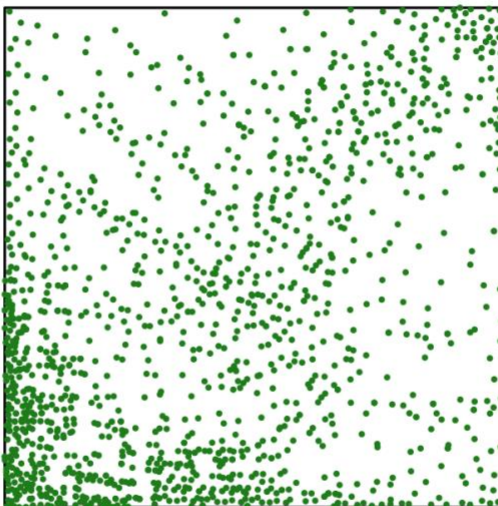
3605 The printout reports the number of points and the window: a little over 1,600 seedlings in a 200
3606 by 200 metre plot. That is the pattern whose structure we now interrogate.

3607 13.2 Spatial randomness

3608 The reference against which every pattern is judged is complete spatial randomness (CSR):
3609 points falling independently and with equal probability everywhere, a homogeneous Poisson
3610 process. Real patterns depart from CSR in one of two directions, toward regularity, where points
3611 avoid each other as in a planted stand, or toward clustering, where points attract as in natural
3612 regeneration around surviving seed trees. The whole toolkit exists to measure that departure and
3613 test whether it is real. A plot of the seedlings already suggests the answer:

```
3614 plot(pp, main = "Mapped conifer seedlings, plot 3", pch = 20, cex = 0.3, cols = "forestgreen")
```

Mapped conifer seedlings, plot 3



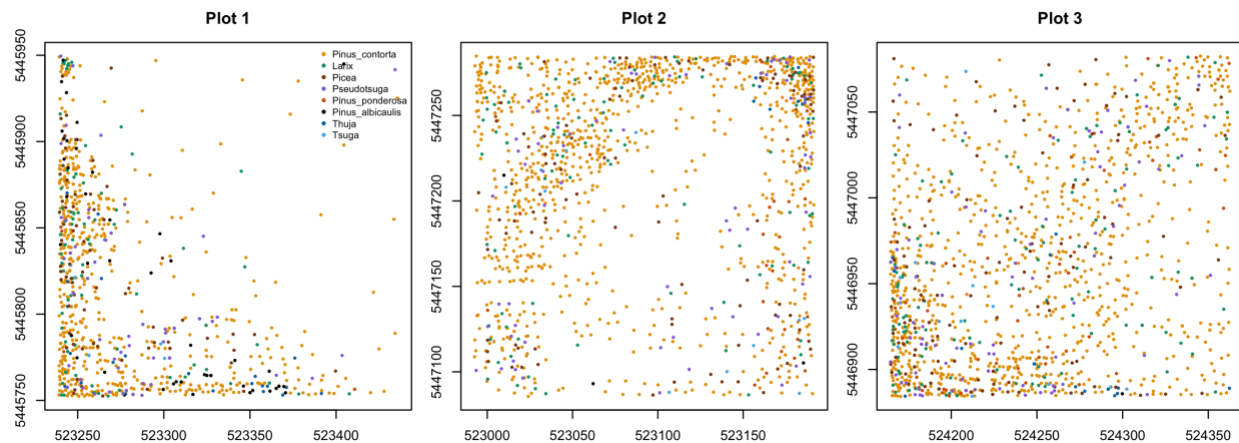
3615

13.3 Species maps

Before any statistic, plot the pattern by species across all three plots. A composition map is the cheapest check on a survey and the fastest way to see that the plots are not replicates of one another:

```
spp_levels <- names(sort(table(seed$species), decreasing = TRUE))
spp_cols <- setNames(
  c("#E69F00", "#009E73", "#8B4513", "#9370DB",
    "#D55E00", "#000000", "#0072B2", "#56B4E9")[seq_along(spp_levels)],
  spp_levels)

par(mfrow = c(1, 3), mar = c(2, 2, 3, 1))
for (p in sort(unique(seed$plot))) {
  s <- subset(seed, plot == p)
  plot(s$x, s$y, asp = 1, pch = 20, cex = 0.5, col = spp_cols[s$species],
       xlab = "", ylab = "", main = paste("Plot", p))
  if (p == 1) legend("topright", legend = spp_levels, col = spp_cols,
                    pch = 20, bty = "n", cex = 0.7)
}
```



Mapped seedlings by species across the three surveyed plots.

```
par(mfrow = c(1, 1))
```

The colours are keyed by name, not by position. That is worth stating because the positional alternative, building a palette and a legend independently and trusting them to line up, is a standing source of published figures whose legend contradicts the plot. Sorting the levels by abundance and naming the palette makes the two impossible to desynchronise.

The three panels are not interchangeable. Lodgepole pine dominates everywhere, but the minor species are distributed differently between plots, and the plots themselves sit at different elevations kilometres apart. Any analysis that pooled them without carrying plot identity would be averaging over three different regeneration environments.

```
round(prop.table(table(seed$plot, seed$species), 1), 3)
#>
#> Larix Picea Pinus_albicaulis Pinus_contorta Pinus_ponderosa Pseudotsuga
```

```

3648 #> 1 0.081 0.069          0.045          0.721          0.004          0.050
3649 #> 2 0.079 0.076          0.002          0.750          0.022          0.054
3650 #> 3 0.084 0.075          0.001          0.733          0.033          0.055
3651 #>
3652 #> Thuja Tsuga
3653 #> 1 0.021 0.008
3654 #> 2 0.006 0.009
3655 #> 3 0.010 0.009

```

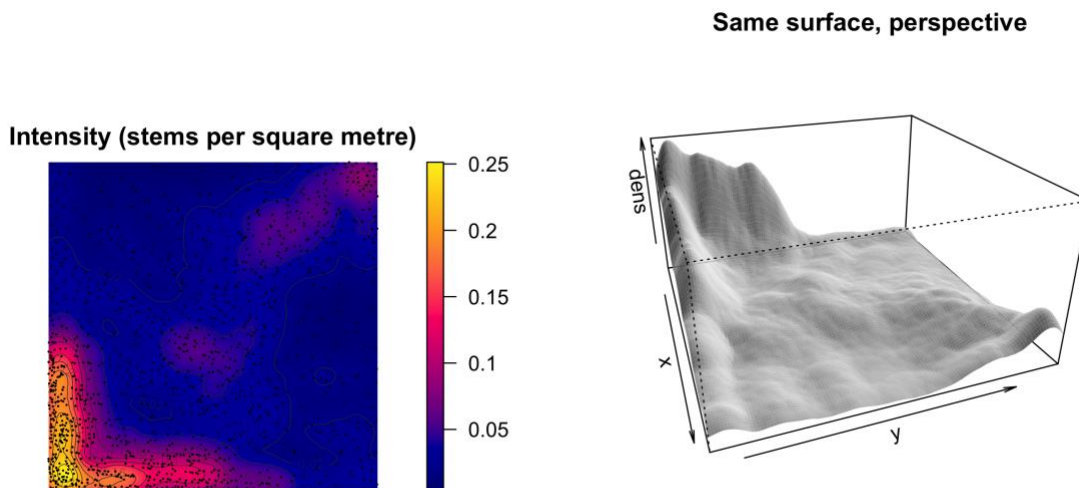
3656 13.4 Intensity

3657 The intensity is the first-order property: the expected number of points per unit area, and how it
3658 varies across the window. A kernel density estimate smooths the points into a continuous intensity
3659 surface, the point-pattern analogue of a heat map, revealing where regeneration concentrated:

```

3660 dens <- density(pp, sigma = 8, kernel = "epanechnikov", diggle = TRUE)
3661
3662 par(mfrow = c(1, 2), mar = c(2, 2, 3, 4))
3663 plot(dens, main = "Intensity (stems per square metre)", las = 1)
3664 contour(dens, add = TRUE, col = "grey20", lwd = 0.5)
3665 points(pp, pch = 20, cex = 0.15)
3666 persp(dens, theta = 70, phi = 25, shade = 0.4, border = NA,
3667       main = "Same surface, perspective")

```



3668

3669 *Kernel intensity of seedlings on plot 3, with contours, and the same surface in perspective.*

3670 `par(mfrow = c(1, 1))`

3671 Two arguments in that call are doing real work. The Epanechnikov kernel is the minimum-variance
3672 choice and matters little in practice, but Diggle's edge correction matters a great deal: without it
3673 the estimate falls away artificially near the window boundary, because points outside the plot that
3674 would have contributed were never mapped. On a 200 metre plot smoothed at 8 metres the
3675 affected band is a few per cent of the area, and it is precisely the band where a naive reading
3676 would conclude that regeneration thins toward the plot edge.

The contours and the perspective view show the same surface twice on purpose. Contours support measurement, letting a reader take a value off the map. Perspective supports recognition, making the number and separation of the peaks immediately legible. Neither alone is sufficient for a figure whose point is that the surface has structure.

Regeneration is patchy, not uniform: dense pockets sit near surviving seed sources and gentler, wetter ground, while other areas stay bare. That spatial unevenness is the signal, and explaining it is what the disturbance and terrain covariates in the study were for.

13.5 Nearest neighbour

Whether points cluster or repel is the second-order property, about the interaction between them. The most direct diagnostic is the nearest-neighbour distance: how far, on average, each seedling sits from its closest neighbour, compared with what CSR predicts. Under CSR the expected mean nearest-neighbour distance is $1/(2\sqrt{\lambda})$, so the ratio of observed to expected is the Clark-Evans index: below one means clustered, above one means regular.

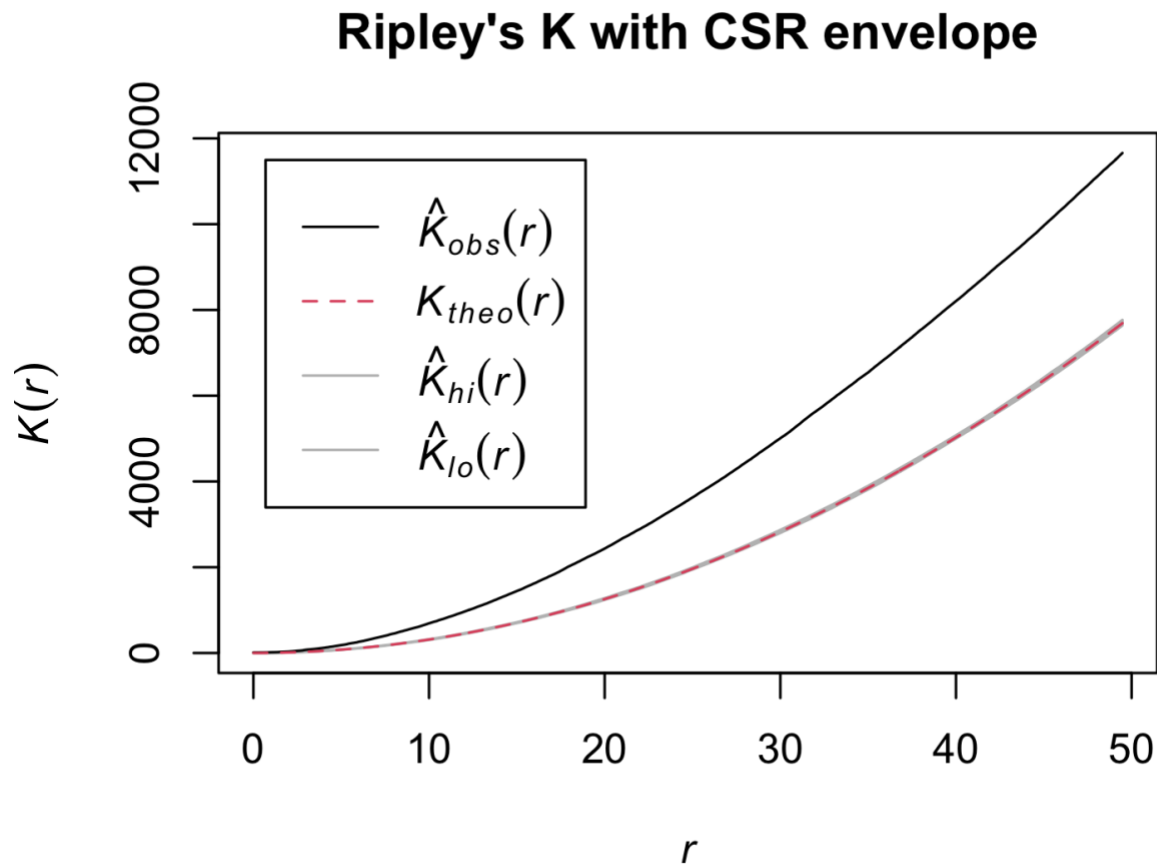
```
ce_ratio <- function(df) {  
  q <- ppp(df$x, df$y, window = owin(range(df$x), range(df$y)))  
  obs <- mean(nndist(q))  
  expected <- 1 / (2 * sqrt(intensity(q)))  
  obs / expected  
}  
round(ce_ratio(p3), 3)          # < 1 indicates clustering  
#> [1] 0.821
```

The ratio sits well below one: seedlings are closer to one another than randomness would allow, the quantitative signature of clustered regeneration.

13.6 Ripley's K

A single number cannot capture how clustering changes with scale, points may cluster tightly at short range yet disperse at long range. Ripley's K function describes the pattern across all distances at once, counting on average how many other points lie within a growing radius of each point, against the CSR expectation. Plotting the observed K with a simulation envelope is the standard test:

```
K <- envelope(pp, Kest, nsim = 19, verbose = FALSE)  
plot(K, main = "Ripley's K with CSR envelope")
```



3708

3709 Where the observed curve rises above the envelope the pattern is more clustered than random
 3710 at that distance; below it, more regular. The seedlings sit above the envelope across the mapped
 3711 range, confirming clustering is real and not an artefact of a few tight groups, and the scale-explicit
 3712 view is why K, and its variance-stabilised form the L function, is the workhorse of the field.

3713 **13.7 Seedling pattern**

3714 The published study runs exactly this arc, then goes further (Murphy et al., 2026). Across 3,886
 3715 mapped seedlings on three four-hectare plots, point-pattern analysis found significant clustering
 3716 for all seven conifer species, with intensity governed by fire severity, distance to seed source,
 3717 topography, and wind. The lean diagnostics above reproduce its core finding on one plot:
 3718 clustered regeneration, quantified by the Clark-Evans ratio and confirmed by Ripley's K.
 3719 Repeating the Clark-Evans ratio species by species shows the clustering is not confined to the
 3720 dominant lodgepole pine but holds across the community:

```

3721 spp <- c("Pinus_contorta", "Larix", "Picea", "Pseudotsuga")
3722 sapply(spp, function(s) {
3723   q <- subset(p3, species == s)
3724   if (nrow(q) < 20) return(NA)
3725   round(ce_ratio(q), 3)
3726 })
3727 #> Pinus_contorta      Larix      Picea      Pseudotsuga
3728 #>      0.859      0.886      0.985      0.969

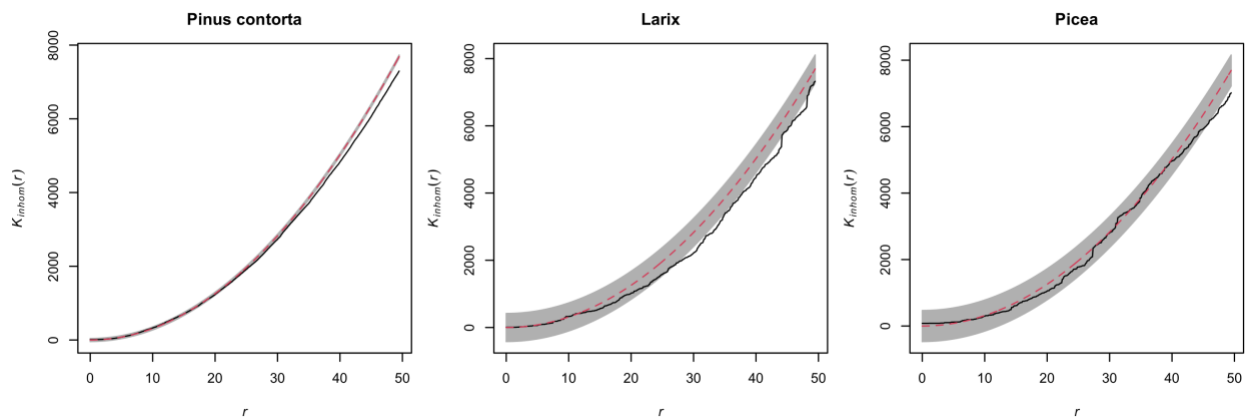
```

Every species returns a ratio below one. A ratio is one number, though, and the scale-explicit version is the per-species envelope. Simultaneous, or global, envelopes are used rather than pointwise ones, because a pointwise band is exceeded somewhere by chance in most realisations and invites reading significance off any excursion:

```

3733 set.seed(2026)
3734 par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))
3735 for (s in c("Pinus_contorta", "Larix", "Picea")) {
3736   q <- subset(p3, species == s)
3737   qp <- ppp(q$x, q$y, window = win)
3738   e <- envelope(qp, Kinhom, nsim = 39, correction = "best",
3739                 global = TRUE, verbose = FALSE)
3740   plot(e, main = sub("_", " ", s), legend = FALSE)
3741 }

```



Inhomogeneous K with simultaneous envelopes, by species, on plot 3.

```

3744 par(mfrow = c(1, 1))

```

The inhomogeneous form divides out a varying intensity, so what remains above the envelope is clustering over and above the trend. Reading these needs one caution, and it is the reason the residual testing section later in this chapter exists: the intensity is estimated from the same pattern being tested, so the estimate absorbs part of the clustering and the test is conservative. An excursion above the envelope here is therefore trustworthy; the absence of one is not evidence of no clustering.

That settles the description. It does not explain anything, and description is where most spatial analysis stops. The rest of this chapter is the step past it.

13.8 Pattern to process

Clark-Evans and Ripley's K both compare the observed pattern against complete spatial randomness. Rejecting CSR is easy and, on its own, close to uninformative: almost no ecological pattern is homogeneous Poisson, so a significant test tells you mainly that you had enough points to detect the obvious. The useful question is different. Given that intensity varies, what does it vary *with*, and is there clustering left over once you account for that?

Answering it requires a model. Two, in fact, layered.

The first-order model says the expected intensity at a location is a function of covariates measured there. Written on the log scale so intensity stays positive, it is a Poisson point process model, ppm , which behaves in almost every respect like a generalised linear model whose response is the pattern itself:

$$\log \lambda(u) = \beta_0 + \beta_1 z_1(u) + \dots + \beta_k z_k(u)$$

The second-order model says the points are not independent even after the covariates are accounted for. Seedlings arrive in groups because a surviving cone-bearing tree drops a cohort around itself. A Thomas cluster process makes that mechanism explicit: parent locations follow a Poisson process of intensity κ , and each parent scatters offspring around itself with a Gaussian kernel of standard deviation σ . Fitting both layers together is what kppm does, and the resulting model is a Cox process: Poisson conditional on a random intensity field (Baddeley et al., 2015).

Both layers matter, but for different reasons. The first tells you what drives regeneration. The second tells you how much to believe the first.

13.9 Covariates

The mapped seedlings carry the covariates the study extracted at each stem: elevation, slope, topographic wetness, distance to the nearest surviving seed source, and the two disturbance histories. Beetle attack is recorded in three stages, and fire in four severity classes:

```
seed$mpb <- factor(seed$mpb_class, 0:2, c("free", "grey", "red"))
seed$fire <- factor(seed$fire_class, 1:4, c("free", "low", "med", "high"))
round(prop.table(table(seed$mpb)), 3)
#>
#> free grey red
#> 0.715 0.038 0.247
round(prop.table(table(seed$fire)), 3)
#>
#> free low med high
#> 0.055 0.378 0.436 0.131
```

Grey-stage attack, where needles have fallen and crowns collapsed, covers under four per cent of the seedling records. Red-stage, the recent attack that still holds dead red needles, covers a quarter. Those proportions match the published study's plot summaries, which is the first check worth running on any dataset you did not extract yourself.

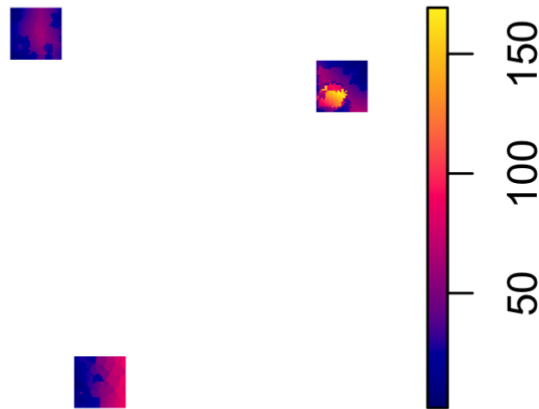
The three plots are separate 200 metre squares kilometres apart, so the observation window is their union, not one bounding box that would count the empty ground between them as surveyed. We buffer each plot by half a metre so no mapped seedling falls exactly on a boundary and gets silently dropped:

```
mkwin <- function(s) owin(range(s$x) + c(-0.5, 0.5), range(s$y) + c(-0.5, 0.5))
plots <- split(seed, seed$plot)
pooled <- do.call(superimpose,
  unname(lapply(plots, function(s) ppp(s$x, s$y, window = mkwin(s)))))
marks(pooled) <- NULL
c(points = npoints(pooled), hectares = round(area.owin(Window(pooled)) / 1e4, 2))
#>   points hectares
#> 3912.00    11.87
```

A ppm needs covariates as continuous surfaces, not as values attached to points, because the model must evaluate intensity everywhere in the window and not only where a seedling happens to be. The study built those surfaces from 30 metre rasters. Those rasters are not distributed with the seedling data, so we reconstruct each surface by assigning every location the value recorded at its nearest mapped seedling:

```
sd_ <- do.call(rbind, plots) # rows in the same order as the pooled pattern
as_im <- function(v) nnmark(ppp(sd_$x, sd_$y, window = Window(pooled), marks = v),
  dimyx = 256)
covs <- list(
  elevation = as_im(sd_$elevation),
  slope = as_im(sd_$slope),
  twi = as_im(sd_$twi),
  distance = as_im(sd_$distance_m),
  mpb_grey = as_im(as.numeric(sd_$mpb == "grey")),
  mpb_red = as_im(as.numeric(sd_$mpb == "red")),
  fire_high = as_im(as.numeric(sd_$fire == "high")),
  fire_low = as_im(as.numeric(sd_$fire == "low")),
  fire_med = as_im(as.numeric(sd_$fire == "med")))
plot(covs$distance, main = "Distance to nearest seed source (m)")
```

Distance to nearest seed source (m)



3824

3825 Be clear about what that reconstruction costs, because it is a real limitation and not a technicality.
3826 Nearest-neighbour interpolation from the sample points recovers a covariate faithfully where
3827 seedlings are dense and guesses where they are sparse. For a common condition the guess is
3828 harmless. For a rare one it is not: a rare class gets painted across the whole neighbourhood of
3829 the few points that carry it, which inflates its apparent extent. Grey-stage attack is exactly that
3830 rare class here, and we will see the consequence shortly. Working from the original rasters is
3831 always better. When you cannot, say so and show where it bites.

3832 13.10 Screening covariates

3833 With nine candidate covariates you need a selection step, and the natural one compares each
3834 single-covariate model against an intercept-only model by likelihood ratio test. The deviance
3835 measures how much explanatory power the covariate adds:

```
3836 null <- ppm(pooled ~ 1)
3837 screen <- t(sapply(names(covs), function(nm) {
```

```

3838 m <- ppm(pooled, trend = as.formula(paste("~", nm)), covariates = covs[nm])
3839 a <- anova(null, m, test = "LRT")
3840 c(beta = unname(coef(m)[2]), deviance = a$Deviance[2], p = a$`Pr(>Chi)`[2])
3841 )))
3842 round(screen[order(-screen[, "deviance"]), ], 4)
3843 #>      beta deviance      p
3844 #> distance -0.0124 362.6413 0.0000
3845 #> fire_low  0.6230 334.7588 0.0000
3846 #> fire_high -0.7361 284.1594 0.0000
3847 #> elevation -0.0005 52.7399 0.0000
3848 #> mpb_grey  0.6904 52.0740 0.0000
3849 #> twi       -0.0649 32.1664 0.0000
3850 #> slope     -0.0072 10.0271 0.0015
3851 #> mpb_red   -0.0809  4.7817 0.0288
3852 #> fire_med  -0.0121  0.1401 0.7081

```

3853 Read the ordering, not the p-values. With nearly four thousand points almost anything clears a
 3854 significance threshold, so the p-value column mostly reports sample size. The deviance column
 3855 is the one that separates the covariates that matter from those that merely pass.

3856 Distance to seed source dominates, which is the expected ecology: seed rain falls off sharply with
 3857 distance from a surviving parent. Fire severity comes next, and its two coefficients point in
 3858 opposite directions. Low-severity burn carries a positive coefficient and high-severity burn a
 3859 strongly negative one, so light fire helps regeneration and severe fire suppresses it. Medium
 3860 severity is the one covariate that fails outright, with a deviance near zero, and it is dropped. Terrain
 3861 enters weakly. Elevation and wetness are significant but explain several times less than distance
 3862 or fire, which is the ordering you would expect on ground where the disturbance history is recent
 3863 and severe.

3864 13.11 Bandwidth selection

3865 Every kernel estimate in this chapter has depended on a smoothing bandwidth chosen by hand.
 3866 Choosing it by likelihood cross-validation instead removes the arbitrariness. `bw.ppl` maximises a
 3867 leave-one-out likelihood over candidate bandwidths, and running it per plot shows whether the
 3868 plots agree:

```

3869 bw_obj <- lapply(plots, function(s)
3870   bw.ppl(ppp(s$x, s$y, window = mkwin(s))))
3871 bw <- sapply(bw_obj, as.numeric)
3872 round(bw, 2)
3873 #>      1      2      3
3874 #> 6.46 6.26 6.48

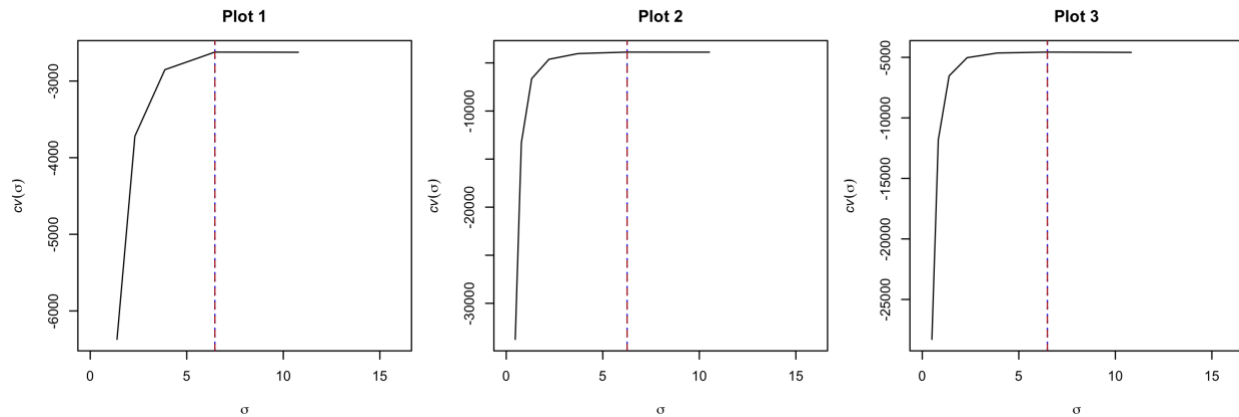
```

3875 Plotting the cross-validation criterion rather than only its optimum shows how sharply the data
 3876 identify that scale:

```

3877 par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))
3878 for (p in names(bw_obj)) {
3879   plot(bw_obj[[p]], main = paste("Plot", p), xlim = c(0, 16))
3880   abline(v = bw[[p]], col = "firebrick", lty = 2)
3881 }

```



Likelihood cross-validation criterion against bandwidth for each plot. The optimum is the maximum of each curve.

```
par(mfrow = c(1, 1))
```

All three land close together, near six and a half metres, comfortably inside the range the published study reports across plots and globally. That consistency is worth more than the number itself. Three independently surveyed plots agreeing on a scale is evidence the scale is a property of the regeneration process rather than of any one plot's layout.

The curves add a caution the single number hides. Each is flat near its maximum, so a range of bandwidths is very nearly as well supported as the optimum, and reporting the optimum to two decimals implies a precision the criterion does not have. Where a curve is flat, state the range.

13.12 Cluster process

Now fit both layers at once: the covariate trend from the screening step, and a Thomas cluster process for what the covariates do not explain.

```
fit <- kppm(pooled,
  trend = ~ distance + twi + mpb_grey + mpb_red + fire_high + fire_low,
  clusters = "Thomas", covariates = covs)
round(coef(fit), 4)
#> (Intercept) distance twi mpb_grey mpb_red fire_high
#> -3.0859 -0.0064 -0.0191 0.5272 -0.0996 -0.4203
#> fire_low
#> 0.3452

signif(fit$clustpar, 4)
#> kappa scale
#> 1.444e-04 2.091e+01
```

The trend coefficients read as multipliers on intensity once exponentiated. Distance is negative, so intensity falls with every metre from a seed source. High-severity fire is strongly negative and low-severity fire positive, reproducing the study's central finding about severity. Red-stage beetle attack is negative, matching the published sign.

The cluster parameters describe the second layer. The fitted scale is roughly twenty metres, so offspring sit within a few tens of metres of their parent. The parent intensity κ is about 0.00014 per square metre, which is a little under one and a half parents per hectare. Together they describe a modest number of well-separated cohorts rather than a fine-grained scatter, which is what a post-fire regeneration surface driven by scattered surviving seed trees should look like.

13.13 Corrected standard errors

Here is the part that changes what you would conclude, and it is the reason this section exists.

A `ppm` assumes points are independent given the covariates. They are not. Seedlings arrive in sibling groups, so the pattern carries far less information than its point count suggests. Fitting an independence model to clustered data leaves the coefficients roughly unbiased but makes the standard errors far too small. The cluster model estimates its variance by a composite likelihood that accounts for the dependence (Waagepetersen, 2007), and comparing the two makes the size of the error visible:

```
est <- coef(fit)
naive <- sqrt(diag(vcov(as.ppm(fit)))) # Poisson, assumes independence
clust <- sqrt(diag(vcov(fit))) # allows for the cluster process

round(cbind(estimate = est,
            se_naive = naive,
            se_cluster = clust,
            inflation = clust / naive,
            z_naive = est / naive,
            z_cluster = est / clust), 3)

#>      estimate se_naive se_cluster inflation z_naive z_cluster
#> (Intercept) -3.086    0.068    0.541    7.922 -45.218   -5.708
#> distance    -0.006    0.001    0.008    9.890  -8.345   -0.844
#> twi         -0.019    0.012    0.076    6.567  -1.654   -0.252
#> mpb_grey     0.527    0.087    0.658    7.532   6.032    0.801
#> mpb_red     -0.100    0.038    0.357    9.390  -2.622   -0.279
#> fire_high   -0.420    0.054    0.394    7.335  -7.831   -1.068
#> fire_low     0.345    0.037    0.262    7.104   9.346    1.316

c(significant_naive = sum(abs(est / naive) > 1.96),
  significant_cluster = sum(abs(est / clust) > 1.96),
  terms = length(est))
#>      significant_naive significant_cluster      terms
#>           6           1           7
```

The correct standard errors are between six and a half and ten times the naive ones. Under the independence assumption six of the seven terms clear the conventional threshold. Under the cluster-corrected variance, one does. Every substantive conclusion you would have drawn from the naive fit evaporates.

Nothing about the data changed. The estimates are identical; only the honesty of the uncertainty changed. Because variance scales as the square of the standard error, an inflation factor of seven to ten means the naive fit treated the data as carrying fifty to a hundred times more information

than it holds. Put the other way, the effective sample size is not 3,912 but a few dozen well-separated cohorts. This is the same problem that inflates confidence in any spatially autocorrelated dataset, and it is why the uncertainty chapter insists that the sampling design and the variance estimator must match.

Treat this as the general lesson. When observations cluster and your model assumes they do not, you will find significance that is not there. The failure is silent, the output looks authoritative, and the only defence is to fit a model that represents the dependence and compare.

13.14 Residual testing

The last question is whether clustering survives the covariates. If distance to seed source and fire severity explained the pattern completely, the residual process would be Poisson and there would be nothing for the Thomas layer to do.

Testing this needs more care than it first appears, and the trap is worth seeing. The inhomogeneous K function, `Kinhom`, generalises Ripley's K by dividing out a varying intensity. If you estimate that intensity nonparametrically from the same data, the estimate absorbs the very clustering you are trying to detect, and the test loses its power or reverses sign entirely. The null must also match: simulating homogeneous CSR while computing an inhomogeneous statistic compares two different things.

The coherent test fixes both. Fit an inhomogeneous Poisson model, hold its fitted intensity fixed, and simulate from that same fitted model:

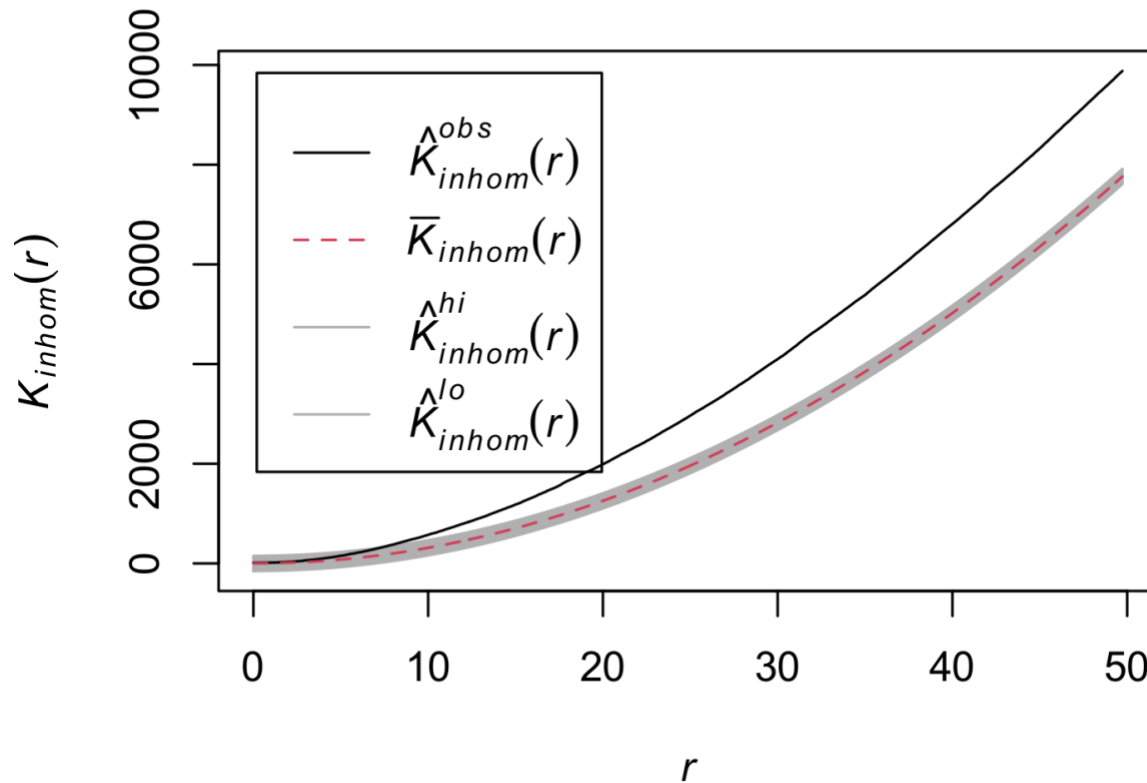
```
s3 <- plots[["3"]]
win3 <- mkwin(s3)
pp3 <- ppp(s3$x, s3$y, window = win3)
c3 <- function(v) nnmark(ppp(s3$x, s3$y, window = win3, marks = v), dimyx = 256)
cv3 <- list(twi = c3(s3$twi),
           distance = c3(s3$distance_m),
           fire_high = c3(as.numeric(s3$fire == "high")),
           fire_low = c3(as.numeric(s3$fire == "low")))

inhom <- ppm(pp3, trend = ~ twi + distance + fire_high + fire_low, covariates = cv3)
lam <- predict(inhom)

set.seed(2026)
env <- envelope(inhom, Kinhom, nsim = 999, global = TRUE, verbose = FALSE,
               simulate = expression(rpoispp(lam)), lambda = lam, normpower = 2)

plot(env, main = "Inhomogeneous K against the fitted trend, 999 simulations")
```

inhomogeneous K against the fitted trend, 999 simula



3989

```
3990 above <- env$r[env$obs > env$hi]
3991 round(c(radii_above = length(above), of = length(env$r),
3992       from_m = min(above), to_m = max(above)), 2)
3993 #> radii_above      of      from_m      to_m
3994 #>      437.00      513.00       7.38      49.73
```

3995 The observed curve leaves the envelope from about seven metres outward and stays above it to
 3996 the edge of the tested range. Clustering is not an artefact of the covariates. Seedlings aggregate
 3997 at scales of tens of metres beyond anything distance to seed source, wetness or fire severity
 3998 explains, which is precisely the residual structure the Thomas layer was fitted to absorb.

3999 Two details in that call are the difference between a real test and a decorative one. `global = TRUE`
 4000 gives a simultaneous envelope, valid to interpret across the whole range of distances at once; the
 4001 default pointwise envelope is only valid at a single distance chosen in advance, and reading a
 4002 pointwise band across all radii inflates the false-positive rate badly. And 999 simulations sets the
 4003 finest resolvable p-value at 0.001. Nineteen simulations, enough for the illustrative envelope
 4004 earlier in this chapter, cannot resolve anything below 0.05.

13.15 Published comparison

The study fitted the same class of model on the original 30 metre rasters, with covariates this reconstruction does not have, and screened them the same way (Murphy et al., 2026). Its likelihood-ratio results are:

Covariate	Published β	Published deviance	Published test
MPB-grey	-0.0233	0.47	0.492, not significant
MPB-red	-0.1461	14.85	0.001
Fire-high	-0.6430	211.64	0.001
Fire-low	0.5682	276.87	0.001
Fire-med	0.0188	0.34	0.563, not significant

Compare that against the screening table computed above. Fire-high, fire-low and MPB-red agree in sign and in rough magnitude, and fire-med is negligible in both. For a reconstruction working from point-extracted covariates rather than the source rasters, that is close agreement, and it is the check that gives you licence to teach from this dataset at all.

One covariate disagrees, and it disagrees loudly. Grey-stage attack comes out strongly positive here and near zero in the published model. This is the artefact promised earlier. Grey-stage is the rare class, under four per cent of records, so nearest-neighbour reconstruction paints it across whole neighbourhoods around the few seedlings that carry it, and the model reads that inflated footprint as a regeneration signal. The cluster-corrected standard error is what rescues the result: under the correct variance the grey-stage coefficient is not significant, which is the published conclusion.

That covariate has a history worth knowing, because it shows the same failure occurring in real research and being caught. When the analysis was re-run during revision, the grey-stage layer was rebuilt, and a value-threshold error entered that briefly made the covariate significant. The error was found, the layer corrected against an independent red-stage reference, and the affected models refitted. The corrected coefficient returns to non-significant, where it had been in the originally reviewed model, and the study's conclusions about red-stage attack and fire severity are unchanged by it.

Two lessons sit in that history and both outrank any coefficient. A rare binary covariate is the most fragile thing in a spatial model, because a small error in its extent is a large error in its proportion.

4029 And a result that appears only under an assumption you have not tested is not a result. Here the
4030 assumption was independence, and the test was the variance comparison in the previous section.

4031 *Model interrogation*

4032 Point-process models reach verification through claims rather than through a prescribed method.
4033 A project asserting successful regeneration, adequate stocking distribution, or a disturbance
4034 attribution is making a spatial-statistical claim, and the standards require its uncertainty to be
4035 quantified honestly even where they do not name the technique.

4036 Four questions do most of the work, in order of how often they find something.

4037 First, what variance estimator was used, and does it allow for spatial dependence? This is the
4038 question that exposed the near order-of-magnitude error above. An analysis reporting Poisson or
4039 ordinary least squares standard errors on clustered spatial data is overstating its confidence, and
4040 by how much is not knowable without refitting.

4041 Second, where did the covariate surfaces come from? Extracted at sample points and interpolated
4042 is not the same as measured on a grid, and rare classes suffer most. Ask for the extent of every
4043 binary covariate as an area, not as a coefficient.

4044 Third, was the null hypothesis the right one? Rejecting complete spatial randomness
4045 demonstrates almost nothing in a landscape with any topography at all. The informative test is
4046 against a fitted trend, not against uniformity.

4047 Fourth, is the simulation envelope global or pointwise, and how many simulations? A pointwise
4048 envelope read across a range of distances is not a valid test, and a p-value cannot be finer than
4049 one over the number of simulations plus one.

4050 None of this requires the verifier to refit the model. Each question is answerable from a methods
4051 description, and a methods description that cannot answer them is itself the finding.

4052 **13.16 Exercises**

- 4053 1. Build the ppp for plot 1 instead of plot 3 and compute its Clark-Evans ratio. Is the regeneration there clustered,
4054 random, or regular, and how does it compare with plot 3?
- 4055 2. Map the intensity surface with a larger smoothing bandwidth ($\sigma = 20$) and a smaller one ($\sigma = 4$). How
4056 does the choice change what you see, and what does that imply about reporting a single heat map?
- 4057 3. Refit the kppm with `clusters = "MatClust"` instead of "Thomas", which scatters offspring uniformly in a disc
4058 rather than by a Gaussian kernel. How much do the trend coefficients move, and how much do the cluster-
4059 corrected standard errors move? Which of the two layers is the more robust to the choice?

4. Drop distance from the trend and refit. Watch what happens to the fitted cluster scale. Explain why removing a covariate that explains large-scale variation makes the cluster process look stronger.
5. Rerun the final envelope with `global = FALSE` and count how many radii fall outside the band. Explain why that count cannot be read as a test.
6. (*Applied.*) Detected clearings in a monitoring image are also a point pattern. Explain how Ripley's K could distinguish scattered natural tree-fall from an organised, clustered harvest front, and why that distinction matters for attributing disturbance.

Appendix A — Statistical Primers

The chapters assume you can read a standard error, combine two uncertainties, and say what a Monte Carlo simulation is doing. If any of that has gone rusty, this appendix is where to repair it. It is deliberately short and it teaches only what the book actually uses. Nothing here is a substitute for a statistics course, and nothing in the chapters depends on reading it first.

A.1 Spread and intervals

Start with a distinction that inventory reports get wrong more often than any other. The standard deviation and the standard error are not the same quantity and they answer different questions.

The standard deviation describes the forest. It measures how much individual observations differ from one another. A stand of uniform planted spruce has a small standard deviation; a mixed natural stand with veterans and saplings has a large one. Adding plots does not shrink it, because it is a property of the thing you are measuring, not of your effort in measuring it.

The standard error describes your sample. It measures how far your sample mean is likely to sit from the true population mean. It shrinks as you measure more, because more measurement genuinely does pin the mean down better:

$$SE = \frac{s}{\sqrt{n}}$$

The square root is the part worth internalising. To halve a standard error you must quadruple the sample. That single fact governs the economics of every inventory design in this book, and it is why the cheap uncertainty reductions are almost never “measure more plots”.

Take the stem data and divide the plot into quadrats, so each quadrat becomes one sample of carbon density:

```
stems <- read.csv("data/scbi_stems.csv")
stems$agb_kg <- exp(-2.48 + 2.4835 * log(stems$dbh))
stems$carbon_Mg <- 0.47 * stems$agb_kg / 1000
```

```

4091 stems$quadrat <- paste(cut(stems$gx, 8), cut(stems$gy, 12))
4092 quadrat_area <- 25.6 / (8 * 12)
4093
4094
4095 qd <- aggregate(carbon_Mg ~ quadrat, data = stems, FUN = sum)
4096 qd$carbon_ha <- qd$carbon_Mg / quadrat_area
4097
4098 n <- nrow(qd)
4099 m <- mean(qd$carbon_ha)
4100 s <- sd(qd$carbon_ha)
4101 se <- s / sqrt(n)
4102 round(c(n = n, mean = m, sd = s, se = se), 2)
4103 #>      n      mean      sd      se
4104 #> 96.00 112.15 23.86 2.44

```

4105 The standard deviation is an order of magnitude larger than the standard error, which is exactly
 4106 as it should be with 96 quadrats. Quadrupling the sample would move the standard error to 1.22
 4107 and leave the standard deviation where it is.

4108 A confidence interval turns the standard error into a statement you can report. It is the mean plus
 4109 or minus some multiple of the standard error, and the multiple is set by how confident you want
 4110 to be:

```

4111 ci <- function(mean, se, level) {
4112   z <- qnorm(1 - (1 - level) / 2)
4113   hw <- z * se
4114   c(z = z, half_width = hw, rel_pct = 100 * hw / mean,
4115     lower = mean - hw, upper = mean + hw)
4116 }
4117 round(rbind(`90%` = ci(m, se, 0.90),
4118             `95%` = ci(m, se, 0.95)), 2)
4119 #>      z half_width rel_pct lower upper
4120 #> 90% 1.64      4.01    3.57 108.14 116.15
4121 #> 95% 1.96      4.77    4.26 107.37 116.92

```

4122 The crediting standards work at 90 per cent, so the multiplier you will meet most often is 1.645.
 4123 What they consume is the half-width expressed as a percentage of the mean, the `rel_pct` column
 4124 above. That is the uncertainty figure a deduction schedule is applied to, and here it is large enough
 4125 to trigger one under every schedule in the uncertainty chapter. Sampling error alone, on a well-
 4126 mapped 25.6 hectare plot, is not a small problem.

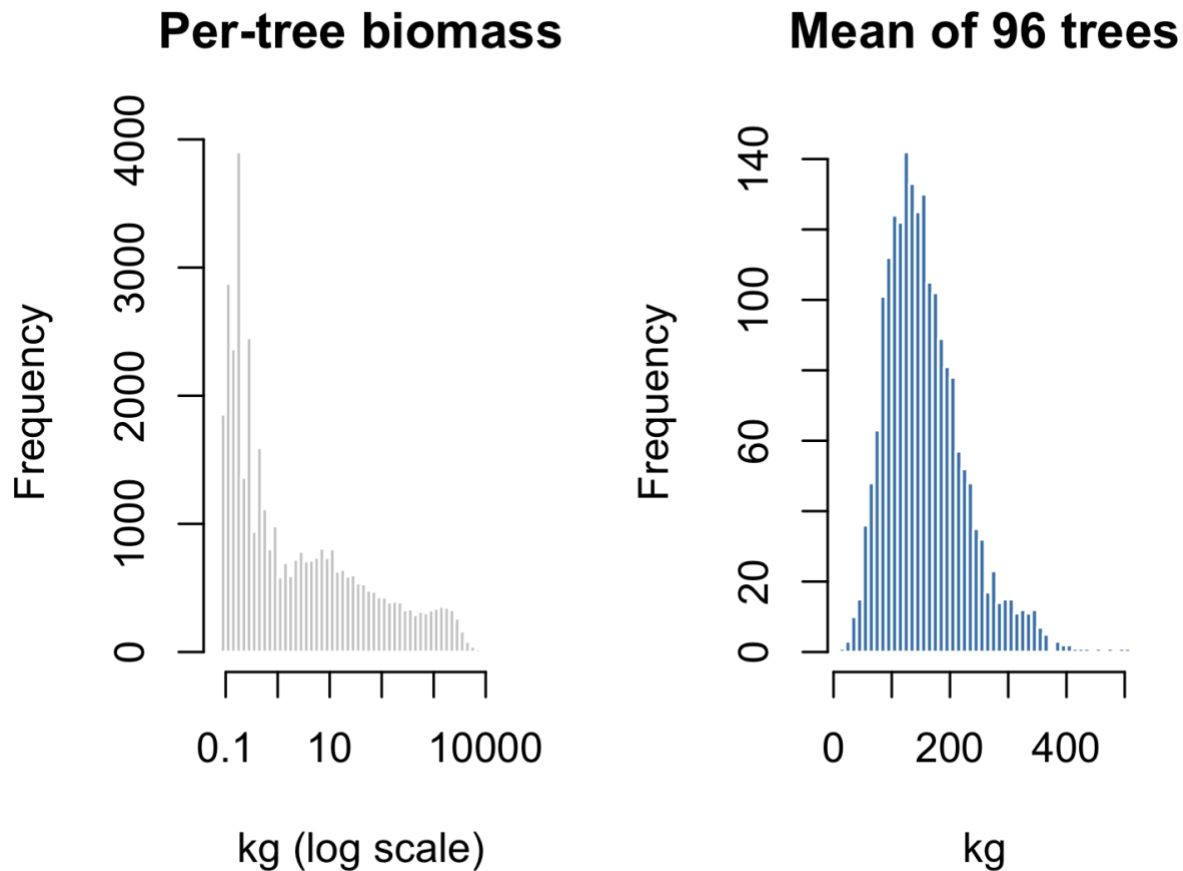
4127 Two cautions. First, the interval is a statement about the mean, not about individual trees. An
 4128 interval that contains the stand mean will not contain most of the trees, and a prediction interval
 4129 for a single tree is far wider. Second, the interval assumes the sample mean is approximately
 4130 normally distributed, which is a much weaker assumption than assuming the data are.

4131 That second point deserves a demonstration, because forest data are about as far from normal
 4132 as ecological data get. Per-tree biomass is violently right-skewed, and yet the mean of a sample
 4133 of them is not:

```

4134 set.seed(1)
4135 draws <- replicate(2000, mean(sample(stems$agb_kg, 96, replace = TRUE)))
4136
4137 par(mfrow = c(1, 2))
4138 # on a linear axis every tree but the largest few falls in one bar, so the
4139 # left panel is drawn on a log axis and labelled back in kilograms
4140 hist(log10(stems$agb_kg), breaks = 40, col = "grey80", border = "white",
4141      main = "Per-tree biomass", xlab = "kg (log scale)", xaxt = "n")
4142 axis(1, at = -1:4, labels = c("0.1", "1", "10", "100", "1000", "10000"))
4143 hist(draws, breaks = 40, col = "steelblue", border = "white",
4144      main = "Mean of 96 trees", xlab = "kg")

```



```

4145
4146 round(c(tree_mean = mean(stems$agb_kg),
4147         tree_median = median(stems$agb_kg),
4148         tree_max = max(stems$agb_kg),
4149         mean_of_96 = mean(draws),
4150         sd_of_means = sd(draws)), 1)
4151 #>   tree_mean tree_median tree_max mean_of_96 sd_of_means
4152 #>    158.6      0.7    21634.8    157.9      68.3

```

4153 The left panel is the forest: a median of 0.7 kg against a maximum of 2.1635^4 kg, with 74 per
 4154 cent of all the biomass sitting in the largest five per cent of the stems. The right panel is a sampling
 4155 distribution, and it is close to symmetric. This is the central limit theorem doing the work that lets
 4156 you put a normal-theory interval around a mean computed from wildly non-normal data. It is also

the reason large trees dominate inventory design: the skew does not stop the mathematics working, but it does mean a sample that misses the big stems misses most of the carbon.

A.2 Combining uncertainties

Carbon estimates are built by multiplying uncertain things together. Area times carbon density gives stock. Activity data times an emission factor gives emissions. Each input carries its own error, and you need the error of the result.

When the inputs are independent, relative errors combine in quadrature: square them, add, take the square root.

$$U_{\text{total}} = \sqrt{U_1^2 + U_2^2 + \dots + U_k^2}$$

The reason is not arbitrary. Errors have signs. Sometimes the area is high while the factor is low, and the two partly cancel. Adding the percentages assumes every error goes the wrong way at the same time, which is possible but not probable, and an estimate built on that assumption is not conservative so much as wrong:

```
quad <- function(...) sqrt(sum(c(...)^2))
c(quadrature = quad(8, 12), naive_sum = 8 + 12)
#> quadrature naive_sum
#> 14.42221 20.00000
```

Now notice what happens when you try to improve either input:

```
rbind(
  `as measured`      = c(ad = 8, ef = 12, total = quad(8, 12)),
  `halve activity data` = c(ad = 4, ef = 12, total = quad(4, 12)),
  `halve factor`     = c(ad = 8, ef = 6, total = quad(8, 6))
)
#>           ad ef total
#> as measured      8 12 14.42221
#> halve activity data 4 12 12.64911
#> halve factor      8 6 10.00000
```

Halving the smaller term buys you 1.77 percentage points. Halving the larger buys 4.42. Quadrature is dominated by its largest term, and that is the single most useful piece of guidance in uncertainty management: find the biggest contributor and spend the money there. Refining anything else is close to wasted effort.

The formula above applies to products. Sums behave differently: when you add carbon pools, their absolute errors combine in quadrature, so the relative error of the total is a magnitude-weighted quadratic mean in which a large pool with modest relative error can outweigh a small

pool with a terrible one. That is the form the ACR improved forest management equation takes, and the uncertainty chapter works it through against the standard.

Two assumptions are buried in all of this. The inputs must be independent, and if they are correlated, quadrature understates the total. And the combination must be roughly linear over the range of the errors. When either fails, stop propagating algebraically and simulate.

A.3 Monte Carlo methods

Monte Carlo simulation replaces the algebra with brute force. Draw each uncertain input from its distribution, recompute the whole estimate, and repeat until the spread of results is itself stable. The spread is the answer.

The method is not an approximation to quadrature; quadrature is an approximation to it. Where the two are both valid they agree, and it is worth seeing that before trusting either:

```
set.seed(42)
N <- 100000
area <- rnorm(N, 1000, 80) # 8% relative standard error
factor <- rnorm(N, 150, 18) # 12% relative standard error
total <- area * factor

round(c(monte_carlo = 100 * sd(total) / mean(total),
        quadrature = quad(8, 12)), 3)
#> monte_carlo quadrature
#> 14.481 14.422
```

They agree to within a few hundredths of a percentage point, which is the reassurance you want: when the function is a simple product of independent normals, the closed form is correct and very much cheaper. Reach for simulation when it is not.

Allometry is the standard case where it is not. Biomass is exponential in the fitted coefficient, so the same coefficient uncertainty produces a different result depending on the tree it is applied to:

```
set.seed(42)
b <- rnorm(N, 2.4835, 0.05) # uncertainty in the allometric exponent

spread <- function(D) {
  agb <- exp(-2.48 + b * log(D))
  c(dbh_cm = D,
    rel_sd_pct = 100 * sd(agb) / mean(agb),
    mean_over_median = mean(agb) / median(agb))
}

round(t(sapply(c(10, 30, 60, 100), spread)), 3)
#> dbh_cm rel_sd_pct mean_over_median
#> [1,] 10 11.586 1.006
#> [2,] 30 17.179 1.014
#> [3,] 60 20.745 1.021
#> [4,] 100 23.397 1.027
```

Two lessons fall out. The relative uncertainty grows with diameter, so quoting a single model uncertainty percentage for a whole inventory is meaningless without saying which trees it applies

to. And the mean exceeds the median at every diameter, because exponentiating a symmetric distribution produces a skewed one. That skew is the same effect the Baskerville correction addresses in the allometry chapter, arriving here from the other direction.

The remaining question is how many iterations to run. Monte Carlo error falls with the square root of the iteration count, the same rule that governs sample size:

```
set.seed(7)
iters <- c(100, 1000, 10000, 100000)
converge <- sapply(iters, function(k) sd(replicate(200, mean(rnorm(k, 150, 18)))))
round(rbind(iterations = iters, sd_of_result = converge), 4)
#>      [,1]      [,2]      [,3]      [,4]
#> iterations  100.0000 1000.0000 1.000e+04 1.00e+05
#> sd_of_result  1.8605   0.5549 1.864e-01 5.45e-02
```

Each hundredfold increase in iterations shrinks the scatter of the answer by roughly an order of magnitude, which is the square-root rule again. This is why ART-TREES requires that Monte Carlo simulations use the 90 per cent confidence interval and a simulation n of 10,000. Below that the reported uncertainty is itself noticeably uncertain, and two honest analysts running the same model would report materially different deductions from the same data.

One operational point matters more than any of the statistics. Set the seed. A verifier must be able to run your script and obtain your number, and an unseeded simulation cannot be checked, only re-estimated. Every simulation in this book is seeded for that reason.

A.4 Applications

Idea	First used in
Standard error, confidence intervals	the uncertainty chapter, sampling error
Skew and the sampling distribution	the allometry chapter, model fitting
Quadrature	the uncertainty chapter, combining sources
Magnitude-weighted quadratic mean	the uncertainty chapter, ACR deduction
Monte Carlo	the uncertainty chapter, propagation
Seeding and reproducibility	throughout

Appendix B — Reproducible Version Control

B.1 Beyond analysis

A defensible result is only half the job. The other half is the discipline around it: making the work reproducible, governing the data to a recognised standard, and delivering credits into a registry that will accept them. This appendix sketches that surrounding practice. Its code is illustrative and is not run at render time.

B.2 Reproducible runtimes

The analysis in this book runs today; the test is whether it runs in three years, on another machine, for a reviewer who was not there. Two habits carry most of that weight. Pin your package versions so an update cannot silently change a result, and script the whole pipeline end to end so there are no undocumented manual steps.

```
# Pin the project's package library to exact versions
renv::init()      # snapshot the library
renv::snapshot()  # write renv.lock, committed with the project
renv::restore()   # a reviewer rebuilds the exact environment
```

A single entry point that runs every stage in order is the other half. When the raw data change, one command regenerates every figure and number, and nothing depends on remembering which script to run first.

```
# run.R, the one command that rebuilds everything
source("01-ingest.R")    # read and validate raw survey data
source("02-process.R")   # projection, extraction, modelling
source("03-report.R")    # render outputs with quarto::quarto_render()
```

B.3 Data protocols

Every number inherits the quality of the survey that produced it. Standardised digital capture, enforced schemas, and a logged accuracy estimate on every GPS fix are what let an auditor trust a plot table at all. The governing reference is the geospatial data-quality framework of the ISO 19100 series, which defines the quality elements, completeness, logical consistency, positional accuracy, thematic accuracy, an auditor should expect to see documented. In practice this means a QA/QC step that checks geometry validity, coordinate ranges, attribute domains, and duplicate records before any analysis begins.

```
# Minimal ingest QA gate
stopifnot(
  all(sf::st_is_valid(layer)),          # geometry validity
  all(sf::st_coordinates(layer)[, 1] > -180), # coordinates in range
  !anyDuplicated(layer$plot_id)         # no duplicate keys
)
```

B.4 Carbon Registries

ART-TREES, Verra (VCS), and ACR each publish methodologies, uncertainty and accuracy requirements, and validation and verification procedures that a project must meet before issuance. The analytical chapters of this book map directly onto their requirements: the area check onto boundary and area rules, the accuracy assessment onto the mandatory confusion matrix, the Monte Carlo onto the uncertainty deduction, the fire-emissions chain onto the reporting equations. Knowing which registry a project targets, and reading its methodology as the specification the analysis must satisfy, is the final piece of the auditor's craft.

B.5 Additional resources

The workflows referenced here, standard operating procedures, demonstration repositories with `renv` and version control, safeguards and QA/QC templates, and data-service automation, are where the habits of this book become an operational system. The analysis is the visible part; the reproducibility, governance, and registry alignment are what make it hold up under scrutiny.

Appendix C — References

- Appel, M., & Pebesma, E. (2019). On-demand processing of data cubes from satellite image collections with the gdalcubes library. *Data*, 4(3), 92. <https://doi.org/10.3390/data4030092>
- Architecture for REDD+ Transactions. (2021). *The REDD+ environmental excellence standard (TREES), version 2.0*. ART Secretariat, Winrock International.
- Baddeley, A., Rubak, E., & Turner, R. (2015). *Spatial point patterns: Methodology and applications with R*. Chapman; Hall/CRC.
- Baskerville, G. L. (1972). Use of logarithmic regression in the estimation of plant biomass. *Canadian Journal of Forest Research*, 2(1), 49–53. <https://doi.org/10.1139/x72-009>
- Batjes, N. H. (2011). Soil organic carbon stocks under native vegetation: Revised estimates for use with the simple assessment option of the carbon benefits project system. *Agriculture, Ecosystems and Environment*, 142(3–4), 365–373. <https://doi.org/10.1016/j.agee.2011.06.007>
- Bivand, R. S., Pebesma, E., & Gómez-Rubio, V. (2013). *Applied spatial data analysis with R* (2nd ed.). Springer.
- Chave, J., Réjou-Méchain, M., Búrquez, A., Chidumayo, E., Colgan, M. S., Delitti, W. B. C., Duque, A., Eid, T., Fearnside, P. M., Goodman, R. C., Henry, M., Martínez-Yrizar, A., Mugasha, W. A., Muller-Landau, H. C., Mencuccini, M., Nelson, B. W., Ngomanda, A., Nogueira, E. M., Ortiz-Malavassi, E., ... Vieilledent, G. (2014). Improved allometric models to estimate the aboveground biomass of tropical trees. *Global Change Biology*, 20(10), 3177–3190. <https://doi.org/10.1111/gcb.12629>
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6(1), 3–73.
- Dalponte, M., & Coomes, D. A. (2016). Tree-centric mapping of forest carbon density from airborne laser scanning and hyperspectral data. *Methods in Ecology and Evolution*, 7(10), 1236–1245.
- Hansen, M. C., Potapov, P. V., Moore, R., Hancher, M., Turubanova, S. A., Tyukavina, A., Thau, D., Stehman, S. V., Goetz, S. J., Loveland, T. R., Kommareddy, A., Egorov, A., Chini, L., Justice, C. O., & Townshend, J. R. G. (2013). High-resolution global maps of 21st-century forest cover change. *Science*, 342(6160), 850–853. <https://doi.org/10.1126/science.1244693>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.
- Hijmans, R. J. (2024). *Terra: Spatial data analysis*. <https://CRAN.R-project.org/package=terra>
- IPCC. (2006). *2006 IPCC guidelines for national greenhouse gas inventories, volume 4: Agriculture, forestry and other land use*. Institute for Global Environmental Strategies.
- IPCC. (2019). *2019 refinement to the 2006 IPCC guidelines for national greenhouse gas inventories, volume 4: Agriculture, forestry and other land use* (E. Calvo Buendia, K.

4345 Tanabe, A. Kranjc, J. Baasansuren, M. Fukuda, S. Ngarize, A. Osako, Y. Pyrozhenko, P.
 4346 Shermanau, & S. Federici, Eds.). IPCC.

4347 Jenkins, J. C., Chojnacky, D. C., Heath, L. S., & Birdsey, R. A. (2003). National-scale biomass
 4348 estimators for united states tree species. *Forest Science*, 49(1), 12–35.
 4349 <https://doi.org/10.1093/forestscience/49.1.12>

4350 Lovelace, R., Nowosad, J., & Muenchow, J. (2019). *Geocomputation with r*. Chapman;
 4351 Hall/CRC.

4352 Murphy, S., Leslie, A., Wilson, J., & Banks, L. K. (2026). Spatial patterns of conifer regeneration
 4353 following mountain pine beetle outbreak (*dendroctonus ponderosae*) and mixed-severity
 4354 wildfire: A point process model of disturbance interactions in the southern selkirk
 4355 mountains, british columbia. *Forest Ecology and Management*, 618, 123985.
 4356 <https://doi.org/10.1016/j.foreco.2026.123985>

4357 Neeff, T. (2021). How uncertainty in emissions estimates affects the integrity of carbon credits.
 4358 *Climatic Change*, 166(1–2), 26. <https://doi.org/10.1007/s10584-021-03123-y>

4359 Olofsson, P., Foody, G. M., Herold, M., Stehman, S. V., Woodcock, C. E., & Wulder, M. A.
 4360 (2014). Good practices for estimating area and assessing accuracy of land change.
 4361 *Remote Sensing of Environment*, 148, 42–57. <https://doi.org/10.1016/j.rse.2014.02.015>

4362 Pebesma, E. (2018). Simple features for r: Standardized support for spatial vector data. *The R*
 4363 *Journal*, 10(1), 439–446. <https://doi.org/10.32614/RJ-2018-009>

4364 Pebesma, E., & Bivand, R. (2023). *Spatial data science: With applications in r*. Chapman;
 4365 Hall/CRC. <https://doi.org/10.1201/9780429459016>

4366 Plowright, A. (2025). *ForestTools: Tools for analyzing remote sensing forest data*. [https://CRAN.R-](https://CRAN.R-project.org/package=ForestTools)
 4367 [project.org/package=ForestTools](https://CRAN.R-project.org/package=ForestTools)

4368 Roussel, J.-R., & Auty, D. (2019). *lidR: Airborne LiDAR data manipulation and visualization for*
 4369 *forestry applications. R package version 2.1.4.*

4370 Roussel, J.-R., Auty, D., Coops, N. C., Tompalski, P., Goodbody, T. R., Meador, A. S., Bourdon,
 4371 J.-F., Boissieu, F. de, & Achim, A. (2020). lidR: An r package for analysis of airborne laser
 4372 scanning (ALS) data. *Remote Sensing of Environment*, 251, 112061.

4373 Simões, R., Camara, G., Queiroz, G., Souza, F., Andrade, P. R., Santos, L., Carvalho, A., &
 4374 Ferreira, K. (2021). Satellite image time series analysis for big earth observation data.
 4375 *Remote Sensing*, 13(13), 2428. <https://doi.org/10.3390/rs13132428>

4376 Verra. (2024). *Module J-ADB-UD: Jurisdictional activity data baseline for unplanned*
 4377 *deforestation, version 1.0*. Verra.

4378 Waagepetersen, R. P. (2007). An estimating function approach to inference for inhomogeneous
 4379 Neyman-Scott processes. *Biometrics*, 63(1), 252–258. [https://doi.org/10.1111/j.1541-](https://doi.org/10.1111/j.1541-0420.2006.00667.x)
 4380 [0420.2006.00667.x](https://doi.org/10.1111/j.1541-0420.2006.00667.x)

4381 Zhang, W., Qi, J., Wan, P., Wang, H., Xie, D., Wang, X., & Yan, G. (2016). An easy-to-use
 4382 airborne LiDAR data filtering method based on cloth simulation. *Remote Sensing*, 8(6),
 4383 501.